

# FEM2D Q4 二维有限元求解器

完整使用文档（理论、代码与 API 参考）

版本 9.28.0

2026-08-07

## 目录

|          |                         |          |
|----------|-------------------------|----------|
| <b>1</b> | <b>软件介绍</b>             | <b>2</b> |
| 1.1      | 项目定位                    | 2        |
| 1.2      | 功能总览                    | 2        |
| 1.3      | 技术特征                    | 3        |
| <b>2</b> | <b>安装与快速开始</b>          | <b>3</b> |
| 2.1      | 方式一：便携包（免安装，推荐演示用）      | 3        |
| 2.2      | 方式二：源码方式（开发/研究用）        | 3        |
| 2.3      | 完整演示流程详解                | 4        |
| 2.4      | 端到端演示走读：demo 背后发生了什么    | 4        |
| 2.5      | 随附演示模型                  | 6        |
| 2.6      | 便携包目录结构                 | 6        |
| 2.7      | Gmsh .geo 语法详解（重点）      | 8        |
| 2.7.1    | 脚本基础：注释、变量、表达式与循环       | 8        |
| 2.7.2    | 几何命令全解                  | 9        |
| 2.7.3    | 进阶建模命令                  | 9        |
| 2.7.4    | 网格控制命令                  | 10       |
| 2.7.5    | 完整实例走读：demo_complex.geo | 11       |
| 2.7.6    | 参数化建模模板：改变量 = 换模型       | 12       |
| 2.7.7    | 常见错误与调试                 | 13       |
| 2.8      | 换电脑使用流程（新机器首次运行）        | 13       |
| 2.8.1    | 第一步：判断自己走哪条路线           | 13       |
| 2.8.2    | 便携包路线：5 步跑通             | 14       |
| 2.8.3    | 源码路线：5 步跑通              | 14       |
| 2.8.4    | 故障速查表                   | 14       |
| 2.8.5    | 自检命令说明                  | 15       |

|          |                                |           |
|----------|--------------------------------|-----------|
| <b>3</b> | <b>命令行用法</b>                   | <b>15</b> |
| 3.1      | 入口                             | 15        |
| 3.2      | 主要参数                           | 15        |
| 3.3      | 参数细节                           | 16        |
| 3.4      | 示例                             | 16        |
| 3.5      | spec 文件语法                      | 17        |
| 3.6      | .geo 中的 @FEM 配置注释              | 17        |
| 3.7      | .txt 中文几何                      | 17        |
| 3.8      | .msh 网格文件详解 (Gmsh 原生格式)        | 17        |
| <b>4</b> | <b>理论基础</b>                    | <b>19</b> |
| 4.1      | 平面线弹性问题的弱形式                    | 19        |
| 4.1.1    | 三大方程 (一行回顾)                    | 19        |
| 4.1.2    | 虚功原理 (弱形式) —— 有限元离散的起点         | 20        |
| 4.2      | 位移离散与单元刚度                      | 20        |
| 4.2.1    | 位移插值与 B 矩阵                     | 20        |
| 4.2.2    | 数值积分 (高斯求积)                    | 20        |
| 4.3      | 单元族                            | 21        |
| 4.3.1    | CST —— 常应变三角形                  | 21        |
| 4.3.2    | Q4 —— 双线性协调四边形 (本章重点)          | 21        |
| 4.3.3    | Q4R —— 单点积分 + 沙漏控制 (本章重点)      | 23        |
| 4.3.4    | Q4I —— QM6 非协调单元               | 26        |
| 4.4      | 边界条件施加                         | 26        |
| 4.4.1    | 消去法 (elimination) —— 默认        | 26        |
| 4.4.2    | 罚方法 (penalty)                  | 26        |
| 4.4.3    | 刚体模态检查                         | 27        |
| 4.5      | 应力计算与恢复                        | 27        |
| 4.5.1    | 单元应力                           | 27        |
| 4.5.2    | 节点平均 (nodal simple / weighted) | 27        |
| 4.5.3    | SPR —— 超收敛点恢复                  | 27        |
| 4.5.4    | L2 投影                          | 27        |
| 4.6      | 误差估计                           | 28        |
| 4.6.1    | Z2 能量误差估计器 (简要)                | 28        |
| 4.6.2    | 应力跳跃 —— 牵引跳跃                   | 28        |
| 4.7      | 分片试验与收敛性                       | 28        |
| 4.8      | 网格质量指标                         | 29        |
| 4.9      | 载荷的有限元离散                       | 29        |
| 4.9.1    | 集中力                            | 29        |
| 4.9.2    | 边界分布面力与压力                      | 30        |
| 4.9.3    | 体力                             | 31        |
| 4.10     | 等应力带法 (isoband)                | 31        |
| 4.11     | 一个完整的手算示例: CST 单元拉伸            | 31        |

|          |                                  |           |
|----------|----------------------------------|-----------|
| 4.12     | 手算示例：Q4 协调单元（ $2 \times 2$ 高斯积分） | 32        |
| 4.13     | 手算示例：Q4R 单点积分与沙漏控制               | 34        |
| <b>5</b> | <b>代码结构</b>                      | <b>36</b> |
| 5.1      | 顶层结构                             | 36        |
| 5.2      | 五阶段数据流                           | 37        |
| 5.3      | 模块职责表                            | 37        |
| 5.3.1    | element/（内核注册协议——行为冻结区）          | 37        |
| 5.3.2    | boundary/（识别管线——识别算法冻结，只加插件/标签）  | 38        |
| 5.3.3    | 核心数值（冻结区）                        | 38        |
| 5.3.4    | 后处理                              | 39        |
| 5.3.5    | 验证与辅助                            | 39        |
| 5.4      | solver.solve 内部流程（Bathe §4.2）    | 39        |
| 5.5      | 冻结区与契约                           | 40        |
| 5.6      | 错误处理体系                           | 40        |
| <b>6</b> | <b>代码深度解析</b>                    | <b>41</b> |
| 6.1      | 设计哲学：为什么代码这样安排                   | 41        |
| 6.2      | 单元内核协议（element/base.py）          | 41        |
| 6.2.1    | 为什么需要协议层                         | 41        |
| 6.2.2    | 协议的核心方法                          | 42        |
| 6.2.3    | 注册表：import 顺序的防呆                 | 43        |
| 6.2.4    | Jacobian 检查：逐单元尺度容差              | 44        |
| 6.2.5    | 载荷求值统一校验                         | 44        |
| 6.3      | Q4 内核代码解析（element/q4.py）         | 45        |
| 6.3.1    | 形函数：教科书公式的直接翻译                   | 45        |
| 6.3.2    | B 矩阵：链式法则                        | 46        |
| 6.3.3    | 批量运动学：为什么缓存、为什么局部坐标              | 46        |
| 6.3.4    | 刚度批量组装：先加权再二次型                   | 47        |
| 6.3.5    | 单元级自检：单位分解 + 刚体模态                | 48        |
| 6.4      | Q4R 沙漏控制代码解析（element/q4r.py）     | 48        |
| 6.4.1    | 仿射补：余因子技巧                        | 49        |
| 6.4.2    | 稳定刚度：单点积分 + 仅剪切参考刚度              | 50        |
| 6.4.3    | 批量路径与材料指纹                        | 51        |
| 6.4.4    | 沙漏能量监控：负能告警而非截零                  | 51        |
| 6.4.5    | Q4R 自检：3 个零模 + 仿射泄漏              | 52        |
| 6.5      | CST 与 Q4I 内核要点                   | 53        |
| 6.5.1    | CST（element/cst.py）：最简内核的完整形态    | 53        |
| 6.5.2    | Q4I（element/q4i.py）：静力凝聚         | 54        |
| 6.6      | 装配代码（assembly.py）                | 54        |
| 6.6.1    | 为什么有四条路径                         | 54        |
| 6.6.2    | 向量化路径：一次 COO 转换                  | 54        |

|       |                                                           |    |
|-------|-----------------------------------------------------------|----|
| 6.6.3 | 对称检查里的 NaN/Inf 守卫                                         | 55 |
| 6.7   | 边界拓扑识别代码 (boundary/)                                      | 56 |
| 6.7.1 | 总入口: detect() 的五步管线                                       | 56 |
| 6.7.2 | 闭环分解: 度检查 + 图遍历                                           | 57 |
| 6.7.3 | 校验与嵌套: 自交检测 + 洞深度                                         | 58 |
| 6.7.4 | 定向: 材料 CCW、洞 CW                                           | 59 |
| 6.7.5 | 插件识别体系: 首个非 None 胜出                                       | 59 |
| 6.7.6 | 语义优先级: 命名总入口                                              | 61 |
| 6.7.7 | 优缺点分析 (诚实评估)                                              | 62 |
| 6.7.8 | 拓扑识别的实战判读 (优缺点实证)                                         | 62 |
| 6.8   | 求解器主流程 (solver.py)                                        | 63 |
| 6.8.1 | 阶段化主流程                                                    | 63 |
| 6.8.2 | 奇异守卫: 秩警告转异常                                              | 64 |
| 6.8.3 | 平衡检查: 残差之外的语义防线                                           | 65 |
| 6.8.4 | 返回契约: 固定键集                                                | 65 |
| 6.9   | 关键数值模块代码解析 (patch_test / quality / error_est / visualize) | 65 |
| 6.9.1 | Z2 估计器的实现 (error_est.py)                                  | 66 |
| 6.9.2 | 网格质量评分的实现 (quality.py, 105 行)                             | 67 |
| 6.9.3 | 分片试验的实现 (patch_test.py)                                   | 68 |
| 6.9.4 | 等应力带法的实现 (visualize.py + runner.py)                       | 68 |
| 6.10  | 设计理由总结: 为什么代码这样安排                                         | 69 |
| 7     | API 参考                                                    | 70 |
| 7.1   | 全局约定                                                      | 70 |
| 7.2   | Mesh 类                                                    | 70 |
| 7.2.1 | 构造器                                                       | 70 |
| 7.2.2 | 节点索引 API (所有 nid/ni/nj 共用契约)                              | 71 |
| 7.2.3 | 结构修改与查询                                                   | 72 |
| 7.3   | 求解与 BC                                                    | 73 |
| 7.4   | 载荷                                                        | 73 |
| 7.5   | 应力与误差估计                                                   | 74 |
| 7.6   | 输入链                                                       | 75 |
| 7.7   | 材料与单元注册                                                   | 76 |
| 7.8   | 边界识别                                                      | 76 |
| 7.9   | 配置与质量                                                     | 77 |
| 7.10  | 装配与其他导出                                                   | 77 |
| 7.11  | Python 编程接口示例                                             | 78 |
| 7.12  | 缺口修复历史 (K1-K13)                                           | 80 |
| 8     | 验证与质量保障                                                   | 81 |
| 8.1   | 六层测试体系                                                    | 81 |
| 8.2   | 统一验收入口 verify_all                                         | 82 |

|        |                                                        |     |
|--------|--------------------------------------------------------|-----|
| 8.3    | 契约测试                                                   | 82  |
| 8.4    | 解析解验证 (E1/E2/E3)                                       | 82  |
| 8.4.1  | E1 悬臂梁 (Euler–Bernoulli 梁理论)                           | 82  |
| 8.4.2  | E2 Kirsch 圆孔 (应力集中)                                    | 83  |
| 8.4.3  | E3 Z2 效应指数 $\theta$                                    | 83  |
| 8.5    | 性能基线                                                   | 83  |
| 8.5.1  | 求解器选型建议                                                | 84  |
| 8.5.2  | 微优化记录 (2026-08-04, 包 4)                                | 85  |
| 8.6    | CI 流水线                                                 | 85  |
| 8.7    | 发布检查清单                                                 | 85  |
| 8.8    | 独立审查历史                                                 | 86  |
| 9      | 测试代码深度解析                                               | 86  |
| 9.1    | 测试的组织: 143 个文件怎么排                                      | 86  |
| 9.2    | 金标准逐位锁: test_solve_refactor_lock.py 逐行                 | 86  |
| 9.2.1  | 文件头 docstring 就是设计文档                                   | 87  |
| 9.2.2  | 容差常量的语义                                                | 87  |
| 9.2.3  | 环境敏感行的剥离: 结构锁死、数值容差                                    | 88  |
| 9.2.4  | 递归数值比较: 键内尺度 atol                                      | 89  |
| 9.2.5  | 九个锁定场景                                                 | 91  |
| 9.2.6  | 主测试与金标准结构                                              | 92  |
| 9.2.7  | 判别性测试: 锁自身的有效性证明                                       | 93  |
| 9.3    | 判别性测试原则: 每修复必附”放回旧实现必失败”                               | 95  |
| 9.4    | 漂移门: regression_compare.py                             | 95  |
| 9.5    | 探针: audit_contract_probe.py                            | 95  |
| 9.6    | fuzz: 固定种子 500                                         | 95  |
| 9.7    | 分支锁: test_{solver,runner,stress,error_est}_branches.py | 96  |
| 9.8    | 解析解验证测试 (E1/E2/E3)                                     | 96  |
| 9.9    | 无 gmsh 纪律                                              | 96  |
| 9.10   | 一个测试文件的逐行走读: test_boundary_adaptive.py                 | 96  |
| 9.10.1 | 文件头: docstring 就是设计契约                                  | 97  |
| 9.10.2 | 辅助构建器: 参数化几何的工厂                                        | 97  |
| 9.10.3 | 第一个测试: 无 CAD 语义时多边形退化为直线段                              | 97  |
| 9.10.4 | 对照组: 细分弦线必须合并、真六边形必须保留                                 | 98  |
| 9.10.5 | 闭环分类: 半径反解到 1e-12                                      | 98  |
| 9.10.6 | 圆角矩形: G1 边界用例                                          | 99  |
| 9.10.7 | 尺度感知: 从 1e-20 到 1e12 的环判定                              | 99  |
| 9.10.8 | 环定向: 鞋带公式分内外                                           | 99  |
| 9.11   | 测试如何保护重构 (总结)                                          | 100 |
| 10     | 注意事项                                                   | 100 |
| 10.1   | 常见错误消息速查                                               | 100 |

|                      |            |
|----------------------|------------|
| <b>11 可扩展性</b>       | <b>102</b> |
| 11.1 扩展点总览：改哪些文件、挂在哪 | 102        |
| 11.2 各扩展点的最小改动步骤     | 103        |
| 11.3 扩展开发流程          | 104        |
| <b>12 可维护性</b>       | <b>104</b> |
| <b>13 未来展望</b>       | <b>105</b> |
| <b>14 版本与验证记录</b>    | <b>106</b> |
| <b>A 常见问题（FAQ）</b>   | <b>106</b> |
| A.1 建模与输入            | 106        |
| A.2 单元选择             | 107        |
| A.3 载荷与边界            | 107        |
| A.4 结果解读             | 107        |
| A.5 开发与验证            | 108        |
| <b>B 术语表</b>         | <b>108</b> |
| <b>C 公式—教材对照表</b>    | <b>109</b> |
| <b>D 项目历史大事记</b>     | <b>109</b> |
| <b>E 参考文献</b>        | <b>111</b> |

# 1 软件介绍

## 1.1 项目定位

FEM2D Q4 是一个二维线弹性有限元求解器，参照 Bathe 《Finite Element Procedures》（第 2 版）逐公式实现，用于教学与研究目的。它覆盖从几何建模、网格生成、求解到误差估计与后处理的完整链路，是一个**公式完整、行为可验证、面向教学**的数值软件——每条核心公式都能在 Bathe 教材中找到出处，每个数值行为都有测试锁定。

- **语言**：Python 3.9+（开发与验证于 3.13），纯 Python 实现，免编译
- **依赖**：numpy / scipy / matplotlib（运行时）；gmsh（网格生成，可选）；pytest（测试）
- **单元**：CST（常应变三角形）、Q4（双线性四边形）、Q4R（单点积分 + 沙漏控制）、Q4I（QM6 非协调单元）
- **规模**：约 5.5 万行；143 个测试文件、1649 个测试函数、1812 项收集测试；覆盖率 98.3%
- **精度**：漂移门 6 个基线值跨平台逐位一致（相对误差 0.000e+00）
- **版本**：9.28.0（main = e397f64）

## 1.2 功能总览

| 类别    | 能力                                                  |
|-------|-----------------------------------------------------|
| 问题类型  | 平面应力 / 平面应变（按单元码自动判定，可显式覆盖）                         |
| 载荷    | 集中力、边界分布面力（线/抛物线/圆弧分布）、法向压力、体力（含空间函数表达式）            |
| 约束    | 消去法（elimination，默认）、罚方法（penalty）；任意 DOF 指定位移        |
| 求解    | 稀疏直接求解（SuperLU）、Jacobi-PCG 迭代（大型模型自动切换）、CG 与 ILU 可选 |
| 应力后处理 | 单元应力、节点平均、L2 投影、SPR 超收敛恢复、主应力 / von Mises / 最大剪应力   |
| 误差估计  | Z2 能量误差（SPR/L2/加权三种恢复）、单元细化指示器、应力跳跃（牵引跳跃）           |
| 网格质量  | Jacobian 检查、质量评级（bad/warn/ok 互斥分类）、刚体模态守卫           |
| 验证工具  | 分片试验、悬臂梁收敛研究、解析解对标（E1/E2/E3）、漂移门                    |
| 可视化   | 云图（含 Gouraud 光滑）、Bathe 等应力带图（isoband）、交互式后处理、轨迹图    |

| 特征     | 内容                                                               |
|--------|------------------------------------------------------------------|
| 行为冻结区  | element/ 内核公式、solver 数值逻辑、error_est 公式、边界识别算法——改动需特殊程序（见 §5.5）   |
| 金标准锁定  | boundary_golden 段 schema + solve_refactor_lock 逐位锁               |
| 漂移门    | 6 个物理量相对误差 $< 10^{-9}$ ，跨平台（Windows/Linux）一致                     |
| 契约表    | docs/api_contract.md: 全部公开 API 的输入/输出/异常契约 + 探针 151 项断言          |
| 微尺度硬指标 | 1e-150 几何 / 1e12 坐标 / 1e-310 载荷全链路保持有限，禁止绝对阈值                    |
| 输入安全   | 数学表达式经 AST 白名单解析（禁 eval），gmsh 脚本调用经净化                            |
| 错误纪律   | 用户误用 → 带上上下文的 ValueError/TypeError; NaN/Inf 一律拒绝；奇异矩阵抛异常而非返回 NaN |

### 1.3 技术特征

## 2 安装与快速开始

### 2.1 方式一：便携包（免安装，推荐演示用）

FEM2D\_project\_Q4\_9.28.0\_PORTABLE\_WIN64.zip (约 190 MB) 内含 Python 3.13.5 (embeddable 免安装版)、全部依赖 (numpy/scipy/matplotlib/gmsh 及其 DLL)、求解器源码与演示模型。解压到任意 Windows 10/11 x64 电脑即可使用，无需安装 Python、无需联网。

- **完整演示：**双击 demo.bat——椭圆孔内压模型全流程（网格生成 → 求解 → 云图）
- **手动命令行：**run.bat 后直接跟参数（如 run.bat run.py models\demo\_complex.geo --fix left --body 0,-78000）；无参数运行进入交互终端（可连续输入命令，exit 退出）；要逐项交互输入（菜单式）则双击 ask\_model.bat；或直接使用包内解释器：

```
python\python.exe run.py models\demo_complex.geo --plane stress
python\python.exe scripts\verify_all.py --fast 环境自检
```

便携包已在全新路径（模拟新电脑）解压验证：求解结果与参考值一致，全量自检 ALL PASS。

### 2.2 方式二：源码方式（开发/研究用）

```
pip install -e .[dev]          安装运行时 + 开发工具链
python run.py models/demo_complex.spec --no-plot
python scripts/verify_all.py --full 全量验证 9 阶段
```



## 2.3 完整演示流程详解

以 `demo_complex.geo` (椭圆孔内压) 为例, 一次分析依次经历五个阶段 (详见 §5.2):

1. **输入分派**: 识别 `.geo` 为 Gmsh 脚本, 通过捆绑的 `gmsh.exe` 执行生成网格 (子进程, `-nt N -2` 参数), 同时解析 `@FEM`: 注释得到材料/载荷/边界配置;
2. **模型构建**: 网格导入 + 拓扑校验 (重复单元/零边/退化/孤立节点/非流形边) + 边界段识别 (区域注册表 → 几何标签 → 纯拓扑检测三级优先级);
3. **条件施加**: 固定/面力/压力/体力四类边界条件装配为载荷向量;
4. **求解与报告**: 装配刚度矩阵、消去法求解、应力恢复、Z2 误差估计, 输出中文结果摘要;
5. **后处理**: 生成位移/应力云图 (PNG + 交互窗口)。

输出物: `_fem2d_out/` 下含 `mesh.msh`、位移/应力云图、等应力带图与 `report.txt` 中文报告。

## 2.4 端到端演示走读: demo 背后发生了什么

§2.3 讲的是操作步骤; 这里讲代码层面同一流程的五个阶段 (对应 `run_demo.py`, 80 行出头, 演示脚本刻意保持“一眼能读完”)。走读的目的: 把前面各章的概念 (导入管线、边界识别、按标签施加、响亮失败) 拼成一条完整链路。

```
msh_path, gmsh_import = generate_geo_with_topology(
    GEO_PATH, output_path=MSH_PATH, plane_type="stress")
except CliError as error:
    print(f"[ERROR] {error}")
    sys.exit(error.exit_code)
```

`.geo` 脚本交给 Gmsh 网格化后, `gmsh_import` 直接从内存拿节点/单元/物理组——没有 `.inp` 中间文件 (§3.8 弃用 `.inp` 输入的原因)。失败路径用 `CliError` 显式退出并带退出码, 而不是让异常一路裸抛到终端。

```
segs = build_boundary_segments(
    mesh, registry=gmsh_import.regions,
    edge_labels=None, geo_path=geo_path,
    diagnostics=BoundaryDiagnostics())
print_segments(segs)
```

`registry` 是 Gmsh 物理组语义 (§6.7.6 优先级链的第 1 级), `edge_labels=None` 表示不强制标签覆盖, `geo_path` 供命名器回查 `.geo` 里的几何名 (“椭圆孔”/“顶部”等中文标签的来源)。`print_segments` 把识别结果逐段打印——用户第一眼看到的就是边界模型, 而不是数字矩阵。

```

FIX_NAMES = ['left', '左', '底部', 'bottom']
for i, s in enumerate(segs):
    label_lower = s.get('label', '').lower()
    if any(name.lower() in label_lower for name in FIX_NAMES):
        for n in s["nodes"]:
            mesh.fix_node(int(n), "both", 0.0)

```

中英文双名匹配（left/左/底部/bottom）——识别管线的标签层（§6.7.6）把物理组名与几何名统一成中文标签，这里直接按标签找边。若标签变化导致匹配失败，脚本 `sys.exit(1)` 并列出了可用分段——**响亮失败**：历史上曾 WARN 后继续求解，欠约束模型要么奇异矩阵报错（把新手引向错误方向），要么部分约束静默给出错误结果。演示脚本是新手入口，必须响亮失败。

```

for a, b in zip(ns, ns[1:]):
    mesh.add_traction(int(a), int(b), 0.0, -1e6)    # 顶部 1MPa 下压
for a, b in zip(ns, ns[1:]):
    mesh.add_pressure(int(a), int(b), 1e6)         # 椭圆孔内压
mesh.body_force = (0.0, -78000.0)                 # 自重

```

三个载荷各演示一种机制：`add_traction`（分布面力，按边）、`add_pressure`（法向压力，方向由相邻单元 CCW 自动确定，§6.7.4 约定）、`body_force`（体积力）。每个载荷施加后都有 FATAL 守卫——标签生成变化曾导致孔压**静默消失**（求解成功但工况不是文档声明的工况），这是比报错更危险的事故模式。

```

result = solve(mesh)
u = result["u"]; vm = result["vm_stress"]
print(f"最大位移: {np.max(np.abs(u)):.6e} m")
print(f"最大 von Mises: {vm.max():.3e} Pa")
interactive_plot(mesh, result, scale=500)

```

**数值单位纪律**：位移以 m（6e 科学计数）、应力以 Pa 与 MPa 双单位打印——默认 SI 制下 Pa 数值大且难读，MPa 是工程习惯（§10.1 第 1 条单位制提醒）。`scale=500` 是变形放大系数：真实位移  $\sim 10^{-5}$  m 在图上肉眼不可见，放大 500 倍才看得出变形形态（这只是显示约定，不影响解本身）。

1. 导入走 Gmsh API 直读，无.inp 中间格式（§3.8）；
2. 边界模型由物理组语义 + 拓扑角色双来源构建（§6.7.6）；
3. 按标签施加约束/载荷，不硬编码节点索引——标签层是识别管线的对外接口；
4. 三类载荷（面力/压力/体力）对应三种 API，方向约定各一（§4.9 载荷离散）；
5. 每个失败路径都响亮退出（FATAL + 可用分段列表），绝不 WARN 后静默继续；
6. 单位与显示纪律（SI 主单位 + MPa 工程习惯 + 变形放大）。



|    |                      |                                    |
|----|----------------------|------------------------------------|
|    | -- site/             | numpy/scipy/matplotlib/gmsh+pytest |
|    | -- bin/gmsh-4.15.dll | gmsh Python 模块的 DLL                |
| -- | _fem2d_out/          | 运行输出 (自动创建)                        |

各条目作用与可删性 (压缩包约 190 MB, 删掉测试/文档可瘦身到约 150 MB):

| 条目                                 | 作用                                                                                                                                         | 说明                                              |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| run.py                             | CLI 入口 (80 行, 转发 runner.main)                                                                                                              | 必需                                              |
| run_demo.py                        | 一键完整演示 (§2.3–2.4 走读对象)                                                                                                                     | 必需 (演示入口)                                       |
| demo.bat / run.bat / ask_model.bat | 双击批处理: cd /d "% dp0" 以自身目录为基准; 文件以 <b>GBK</b> 编码保存 (中文 Windows 原生代码页, 零编码竞态); run.bat 无参数进入交互终端 (可连续输入命令, exit 退出); ask_model.bat 为交互式菜单向导 | 必需 (无命令行经验用户入口)                                 |
| README.txt                         | 包内快速开始说明 (与实际 bat 内容一致)                                                                                                                    | 建议保留                                            |
| fem2d/                             | 求解器核心源码包 (element/, boundary/, 五阶段管线)                                                                                                      | 必需                                              |
| scripts/                           | verify_all.py / regression_compare.py 等验证与工具脚本                                                                                             | 必需 (环境自检); fuzz/打包脚本可删                          |
| tests/                             | 166 个 pytest 测试文件 (§9 章主题)                                                                                                                 | 可删 (运行与分析无关, 调试/学习才用)                           |
| docs/                              | 设计/契约/审查文档 (.md)                                                                                                                           | 可删                                              |
| models/                            | 演示与基准几何 (.geo) + 工况 (.spec)                                                                                                                | 必需 (演示/练习)                                      |
| tools/gmsh-4.15.2-Windows64/       | gmsh.exe 独立可执行文件; CLI 子进程生成网格 (§2.3 第一阶段)                                                                                                  | 必需 (.geo 路径); 缺失时.geo 不可用, .msh/.spec 仍可 (§9.9) |
| python/                            | Python 3.13.5 embeddable 免安装解释器 (python.exe + DLL + 标准库)                                                                                   | 必需                                              |
| python/site/                       | 第三方依赖 (numpy/scipy/matplotlib/gmsh+pytest 及各自 DLL)                                                                                         | 必需                                              |

|                                                    |                                                                     |
|----------------------------------------------------|---------------------------------------------------------------------|
| python/site/bin/gmsh-4.15.dll                      | gmsh Python 模块的 DLL: 必需<br>API 直读.msh (run_demo 路径, §2.4 第一阶段)      |
| python/tkinter/、_tkinter.pyd、tcl86t/tk86t.dll、tcl/ | Tk 图形后端 (TkAgg): matplotlib 交互窗口弹窗<br>必需 (窗口云图); 仅用 --save 存 PNG 可删 |
| _fem2d_out/                                        | 运行输出 (网格/云图/报告), 自动创建<br>可删 (每次运行重建)                                |

移动/解压注意事项 (换电脑失败的高发原因, 详见 §2.8):

1. 压缩包内没有外层目录——资源管理器”全部解压缩”会自动建同名文件夹; 用命令行 unzip 时务必先建目录再解压, 否则文件散落在当前目录;
2. 整体搬运: python/、tools/ 与源码的相对路径是固定的 (bat 用 %~dp0 定位自身目录), 只拷 run.py 或只拷 exe 必挂;
3. 杀毒软件可能隔离 python.exe / gmsh.exe——解压后先确认文件齐全再双击;
4. 路径建议: 解压到纯 ASCII 路径 (如 D:\FEM2D); 中文字符路径在个别终端 + gmsh 子进程组合下可能出现编码问题, 遇到先换路径再排查。

## 2.7 Gmsh .geo 语法详解 (重点)

.geo 是本项目的主要输入形态 (§3.1 按扩展名分派)。为什么重要: 它是 Gmsh 的脚本语言, 把几何 (点/线/面) 与网格控制 (尺寸/算法) 写在一个文本文件里, 可编程 (变量、循环、数学函数), 还能通过 @FEM: 注释把工况也放进来 (§3.6) ——”几何 + 工况”单文件。求解器拿到.geo 后交给捆绑的 Gmsh 执行并直接读回网格 (§2.3 第一阶段), 不需要手写网格文件。

### 2.7.1 脚本基础: 注释、变量、表达式与循环

```
// 行注释 (demo_complex.geo 头部即全中文注释)
/* 也可以写块注释 */
lc = 0.03; // 变量赋值, 每条语句以分号结尾
W = 2.0; H = 1.5; FR = 0.3; // 一行多语句
```

- 注释: // 行注释、/\* \*/ 块注释——脚本可读性靠注释, demo 文件里每个点/线都有中文注释说明几何意图;
- 变量: 直接赋值即创建, 无类型声明; 整型/浮点/字符串自动区分; 可做算术运算 (ax - aR 等);
- 数学函数: Pi 常量, Cos() Sin() Sqrt() Atan2() Mod() Floor() 等, 可嵌套 (2\*Pi\*i/n1);
- For 循环 (椭圆孔分段的写法):

```

For i In {0:n1-1}
  ang = 2*Pi*i/n1;
  Point(200 + i) = { cx1 + a1*Cos(ang), cy1 + b1*Sin(ang), 0, lc};
EndFor

```

循环范围 {a:b} 含两端点，循环变量可参与算术；范围必须整数值（先算好整数再循环，别在范围里写除法）。

**编号规则：**每种实体（Point/Line/Circle/Curve Loop/Plane Surface/Physical）有独立命名空间，同类内编号唯一。编号不要求连续——demo 里外边界用 1-17、椭圆孔用 200+、孔边线用 300+，分段编号是组织长脚本的常用手法（新实体用大号段，不会与已有编号冲突）。

### 2.7.2 几何命令全解

| 语法                                     | 语义 / 要点                                                 |
|----------------------------------------|---------------------------------------------------------|
| Point(id) = {x, y, z, lc};             | 空间点；2D 问题 z 恒为 0；第 4 个参数 lc 是该点附近的网格特征长度（越小越密）          |
| Line(id) = {p1, p2};                   | 两点的直线段                                                  |
| Circle(id) = {起点, 圆心, 终点};             | 圆弧（三点式）；起点绕圆心转到终点；张角必须小于 180°（Gmsh 硬限制，见 §2.7.6）        |
| Curve Loop(id) = {e1, e2, ...};        | 曲线段围成的闭合环：首尾相接、方向一致；反方向写负号 {-e3}                        |
| Plane Surface(id) = {环 1, 环 2, ...};   | 第一个环为外边界，后续环是孔（方向相反）；嵌套环 = 环内挖环                         |
| Physical Surface("domain", id) = {面};  | 给面分组命名（导出进.msh 的 \$PhysicalNames）                       |
| Physical Curve("left", id) = {边, ...}; | 把若干曲线归为一个边界组——求解器边界标签的来源（§2.4 第三阶段 fix = left, 底部按标签匹配） |

Physical 标签中英文皆可（demo 里 left/right/底部/顶部/凹弧/椭圆孔混用），求解器标签层统一成中文语义（§6.7.6）。标签名是用户与识别管线的接口——命名规范（语义清晰、覆盖全部边界）直接决定边界条件能不能按标签施加。

### 2.7.3 进阶建模命令

基础表之外的常用命令。先记住一个分水岭：内置内核（上面的点/线/面语法）稳定、教学最友好；OpenCASCADE 内核（SetFactory("OpenCASCADE") 切换）才有布尔运算、圆角倒角、真样条曲面。项目捆绑的 gmsh.exe 两个内核都支持，run.py 只按.geo 内容执行，与内核无关；建模习惯上：常规板件用内置内核，复杂外形（布尔挖孔、倒角、曲边）切 OpenCASCADE。

| 语法                                                             | 语义 / 要点                                                                                                    |
|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Ellipse(id) =<br>{起点, 圆心, 辅助点, 终点};                            | 椭圆弧: 起点/圆心定一个半轴方向, 辅助点定长短轴比; 张角仍 $< 180^\circ$ , 全椭圆拆多段——demo 用 20 段多边形近似                                  |
| Polyline(id) = {p1, p2, ...};                                  | 一条命令画多段折线, 只占一个 <b>实体号</b> (标签/引用更省心), 等价若干 Line                                                           |
| Spline(id) = {p1, p2, ...};                                    | Catmull-Rom 样条, <b>曲线穿过全部给定点</b> ; 拟合采样点轮廓用这个                                                              |
| BSpline(id) = {控制点, ...};                                      | B 样条, 曲线 <b>不穿过</b> 控制点 (靠近); Bezier 同理; 设计曲线用这些                                                           |
| Extrude \{dx,dy,dz\} \{ Line\{...\}; \}                        | 线沿矢量拉伸成面——2D 矩形板<br>一条命令生成 3 条新边 + 1 个新面, <b>新实体编号自动分配</b> (引用前先在 gmsh 界面查看编号)                             |
| Revolve \{ {x,y,z},<br>\{dx,dy,dz\}, ang \} \{ Line\{...\}; \} | 绕轴旋转拉伸; 圆环、弧板、回转件的 2D 截面建模                                                                                 |
| Translate \{dx,dy,dz\}<br>\{ 实体...; \}                         | 平移实体; Rotate、Mirror、Scale 同族 (参数格式类似, 见 Gmsh 手册); Duplicata 包裹 = <b>复制</b> 而非移动                            |
| BooleanUnion()<br>BooleanIntersection()<br>BooleanDifference() | 布尔运算 (挖孔/并体/交体); <b>必须先 SetFactory("OpenCASCADE");</b><br>被 减 实 体 用<br>\{ Surface\{a\}; Delete; \}<br>就地删除 |
| Coherence;                                                     | 合并重合的点/线 (拓扑净化), 布尔运算后收尾常用                                                                                 |
| Point\{a\} In Surface\{b\};                                    | 嵌入点/线: 把载荷作用点钉在面内, 避免网格节点恰好在施力位置的尴尬                                                                        |

Extrude/Revolve 生成的实体编号是**自动分配**的, 无法在 .geo 里预知; 模板化建模时要么先跑一遍记下编号, 要么仍手写点/线拿回编号控制权 (demo\_complex.geo 即后者——全部显式编号, 可读性优先)。

#### 2.7.4 网格控制命令

```

Mesh.Format = 39;           // 输出 MSH 4.1 文本格式（项目统一）
Mesh.SaveAll = 1;           // 保存所有实体（默认只存物理组）
Recombine Surface{1};       // 三角网格重组为四边形 —— Q4 网格的关键
Mesh 2;                     // 生成 2 维网格（曲面网格化）
Save "demo_complex.msh";    // 写出网格文件

```

- **lc 特征长度**：每个 Point 的 lc 控制该点附近网格单元尺寸；全部用同一个 lc 变量 = 全模型统一密度（demo 的 lc = 0.03 生成约 1.9 万四边形）。lc 减半 → 单元数约乘 4，求解时间非线性上升——先大 lc 验证工况，再加密；
- **Recombine Surface{1}**：把面上三角网格重组为四边形。demo\_complex.geo 内置此行 → 默认跑出来就是 Q4 (§2.5 表)；运行时 --quad 等价于强制 Mesh.RecombineAll = 1 + 切换重组算法（gmsh\_adapter.py:760–762）。求解器拒绝三角/四边形混合网格——重组不完整时（复杂表面/洞边）报错提示指向此问题；
- Mesh 2；之后的 Save 写出的内容由 Mesh.Format 决定（4.1 文本格式，§3.8 详解）。

### 2.7.5 完整实例走读：demo\_complex.geo

官方示例 models/demo\_complex.geo（106 行）是上述命令的组合范例，按四段读：

①全局参数与底边半圆凹口（变量 + 弧拆分）：

```

W = 2.0;  H = 1.5;  FR = 0.3;
aR = 0.35;  ax = 0.8;           // 凹弧半径，弧心 x 坐标
Point(1) = { -W+FR, -H, 0, lc}; // 左下圆角终点
Point(2) = { ax-aR, -H, 0, lc}; // 凹弧左端点
Point(4) = { ax, -H, 0, lc};    // 凹弧圆心
Point(17) = { ax, -H+aR, 0, lc}; // 凹弧顶点（半圆最高点）
Line(1) = {1, 2};
Circle(2) = {2, 4, 17};         // 凹弧左半段：起点2 圆心4 终点17
Circle(11) = {17, 4, 3};        // 凹弧右半段（两个90°弧拼成180°）

```

要点：半圆凹口被拆成两个 90° 弧——Gmsh 的 Circle 要求张角 < 180°，半圆必须拆（文件注释明确写着这条约束）。

②外边界闭环（10 条曲线首尾相接）：

```

Curve Loop(100) = { 1, 2, 11, 3, 4, 5, 6, 7, 8, 9, 10 };

```

环内每段的方向就是 Circle/Line 定义的起点 → 终点方向，必须沿环一致；首尾闭合（最后一段终点 = 第一段起点）。

③椭圆孔（For 循环 + 范围列表）：

```

For i In {0:n1-1}
  Point(200 + i) = { cx1 + a1*Cos(ang), cy1 + b1*Sin(ang), 0, lc};
EndFor
For i In {0:n1-1}
  Line(300 + i) = { 200 + i, 200 + ((i+1)%n1) };

```



```
EndFor
Curve Loop(301) = { 300:300+n1-1 }; // 范围写法 {a:b} 含两端
```

椭圆用 20 段多边形近似；孔环的方向与外环相反（内孔边界）。

#### ④面 + 标签 + 网格:

```
Plane Surface(1) = { 100, 301 }; // 外环 + 孔环 = 带孔面
Physical Surface("domain", 200) = {1};
Physical Curve("left", 101) = {9};
Physical Curve("底部", 103) = {1, 3}; // 底边两段并成一个标签
Physical Curve("椭圆孔", 112) = {300:319};
Recombine Surface{1};
Mesh 2;
Save "demo_complex.msh";
```

标签把”底部”的两条曲线归为一组——边界条件 `fix = 底部` 一次匹配整条底边 (§2.4 第三阶段的标签匹配来源)。这就是完整的”几何 → 可求解模型”闭环。

### 2.7.6 参数化建模模板：改变量 = 换模型

.geo 是脚本语言，最高效的用法是**参数化模板**：把尺寸/位置/ 网格密度全部提到文件头部变量，改几个数重跑一遍就是新模型。这也是”几何 + 工况单文件”的完整形态 (§3.6 的 @FEM: 注释可叠加在任意.geo 上)。以带圆孔矩形板为例（比 demo 更简，所有边界都打了 Physical 标签，求解时 `fix/traction` 直接按名引用）：

```
W = 2.0; H = 1.0; // 板尺寸
CX = 1.0; CY = 0.5; R = 0.3; // 孔心 + 半径
lc = 0.05; // 网格密度
Point(1) = {0, 0, 0, lc}; Point(2) = {W, 0, 0, lc};
Point(3) = {W, H, 0, lc}; Point(4) = {0, H, 0, lc};
Line(1) = {1, 2}; Line(2) = {2, 3};
Line(3) = {3, 4}; Line(4) = {4, 1};
Curve Loop(1) = {1, 2, 3, 4};
n = 16; // 孔多边形段数
For i In {0:n-1}
  ang = 2*Pi*i/n;
  Point(100 + i) = { CX + R*Cos(ang), CY + R*Sin(ang), 0, lc};
EndFor
For i In {0:n-1}
  Line(200 + i) = {100 + i, 100 + ((i+1)%n)};
EndFor
Curve Loop(2) = {200:200+n-1};
Plane Surface(1) = {1, 2}; // 外环 + 孔环 = 带孔面
Physical Surface("domain", 200) = {1};
Physical Curve("left", 101) = {4};
Physical Curve("right", 102) = {2};
Physical Curve("top", 103) = {3};
```

```
Physical Curve("bottom", 104) = {1};
Recombine Surface{1};
Mesh 2;
Save "params.msh";
```

用法：想算孔偏右、板加长的工况，只改  $CX = 1.5$ ； $W = 2.5$ ；两个数重跑即可。编号分段（外框 1-4、孔点 100+、孔边 200+）保证变量化后不会撞号；Physical 标签与变量无关，永远能按名施加边界条件。把这份模板存成 `templates/` 目录的基础件，新工况从复制开始，比从零写 `.geo` 快得多（§5 顶层结构有 `templates/` 的说明）。

### 2.7.7 常见错误与调试

- 弧超过  $180^\circ$ ：Gmsh 直接报错——拆成几个  $< 180^\circ$  的弧（完整实例走读里的凹口是现成模板）；
- 环不闭合 / 方向不一致：建面失败，报“Bad orientation”类错误——逐段核对首尾点编号与方向；
- 编号重复：同类实体编号冲突报错——换分段编号；
- 忘写分号：报错定位在下一行——逐句补；
- For 范围含小数（如  $\{0:n/2\}$ ）：范围必须整数——先取整或用整数计数变量；
- lc 过小：单元数爆炸、生成很慢——先用大 lc 验证工况；
- 生成成功但求解报“混合网格”：Recombine 未完全生效（复杂表面）——检查退化区或用 `--quad` 强制；
- 布尔运算报“Unknown” / 命令不识别：忘了 `SetFactory("OpenCASCADE")`——内置内核无 Boolean，切工厂后重跑；
- 引用 Extrude 自动编号的实体失败：新实体编号由 Gmsh 分配，`.geo` 里写死会错——先用 gmsh 界面跑一遍查编号，或改手写点/线；
- Spline 点数太少：样条至少 3 个点（BSpline 至少 2 个控制点）——报错信息里能看出点数。

调试手段：先用 `gmsh.exe` 手动打开 `.geo`（Gmsh 图形界面能直观看到几何与网格，错误定位最快），确认无误后再交给 `run.py` 求解。

## 2.8 换电脑使用流程（新机器首次运行）

本节约为“昨天换电脑跑不了”的场景写的排查专节。先记住结论：**便携包不依赖系统 Python、不依赖 PATH、不依赖当前目录**——bat 用 `cd /d "%~dp0"` 定位自身目录（§2.6 表）。所以换电脑跑不了几乎只有三类原因，按序排查即可。

### 2.8.1 第一步：判断自己走哪条路线

- 目录里有 `python\python.exe`（便携包）→ §2.8.2；
- 没有，是 pip 安装的源码方式 → §2.8.3（新机器没装 Python / gmsh 是最常见原因）。

### 2.8.2 便携包路线：5 步跑通

1. **完整解压**：资源管理器右键”全部解压缩”得到同名文件夹（zip 内无外层目录，命令行 unzip 先建目录，§2.6 注意事项）；
2. **位置**：建议纯 ASCII 路径（D:\FEM2D 等），中文路径遇到编码问题先换路径；
3. **双击 demo.bat**：看到”FEM2D Q4 v9.28.0 便携演示”横幅与网格生成进度 → 终端打印最大位移/最大 von Mises → 弹出云图窗口 = 成功；
4. **环境自检**：run.bat scripts\verify\_all.py --fast ——全部 PASS 即环境健康；
5. **跑其他模型**：run.bat run.py models\demo\_complex.spec 或 run.bat run.py models\demo\_complex.geo --fix left --body 0,-78000（参数直接跟在 bat 后面）；想逐项交互输入则双击 ask\_model.bat 交互向导。

### 2.8.3 源码路线：5 步跑通

1. 安装 Python 3.10+（CI 用 3.13；官方安装包勾选 Add python.exe to PATH）；
2. 命令行进入项目目录：python -m pip install -e .[dev]；
3. gmsh 检查：python -c "import gmsh; print(gmsh.\_\_version\_\_)" ——失败则 pip install gmsh（.geo 路径必需；只读.msh 可跳过，§9.9 无 gmsh 纪律）；
4. python scripts\verify\_all.py --fast 全部 PASS；
5. python run.py models\demo\_complex.spec --no-plot。

### 2.8.4 故障速查表

| 症状                                  | 最常见原因                                       | 解决                                            |
|-------------------------------------|---------------------------------------------|-----------------------------------------------|
| 双击 demo.bat 闪退/无反应                  | 解压不完整；杀毒隔离 python.exe                       | 重新完整解压（约 190MB），目录加白名单                        |
| 提示'python' 不是内部或外部命令                | 在系统 cmd 直接敲 python（源码路线没装 Python / PATH 未配） | 便携包用 python\python.exe（bat 已自动）；源码路线先装 Python |
| 报 gmsh 相关错误（.geo 路径）                | tools/ 或 python/site 缺失                     | 便携包重新解压；源码路线 pip install gmsh                 |
| 求解报”Mesh contains invalid elements” | 网格生成参数问题，不是环境问题                             | 看 jacobian_report 输出与 §10 排查流程                |
| 旧电脑能跑、新电脑不能                         | 只复制了部分文件；用了旧桌面快捷方式                          | 整个目录一起拷，重新双击（bat 以自身目录为基准，文件齐就能跑）             |
| 满屏 font/Agg 警告、窗口弹不出                | 旧版包无 tkinter（Agg 后端无窗口）                     | 新版包已内置 tkinter 可弹窗；任何情况 --save 存 PNG 兜底       |

|                                                 |                                                                   |                                                            |
|-------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------|
| 终端中文乱码                                          | 代码页不是 UTF-8                                                       | bat 已 chcp 936 (GBK 原生); 手动 cmd 先执行 chcp 936               |
| 运行 run.bat 只出 Python 版本号 + >>> 提示符              | 旧版 bat 无参数时把 python\python.exe 丢给 cmd 交互解释器                       | 新版 run.bat 无参数进入交互终端(FEM2D> 提示符, exit 退出)                  |
| 报 'dp0"' 不是内部或外部命令或 ' 锯斤拷...' is not recognized | 旧版 bat 以 UTF-8+LF 保存,在 GBK 代码页 cmd 下字节错位 (chcp 65001 切换与文件读取存在竞态) | 重新解压新版 zip; bat 已改为 GBK+CRLF 编码 (中文 Windows 原生代码页, 无切换无竞态) |

2.8.5 自检命令说明

verify\_all (scripts/ 下) 是环境健康的权威判据: --fast (约 2-3 分钟: 核心测试 + 探针 + 漂移门 + 无 gmsh 模拟); --full 九阶段全量; --drift 只跑漂移门; --no-gmsh 模拟无 gmsh 环境。无 gmsh 也能全 PASS——1600+ 测试在无 gmsh 下 skip 而不失败 (§9.9 纪律), 这是”环境不完整但还能验证”的设计。

3 命令行用法

3.1 入口

主入口 run.py (纯转发到 runner.main), 位置参数为输入文件:

python run.py <input> [options]

<input> 支持四种格式, 自动识别:

| 扩展名   | 内容                          | 处理方式             |
|-------|-----------------------------|------------------|
| .spec | 工况配置 (键值对, 可引用几何/网格文件)      | 键值解析 + 引用文件派发    |
| .geo  | Gmsh 几何脚本 (可含 @FEM: 配置注释)   | 调用 gmsh.exe 生成网格 |
| .txt  | 中文几何描述 (简易建模)               | 内置解析器生成网格        |
| .msh  | Gmsh 网格文件 (含 PhysicalNames) | 直接导入             |

3.2 主要参数

|                              |                                |
|------------------------------|--------------------------------|
| --plane {stress,strain}      | 平面应力/应变 (默认按单元码自动判定)           |
| --E / --nu / --lc<br>lc 按几何) | 材料与网格尺寸 (默认 E=2.10e11, nu=0.3, |
| --elem-type {CST,Q4,Q4R,Q4I} | 单元类型覆盖                         |
| --fix / --fix-ux / --fix-uy  | 边界约束 (节点号或几何边名)                |
| --traction / --force         | 面力/集中力                         |

```

--body                    体力（支持含 x/y 的表达式）
--error-method {auto,spr,l2,weighted} 误差恢复方法
--no-plot / --keep-open  后处理控制
--save SAVE / --output-dir 结果输出
--linear-solver {auto,direct,cg,cg-block,ilu} 线性求解器
--method {elimination,penalty} 约束施加方法
--self-test              内置自检

```

### 3.3 参数细节

| 参数                   | 取值                          | 说明                                               |
|----------------------|-----------------------------|--------------------------------------------------|
| --plane              | stress / strain             | 不传时按单元码自动判定（CPS* → stress, CPE* → strain）；显式传值优先 |
| --elem-type          | CST / Q4 / Q4R / Q4I        | 对生成的网格做单元类型覆盖（如三角形网格 → CST；四边形网格 → Q4 族）         |
| --fix                | n:dof:val                   | 约束节点 n 的 dof (x/y/both)，可多次；也接受几何边名              |
| --fix-ux<br>--fix-uy | / 节点号                       | 批量固定一组节点的 x / y 位移                               |
| --traction           | edge:tx,ty[:p l n]          | 边上的分布面力；p= 抛物线、l= 线性、n= 法向压力                     |
| --force              | n:fx,fy                     | 节点集中力                                            |
| --body               | bx,by                       | 体力分量，可用含 x/y 的数学表达式（AST 白名单）                     |
| --lc                 | 实数                          | 网格特征尺寸，越小网格越密                                    |
| --error-method       | auto/spr/l2/weighted        | 误差估计的应力恢复方法；auto 优先 SPR                          |
| --linear-solver      | auto/direct/cg/cg-block/ilu | auto ≤ 10 万 DOF 用 direct，更大用 Jacobi-PCG          |
| --method             | elimination/penalty         | 约束施加方法；penalty 仅兼容 direct 类求解器                   |

### 3.4 示例

```

# 1. 演示模型（椭圆孔内压）：完整流程 + 云图
python run.py models/demo_complex.geo

# 2. 工况文件（右端拉伸 1 MPa）：无绘图，命令行结果
python run.py models/demo_complex.spec --no-plot

# 3. 悬臂梁：固定左端，端部集中力，Q4 单元，L2 恢复
python run.py models/cantilever.geo --fix-ux 0,1,2 --fix-uy 0,1,2 \
    --force 9:0,-1000 --elem-type Q4 --error-method l2

```

```
# 4. 大型模型：显式迭代求解器 + 性能自检
python run.py big.geo --linear-solver cg --plane strain
```

### 3.5 spec 文件语法

.spec 是键值对文本，编码一次分析的完整工况（材料、网格、载荷、边界）。支持：缺 = 或空值报错（带行号）、BOM 自动剥离、未知键警告忽略（前向兼容）、E/nu/lc 等数值语义在 config 校验层统一拦截。

```
# demo_complex.spec 示例
E = 2.1e11          # 杨氏模量 (Pa)
nu = 0.3            # 泊松比
t = 0.01            # 厚度 (m)
lc = 0.5            # 网格特征尺寸
plane = stress       # 平面应力
mesh = models/demo_complex.geo
fix = 1:x,2:y        # 节点约束
force = 5:0,1000     # 集中力
```

### 3.6 .geo 中的 @FEM 配置注释

.geo 是 Gmsh 脚本，其中的 @FEM: 注释会被解析为配置（与 spec 相同的键值格式），并随网格一起加载，实现”几何 + 工况”单文件：

```
// @FEM: E=2.1e11
// @FEM: nu=0.3
// @FEM: fix=1:x
// @FEM: traction=3:0,1e6:n
```

### 3.7 .txt 中文几何

.txt 提供简易中文几何描述（矩形/圆孔/椭圆孔组合 + 固定/力/压力关键词），面向快速教学建模，由内置解析器直接生成结构化网格，无 gmsh 依赖。

### 3.8 .msh 网格文件详解（Gmsh 原生格式）

.msh 是本软件实际使用的网格交换格式：Gmsh 原生格式，经 Gmsh API 直接导入（fem2d/gmsh\_adapter.py 的 import\_msh() → \_extract\_mesh()），不经过任何中间格式。无论是 .geo 生成还是外部工具（Gmsh、tMG、Salome 等）导出的 .msh，都在这一条链路上被解析。

文件结构。ASCII 文本版（v2.2）的最小单三角形网格示例：

```
$MeshFormat
2.2 0 8
```

```

$EndMeshFormat
$PhysicalNames
1
1 2 "square"
$EndPhysicalNames
$Nodes
3
1 0 0 0
2 1 0 0
3 0 1 0
$EndNodes
$Elements
1
1 2 2 1 1 2 3
$EndElements

```

各段含义（导入器真正关心的只有三段）：

| 段名              | 含义                              |
|-----------------|---------------------------------|
| \$MeshFormat    | 版本号 + 0（ASCII）/1（二进制）+ 8（双精度）   |
| \$PhysicalNames | 物理组标签（"square" 等），驱动语义边界识别与区域命名 |
| \$Nodes         | 节点编号 + 坐标 x y z（每行一个）           |
| \$Elements      | 每行 = 编号、类型码、物理组、几何实体、节点……       |

\$Elements 的类型码：2 = 3 节点三角形（TRI3）、3 = 4 节点四边形（QUAD4）、15 = 3 节点线（T3D2，只出现在边界/构造实体中）。上例行 1 2 2 1 1 2 3 即：元素 1、类型 2（三角形）、物理组 2、几何实体 1、节点 1-2-3。

导入管线的硬规则（\_extract\_mesh()，逐条有测试锁定）：

1. 只接受一阶线性 2D 单元（order=1 且节点数等于主节点数，3 或 4）；二阶/混合阶单元抛 GmshTopologyError。
2. 同质网格：三角形与四边形混排的网格被拒绝（本软件每个模型只能是一种单元族）。
3. 只保留被 2D 单元引用的节点：Gmsh 的 getNodes() 会返回构造实体（点/线/体积网格）上的节点，导入器把未被任何 2D 单元引用的节点一律删除并重映射编号——否则它们会在总刚中留下全零行/列，造成“约束看似充分却奇异矩阵”（历史教训，见 §10.1）。
4. 单元方向归一化为逆时针（normalize\_element\_orientation），外法向方向从此确定（§6.7.4 定向的压力法向约定依赖它）。
5. Z 平坦性检查：活动 2D 节点的 z 坐标跨度必须不超过  $\max(\text{span} \cdot 10^{-8}, \text{ULP 兜底})$ ；非平面（2.5D）网格被拒绝而非静默投影——容差是相对尺度的，微米/纳米级模型不会被误杀（§8 微尺度纪律）。
6. 单元类型按节点数自动命名为 CPS3/CPE3/CPS4/ CPE4（前缀由平面应力/平面应变决定），后续全部代码只认这四种名字。

为什么不再用.inp 中间格式 (2026-08 移除 Abaqus 输入口)。历史版本曾支持 .inp 输入: 逐行解析 \*NODE、\*ELEMENT, TYPE=...、\*ELSET 等关键字。手写解析器在真实文件面前暴露了一批边界情况缺陷: TYPE= 子串匹配把 CPS4 误判成 CPS3 (ELSET 名 CPS3\_GROUP 也会触发); Gmsh 导出的 .inp 混合了 T3D2 线单元节点, 导入后产生孤立节点 → 奇异矩阵; .geo 脚本被误当 .inp 传入时报错信息指向错误方向。改走 Gmsh API 直读 .msh 后, 网格拓扑、物理组语义、孤立节点过滤全部由库级保证, 整类”手写解析器边界情况”错误消失。.inp 仅保留为导出格式 (写 Abaqus 风格文件供其他软件使用), 不再是输入格式。

常见错误速查 (完整列表见 §10.1):

- mixed triangle/quad mesh — 混排网格: 对四边形需求做重组 (--quad 触发 re-combination) 或网格分文件导入;
- Unsupported Gmsh displacement element — 高阶/异质单元: 设 Mesh.Order = 1 强制一阶, 或后处理降阶;
- non-constant z-coordinates — 2.5D 网格: 先投影到平面再导出;
- No supported first-order 2-D elements — 网格里只有线/点单元: 检查 \$Elements 段类型码。

## 4 理论基础

本章给出求解器所实现数值方法的数学推导, 公式编号与 Bathe 《Finite Element Procedures》(2nd ed.) 对应。教学目的是: 每条核心公式都能在教材中找到出处, 这也是本项目的公式纪律。

**阅读指引:** 本章的重心放在协调单元 (CST/Q4) 的构造与性质, 以及沙漏控制 (Q4R 单点积分的稳定化) —— 这两块既是线性单元教学最核心的内容, 也是本求解器与商业软件一致的关键机制。弹性力学三大方程 (平衡/几何/本构) 只作一行回顾, 不再展开推导。

### 4.1 平面线弹性问题的弱形式

#### 4.1.1 三大方程 (一行回顾)

平面问题在域  $\Omega$  (边界  $\Gamma = \Gamma_u \cup \Gamma_t$ ) 上由三组方程描述 (Bathe Eq. 4.1–4.12): 平衡方程  $\mathbf{L}^T \boldsymbol{\sigma} + \mathbf{f} = \mathbf{0}$ 、几何方程  $\boldsymbol{\varepsilon} = \mathbf{L} \mathbf{u}$  ( $\boldsymbol{\varepsilon} = [\varepsilon_x, \varepsilon_y, \gamma_{xy}]^T$ ,  $\mathbf{L}$  为微分算子)、本构方程  $\boldsymbol{\sigma} = \mathbf{D} \boldsymbol{\varepsilon}$ 。平面应力 ( $\sigma_z = 0$ , Bathe Eq. 4.11) 与平面应变 ( $\varepsilon_z = 0$ , Bathe Eq. 4.12) 的弹性矩阵:

$$\mathbf{D}_{stress} = \frac{E}{1-\nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}, \quad \mathbf{D}_{strain} = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & 0 \\ \nu & 1-\nu & 0 \\ 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix} \quad (1)$$

其中  $E$  为杨氏模量、 $\nu$  为泊松比。边界条件 (Bathe Eq. 4.3–4.5):  $\mathbf{u} = \bar{\mathbf{u}}$  on  $\Gamma_u$  (本质边界),  $\mathbf{t} = \bar{\mathbf{t}}$  on  $\Gamma_t$  (自然边界)。



### 4.1.2 虚功原理（弱形式）——有限元离散的起点

对平衡方程取虚位移  $\delta \mathbf{u}$  的加权积分（虚功原理，Bathe Eq. 4.33）：

$$\int_{\Omega} \delta \boldsymbol{\varepsilon}^T \mathbf{D} \boldsymbol{\varepsilon} d\Omega = \int_{\Omega} \delta \mathbf{u}^T \mathbf{f} d\Omega + \int_{\Gamma_t} \delta \mathbf{u}^T \bar{\mathbf{t}} d\Gamma \quad (2)$$

左侧是内力虚功（刚度项），右侧是外力虚功（载荷项）。全部有限元公式都从这里出发：只要给出单元内位移插值（形函数），式 (2) 就机械地产生单元刚度矩阵与载荷向量。这也是“协调单元”的定义所在——形函数在单元边界上保持位移连续，弱形式才合法。

## 4.2 位移离散与单元刚度

### 4.2.1 位移插值与 B 矩阵

将  $\Omega$  划分为  $n_e$  个单元，单元内位移用形函数插值（Bathe Eq. 4.21）：

$$\mathbf{u}^{(e)} = \mathbf{N}^{(e)} \mathbf{d}^{(e)}, \quad \mathbf{N} = \begin{bmatrix} N_1 & 0 & N_2 & 0 & \cdots \\ 0 & N_1 & 0 & N_2 & \cdots \end{bmatrix} \quad (3)$$

其中  $\mathbf{d}^{(e)}$  为单元节点自由度向量。应变-位移阵（Bathe Eq. 4.24）：

$$\boldsymbol{\varepsilon} = \mathbf{B} \mathbf{d}^{(e)}, \quad \mathbf{B} = \mathbf{L} \mathbf{N} \quad (4)$$

代入虚功原理 (2)，得单元刚度矩阵与载荷向量（Bathe Eq. 4.30–4.31）：

$$\mathbf{k}^{(e)} = \int_{\Omega^{(e)}} \mathbf{B}^T \mathbf{D} \mathbf{B} d\Omega \quad \mathbf{f}^{(e)} = \int_{\Omega^{(e)}} \mathbf{N}^T \mathbf{f} d\Omega + \int_{\Gamma_t^{(e)}} \mathbf{N}^T \bar{\mathbf{t}} d\Gamma \quad (5)$$

**整体装配：**按自由度编号  $d_{2i} = u_i$ （x 向）、 $d_{2i+1} = v_i$ （y 向），对每个单元把  $\mathbf{k}^{(e)}$  叠加进全局刚度  $\mathbf{K} = \sum_e \mathbf{k}^{(e)}$ （Bathe Eq. 4.18 的 DOF 约定），载荷向量同理  $\mathbf{F} = \sum_e \mathbf{f}^{(e)}$ 。装配在 `assembly.py` 中以稀疏 COO 结构实现（向量化路径）。

### 4.2.2 数值积分（高斯求积）

式 (5) 在等参单元中写为母单元上的积分（Bathe Eq. 5.39）：

$$\mathbf{k}^{(e)} = \int_{-1}^1 \int_{-1}^1 \mathbf{B}(\xi, \eta)^T \mathbf{D} \mathbf{B}(\xi, \eta) t \det \mathbf{J} d\xi d\eta \approx \sum_{i,j} w_i w_j \mathbf{B}_{ij}^T \mathbf{D} \mathbf{B}_{ij} t \det \mathbf{J}_{ij} \quad (6)$$

其中  $\mathbf{J}$  为坐标变换 Jacobian， $\mathbf{B}$  的导数项  $\partial N_i / \partial x = \mathbf{J}^{-1} \partial N_i / \partial \xi$ 。Q4 用  $2 \times 2$  高斯积分（4 点），CST 刚度矩阵为常数阵（面积积分解析完成，见下）。

### 4.3 单元族

#### 4.3.1 CST——常应变三角形

线性三角形单元 (Bathe §5.2.3), 形函数为面积坐标:

$$N_i = \frac{a_i + b_i x + c_i y}{2A}, \quad A = \frac{1}{2} \det \begin{bmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{bmatrix} \quad (7)$$

由于形函数线性,  $\mathbf{B}$  为常数阵 (应变为常数, 故称常应变单元), 单元刚度闭合形式:

$$\mathbf{k}^{(e)} = t \mathbf{A} \mathbf{B}^T \mathbf{D} \mathbf{B} \quad (8)$$

**性质:** 实现最简单、计算最快; 弯曲刚度偏刚 (线性单元的低阶表现, 见 §7.4 E1 结果); 无转角自由度——”固定端”约束在 CST 中等效于一排固定铰 (锁整条边所有节点的平动)。

#### 4.3.2 Q4——双线性协调四边形 (本章重点)

四节点双线性等参单元 (Bathe §5.2.4) 是**协调单元** (conforming element) 的典范: 形函数在单元内  $C^0$  连续、跨单元边界位移连续, 弱形式 (2) 严格成立。下面的内容按”构造 → 性质 → 工程行为 → 代码中的验证”的顺序展开。

每个四边形在物理空间  $(x, y)$  的节点坐标  $[x_i, y_i]$  通过**双线性形函数**映射到母单元  $(\xi, \eta) \in [-1, 1]^2$  (Bathe Eq. 5.39 的等参思想):

$$N_i(\xi, \eta) = \frac{1}{4}(1 + \xi\xi_i)(1 + \eta\eta_i), \quad i = 1, \dots, 4, \quad \mathbf{x} = \sum_{i=1}^4 N_i(\xi, \eta) \mathbf{x}_i \quad (9)$$

1. **Kronecker 性质:**  $N_i(\xi_j, \eta_j) = \delta_{ij}$ ——节点值即形函数在该节点的值, 边界条件可直接施加在节点上;
2. **单位分解** (partition of unity):  $\sum_i N_i \equiv 1$ ——保证刚体平移  $\mathbf{u} = \mathbf{c}$  被精确再现 (零应变, 零内力);
3. **线性完备性:**  $\sum_i N_i x_i = x$ 、 $\sum_i N_i y_i = y$ ——等参映射自身由形函数定义, 故恒成立; 对位移场则保证任意线性位移场 (含常应变场) 被精确再现。这是**分片试验**能通过的充分条件。

第 2、3 条是”单元在极限网格下收敛”的理论保障 (Bathe §4.4.2), 本项目在 `Q4Element.verify_mesh()` 中用数值检查把它们锁死 (见 §6.3 代码讲解)。

**协调单元**的定义 (Bathe §4.4.1): 位移场在单元内连续、且**跨单元边界连续**——整体位移场  $\mathbf{u}^h$  属于  $C^0$ 。Q4 满足这条性质的论证基于一个观察: 形函数沿单元边退化为一维线性插值, 只依赖该边上的两个节点。

以母单元  $\xi = +1$  边 (连接节点 2、3) 为例, 代入式 (9):

$$N_2(1, \eta) = \frac{1+\eta}{2}, \quad N_3(1, \eta) = \frac{1-\eta}{2}, \quad N_1(1, \eta) = N_4(1, \eta) = 0 \quad (10)$$

因此该边上的位移插值为  $\mathbf{u}(\eta) = \frac{1+\eta}{2} \mathbf{u}_2 + \frac{1-\eta}{2} \mathbf{u}_3$ ——它是关于边上位置参数的**线性函数**, 完全由端点值  $\mathbf{u}_2, \mathbf{u}_3$  决定。对另外三条边 ( $\xi = -1, \eta = \pm 1$ ) 作同样的代入得到相同结论: **每条边上的插值只由该边两个端点的节点值唯一确定**。

现在看两个相邻单元  $e_1, e_2$  共享一条边  $e$ 。这条边在两个单元的局部编号中方向可能相反 ( $e_1$  里是  $i \rightarrow j$ ,  $e_2$  里是  $j \rightarrow i$ ), 但两个单元的插值都只能给出“在  $e$  上随物理位置线性变化、端点值分别为  $\mathbf{u}_i, \mathbf{u}_j$ ”的同一个函数——参数化方向只改变  $t$  的走向, 不改变函数值。于是

$$\mathbf{u}^h|_e^{(e_1)} = \mathbf{u}^h|_e^{(e_2)} = (1-t) \mathbf{u}_i + t \mathbf{u}_j, \quad t \in [0, 1] \quad (11)$$

**跨边位移连续**——这就是“协调”二字的确切含义。代码层面, `verify_mesh()` 中的单位分解检查之所以能间接锁住这条性质, 正是因为这个证明的逆否: 若某条边上形函数之和不为 1, 则相邻单元在该边上的插值必然分裂 ( $C^0$  被破坏)。

1. **弱形式中的单元间边界项消失**: 虚功原理 (2) 中的内力虚功在单元公共边上的贡献由牵引矢量  $\mathbf{t} = \boldsymbol{\sigma} \mathbf{n}$  给出。位移虚位移在公共边上连续 (式 (11)), 两侧单元贡献的虚功恰好相互抵消——这是 §4.6.2 牵引跳跃在收敛极限下  $\llbracket \mathbf{t} \rrbracket \rightarrow \mathbf{0}$  的同一机理。于是总刚度阵就是各单元刚度阵的**简单叠加**, 无需任何边界修正项——协调性是“组装”这个操作合法的前提;
2. **应力跨边不连续**:  $C^0$  只约束位移, 不约束导数。相邻单元的应变 (位移一阶导) 在共享边上可以不同——这正是 §4.6.2 应力跳跃 (牵引跳跃) 存在的物理根源: 协调单元保证位移连续, 不保证应力连续, 误差评估必须度量这条跳跃 (等应力带法 §4.10 的判读基础);
3.  **$C^0$  族与板壳的界限**: 要求导数连续 ( $C^1$ ) 的单元 (如 Kirchhoff 板) 需要更复杂的插值 (Hermite 形函数、转动自由度)。本项目全部单元 (CST/Q4/Q4R/Q4I) 均为  $C^0$  平面单元, 因此**不适用于**要求  $C^1$  的经典板弯曲问题——这是单元族的固有边界, 不是实现缺陷。

位移插值  $\mathbf{u} = \mathbf{N} \mathbf{d}^{(e)}$  代入几何方程得应变-位移矩阵 (Bathe Eq. 4.24)。关键一步是**导数链式法则**: 形函数定义在自然坐标, 而应变需要物理坐标导数, 故

$$\frac{\partial N_i}{\partial x} = \mathbf{J}^{-1} \frac{\partial N_i}{\partial \xi}, \quad \mathbf{J} = \begin{bmatrix} \partial x / \partial \xi & \partial y / \partial \xi \\ \partial x / \partial \eta & \partial y / \partial \eta \end{bmatrix} = \sum_{i=1}^4 \begin{bmatrix} (\partial N_i / \partial \xi) x_i & (\partial N_i / \partial \xi) y_i \\ (\partial N_i / \partial \eta) x_i & (\partial N_i / \partial \eta) y_i \end{bmatrix} \quad (12)$$

q4.py 中 `B_matrix()` 正是这个公式的直接实现：先算  $\partial N_i/\partial \xi$ 、 $\partial N_i/\partial \eta$ （矩阵乘法），再 `np.linalg.solve(jacobian, dN_nat)` 完成链式法则。

刚度阵 (5) 在母单元上写成 (Bathe Eq. 5.39)：

$$\mathbf{k}^{(e)} = \int_{-1}^1 \int_{-1}^1 \mathbf{B}^T \mathbf{D} \mathbf{B} t \det \mathbf{J} d\xi d\eta \approx \sum_{i=1}^2 \sum_{j=1}^2 w_i w_j \mathbf{B}(\xi_i, \eta_j)^T \mathbf{D} \mathbf{B}(\xi_i, \eta_j) t \det \mathbf{J}(\xi_i, \eta_j) \quad (13)$$

对平行四边形 ( $\det \mathbf{J}$  常数)，被积函数是  $\xi$ 、 $\eta$  的二次多项式（形函数双线性  $\times$  双线性 = 二次， $\det \mathbf{J}$  为常数）， $2 \times 2$  高斯点（每维 2 点）可精确积分 3 阶以内多项式——恰好足够，故称“全积分” (Bathe §5.7.4)。对任意扭曲四边形， $\det \mathbf{J}$  是双线性函数，被积函数升到四阶， $2 \times 2$  只能近似——这是等参单元的固有离散误差，不是实现缺陷。

$1 \times 1$  单点积分则不足：它只精确积分线性多项式，二次项被“采样到零”——这正是 §4.3.3 (Q4R) 沙漏模式的根源。

全积分 Q4 在纯弯曲工况下会过刚：单元位移场是双线性的，无法表示纯弯曲所需的  $\sigma_x \sim y$  应力分布，只能“借用”剪切变形来凑——而这些虚假的寄生剪切变形 (parasitic shear) 在粗网格下贡献了相当大的虚假剪切能，使弯曲刚度被人为抬高 (Bathe §5.5.1 的锁定讨论)。本项目 E1 悬臂梁实测：Q4 挠度误差随加密  $12.9\% \rightarrow 3.2\% \rightarrow 0.38\%$ ，比 CST ( $32.5\% \rightarrow 0.17\%$ ) 同阶收敛但粗网格误差更小；消除锁定的两条工程路线见下——Q4R 单点积分 (§4.3.3) 与 Q4I 非协调模式 (§4.3.4)。

q4.py 的两处实现细节体现了本项目“微尺度鲁棒性”纪律（全部有测试锁定）：

- **局部坐标**：`element_stiffness()` 先把单元坐标平移到以第一个节点为原点再计算——绝对坐标在  $10^{12}$  量级时， $\mathbf{B}^T \mathbf{D} \mathbf{B}$  的浮点乘加引入  $\sim \text{ulp}(\text{原点})$  的相对误差，实测刚度偏差  $\sim 9 \times 10^{-5}$ ；
- **Jacobian 逐点检查**：`build_geometry()` 在  $2 \times 2$  高斯点之外额外检查 5 个点（4 角点 + 中心）的  $\det \mathbf{J}$ ，任何非正值立即拒绝 (`stiffness_batch()` 与标量路径 `element_stiffness()` 双侧守卫)——扭曲过度的单元在组装前就被抓住，而不是让奇异矩阵在求解器里爆炸；
- **退化度量**：`degeneracy_measure() = 面积/最长边`  $\sim 2$ （无量纲）——矩形退化为长宽比的倒数，扭歪再乘  $\sin \theta$ ，数学塌缩级 ( $< 10^{-8}$ ) 判为退化，与单元绝对尺寸无关。

#### 4.3.3 Q4R——单点积分 + 沙漏控制（本章重点）

Q4 的剪切锁定源自全积分对低阶位移场“太诚实”；反过来把积分降到  $1 \times 1$  单点，刚度阵变软、锁定消失，但会引入更危险的问题——零能量模式（沙漏）。Q4R 是两者的折中：单点积分 + 稳定化。本节按“为什么会有沙漏  $\rightarrow$  稳定化公式  $\rightarrow$  几何感知的构造  $\rightarrow$  工程限制”展开。

$1 \times 1$  高斯点在单元中心  $(\xi, \eta) = (0, 0)$  只有一个采样点。代入式 (13) 后, 被积函数中所有关于  $(\xi, \eta)$  奇次的项在中心处取零值——被积函数只在中心被评估一次, 这些奇次项“逃过”了积分。位移场上对应存在非仿射零能模式。零能条件的定量刻画如下: 单点积分下应变能正比于中心应变  $\epsilon(0) = \mathbf{B}(0, 0) \mathbf{u}_e$ , 零能模式即  $\mathbf{B}(0, 0) \mathbf{u}_e = \mathbf{0}$  的解。 $\mathbf{B}(0, 0)$  是  $3 \times 8$  矩阵、秩 3, 故零空间维数  $8 - 3 = 5$ : 其中 3 个是刚体模态 (零应变, 物理允许), 多出的 2 个是虚假模式—— $u$ 、 $v$  两个位移方向各一个, 型态为

$$\mathbf{g} = [+1, -1, +1, -1]^T \quad (\text{节点位移型态} \Leftrightarrow \text{插值场 } u = \xi\eta) \quad (14)$$

代入形函数:  $u = N_1 - N_2 + N_3 - N_4 = \xi\eta$ , 中心处  $\partial u / \partial \xi = \partial u / \partial \eta = 0$ , 即  $\mathbf{B}(0, 0) \mathbf{g} = \mathbf{0}$ ——中心应变恰好为零, 不产生任何应变能: 刚度矩阵对它们给出零特征值, 矩阵秩亏。如果不加控制, 求解器对它们没有任何阻力: 位移解可以叠加上任意大的沙漏幅值而不改变能量, 网格呈现出交替错动的“沙漏”变形 (hourglass pattern), 应力云图被污染。这就是 Abaqus 文档中 CPS4R 必须配 hourglass stiffness 的原因。

**注意区分:** 型态  $[-1, +1, +1, -1]$  (插值场  $u = \xi$ ) 在中心处  $\partial u / \partial \xi = 1 \neq 0$ ——它是有能量的线性 (仿射) 拉伸模式, 不是沙漏。沙漏的本质是非仿射。这从  $2 \times 2$  全积分一侧看得更清楚: 全积分对  $\xi, \eta$  的二次多项式精确 (每维 2 点), 双线性位移场的应变是双线性函数, 故  $\mathbf{B}^T \mathbf{D} \mathbf{B}$  被精确积分; 若  $\mathbf{u}_e$  使  $\mathbf{B}(\xi, \eta) \mathbf{u}_e \equiv \mathbf{0}$  在单元内处处成立, 则  $\mathbf{u}_e$  只能是刚体运动 (3 维)——全积分 Q4 没有沙漏。单点积分把零能条件退化为“仅在中心成立”, 解空间从 3 维扩到 5 维, 多出的 2 个  $\xi\eta$  型模式就是沙漏: 采样点越少, 零能模式越多 ( $1 \times 1 \rightarrow 2$  个;  $2 \times 2 \rightarrow 0$  个)。Q4R 走第三条路: 保留单点积分的廉价与无锁定, 用稳定项给这 2 个模式重新装上能量 (见下)。

Q4R 刚度 = 单点积分刚度 + 稳定项 (本实现为仿射补投影稳定, affine-projector stabilization):

$$\mathbf{k} = \mathbf{k}_1 + \alpha \mathbf{H} (\mathbf{H}^T \mathbf{K}_{shear} \mathbf{H}) \mathbf{H}^T \quad (15)$$

三个组成部分的设计理由:

1.  $\mathbf{k}_1$ ——单点积分刚度: 只含仿射 (线性) 位移模式的能量。弯曲不再被寄生剪切锁死, 单元变软、对扭曲网格稳健;
2.  $\mathbf{H}$ ——仿射补的零空间基:  $\mathbf{H}$  的列张成“所有仿射节点位移场的正交补”。平移、转动、三个常应变场 (共 6 个仿射模式) 满足  $\mathbf{H}^T \mathbf{v}_{affine} = \mathbf{0}$ , 因此稳定项对它们是精确为零——稳定化不改变任何物理正确的结果, 只“罚”掉非仿射的沙漏方向;
3.  $\mathbf{K}_{shear}$ ——仅剪切模量的参考刚度: 用  $2 \times 2$  高斯点、只含剪切模量 ( $2\mu$  对角) 的本构构造的参考刚度。选剪切模量而非完整  $\mathbf{D}$  的原因是: 完整  $\mathbf{D}$  会把体积 (锁定) 方向也稳定进刚度, 重新引入锁定; 只有剪切项才能给稳定项正确的物理量纲 ( $N/m$ ) 而不加体积刚度。

$\alpha$  为沙漏系数 (默认 0.10), 是稳定项的强度旋钮: 过大  $\rightarrow$  单元重新变刚 (接近全积分行为); 过小  $\rightarrow$  沙漏未被充分抑制。

q4r.py 的 `_affine_complement()` 用了一个优雅的代数技巧，不需要对每个单元做一次  $QR$  分解：仿射位移场的四个分量  $[1, x_i, y_i]^T$  构成  $4 \times 3$  矩阵，其零空间的标量基向量恰是  $[1, x_i, y_i]^T$  的余子式 (cofactors)。对四个节点  $i = 1..4$  排成循环序，该向量由三角形两倍面积 ( $\vec{ab} \times \vec{ac}$ ) 直接给出：

$$h_i = \frac{1}{\|\mathbf{h}\|} \begin{cases} 2A(1, 2, 3) \\ -2A(0, 2, 3) \\ 2A(0, 1, 3) \\ -2A(0, 1, 2) \end{cases} \quad (16)$$

其中  $2A(a, b, c)$  是以节点  $a, b, c$  为顶点的三角形两倍面积 (叉积)。与“完整  $8 \times 6$   $QR$  分解取正交补”数学上等价，但每单元只花  $\sim 12$  次乘加。这个技巧同时带来几何感知： $h_i$  随单元形状变化，平行四边形单元给出精确的  $[1, -1, 1, -1]$ ，扭曲单元自动调整方向——与“沙漏刚度必须随几何缩放”的工程经验一致。坐标居中/缩放只改变条件数，不改变仿射子空间本身。

稳定项虽然抑制了沙漏，但沙漏模式并未完全消失——它现在携带小能量 ( $\sim \alpha$  量级)。求解器必须监控它：`hourglass_energy()` 把位移投影到  $\mathbf{H}$  上计算稳定能：

$$E_{HG}^{(e)} = \frac{1}{2} \mathbf{u}_e^T (\alpha \mathbf{H} \mathbf{H}^T \mathbf{K}_{shear} \mathbf{H} \mathbf{H}^T) \mathbf{u}_e \quad (17)$$

两个实现细节值得注意：

- **材料指纹失效检查：** $\mathbf{H}^T \mathbf{K}_{shear} \mathbf{H}$  依赖  $E, \nu, plane$ 。若装配后改材料/平面态，几何缓存不会自动失效——代码记录 (`E, nu, thickness, plane_type`) 指纹，不匹配时按当前材料重算 (历史教训：`plane` 切换后沙漏能沿用旧  $\mathbf{D}$ ，积分点应力相对差  $\sim 9\%$ )；
- **负能告警而非截零：**稳定项构造上半正定，负能量只可能来自负系数或实现错误。代码对显著负能发 `RuntimeWarning` 而不是静默 `max(energy, 0)`——无条件截零会把实现错误掩盖成“正常” (纪律：错误不会静默)。

求解完成后 `solve()` 报告 `hourglass_energy_ratio` (沙漏能/内能)。但注意——沙漏能占比不是可靠性的判据 (实测：长宽比  $\geq 50$  或单排网格下，稳定项主导时占比反而  $\sim 90\%$ ，而解已经严重过刚；中等长宽比下沙漏主导时占比也  $> 90\%$ )。判据改用单元长宽比在求解前告警 (`_q4r_aspect_ratio_warning()`，见 §6.8)。

本实现的 compact 稳定公式 (稳定尺度  $\mathbf{H}^T \mathbf{K}_{shear} \mathbf{H}$  不随长宽比衰减，而单点弯曲刚度随  $(h/L)^3$  衰减)：

- 长宽比  $\geq 50$  或单排网格：稳定项主导  $\rightarrow$  解严重过刚 (实测悬臂端挠度只有解析值的  $2\%$ )；
- 中等长宽比 ( $L/h \sim 10$ 、少排网格)：沙漏主导  $\rightarrow$  过软 (实测  $\sim 5\times$  解析挠度)；
- 常规多排网格 (长宽比  $< 10$ )：分片试验、悬臂  $\sim 1.0$ 、SCF、收敛率全部正确验证。

**薄板/细长模型优先选 Q4I** (无稳定项，见下)。商业软件用增强假设应变/自适应缩放后适用范围更宽——这是本实现 compact 公式的理论特性，不是编码错误。



| 单元  | 积分                 | 弯曲行为        | 适用场景                 |
|-----|--------------------|-------------|----------------------|
| Q4  | $2 \times 2$ 全积分   | 粗网格剪切锁定（过刚） | 规则网格、教学基准、误差估计研究     |
| Q4R | $1 \times 1$ + 稳定  | 锁定消失，细长网格失真 | 大型模型（快）、扭曲网格、粗网格工程分析 |
| Q4I | $2 \times 2$ + 非协调 | 无锁定、无稳定项    | 薄板/细长弯曲问题（首选）        |

#### 4.3.4 Q4I——QM6 非协调单元

QM6 (Taylor 1976, Bathe §5.5.1) 在 Q4 位移场中加入**非协调模式**消除弯曲锁定的寄生剪切。非协调位移导数  $\mathbf{B}_{inc}$  使单元刚度扩展为分块形式，内部自由度通过**静力凝聚**消除 (Bathe Eq. 4.65 的静态凝聚流程)：

$$\begin{bmatrix} \mathbf{k}_{dd} & \mathbf{k}_{d\lambda} \\ \mathbf{k}_{\lambda d} & \mathbf{k}_{\lambda\lambda} \end{bmatrix} \begin{bmatrix} \mathbf{d} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ \mathbf{0} \end{bmatrix} \implies \mathbf{k}_{cond} = \mathbf{k}_{dd} - \mathbf{k}_{d\lambda} \mathbf{k}_{\lambda\lambda}^{-1} \mathbf{k}_{\lambda d} \quad (18)$$

性质：在保持 Q4 简单性的同时大幅改善弯曲响应，是工程弯曲问题的首选四边形单元；实现复杂度最高（凝聚分块），见 `element/q4i.py`。

### 4.4 边界条件施加

#### 4.4.1 消去法 (elimination) ——默认

将自由度划分为自由集  $a$  与约束集  $b$  (Bathe §4.2.2 Eq. 4.40)：

$$\begin{bmatrix} \mathbf{K}_{aa} & \mathbf{K}_{ab} \\ \mathbf{K}_{ba} & \mathbf{K}_{bb} \end{bmatrix} \begin{bmatrix} \mathbf{u}_a \\ \mathbf{u}_b \end{bmatrix} = \begin{bmatrix} \mathbf{F}_a \\ \mathbf{F}_b \end{bmatrix} \implies \mathbf{u}_a = \mathbf{K}_{aa}^{-1} (\mathbf{F}_a - \mathbf{K}_{ab} \bar{\mathbf{u}}_b) \quad (19)$$

约束自由度从系统中消去，反力由第二行回代求得： $\mathbf{R}_b = \mathbf{K}_{ba} \mathbf{u}_a + \mathbf{K}_{bb} \bar{\mathbf{u}}_b - \mathbf{F}_b$ 。实现为 `bc.apply_elimination()`，支持 `direct/cg/cg-block/ilu` 四种线性求解器分支。纯 **Dirichlet** 情形（自由集为空）特判直接返回，不调用求解器。

#### 4.4.2 罚方法 (penalty)

约束自由度对角元加上罚刚度（**加法式**；历史教训：乘法式会把刚度平方，条件数灾难）：

$$K_{ii} += \alpha, \quad \alpha = \max_i |K_{ii}| \times 10^8 \quad (\text{自动}); \quad F_i = \alpha \bar{u}_i \quad (20)$$

罚因子下限受相对判据保护（ $< \max |K_{ii}| \times 10^4$  报错），溢出风险有显式守卫。约束是近似的（ $10^{-8}$  相对量级），用于教学对比；默认推荐消去法（精确 + 反力干净）。

### 4.4.3 刚体模态检查

求解前逐连通分量检查约束是否抑制全部刚体模态（2 平动 + 1 转动，rank 检查）。约束矩阵用中心化坐标构造（逐分量中心化 + 归一化；历史教训：大坐标下直接 SVD 会误判）：

$$\mathbf{R}_i = \begin{bmatrix} 1 & 0 & -y'_i \\ 0 & 1 & x'_i \end{bmatrix}, \quad x'_i = x_i - \bar{x}, \quad y'_i = y_i - \bar{y} \quad (21)$$

欠约束（rank < 3）直接 `RuntimeError` 拒绝求解，绝不静默给出无意义解。

## 4.5 应力计算与恢复

### 4.5.1 单元应力

求解得节点位移  $\mathbf{u}$  后，单元/积分点应力：

$$\boldsymbol{\sigma} = \mathbf{D} \mathbf{B} \mathbf{d}^{(e)} \quad (22)$$

单元间应力不连续（弱形式只保证位移连续）。后处理提供三种“磨平”方案把单元应力恢复为连续节点场。

### 4.5.2 节点平均（nodal simple / weighted）

单元应力按算术平均或面积加权平均到节点。代价最低，但峰值被周围低值拉低（CST 磨平后峰值 ~17% 下降，属预期，文档有记录）。

### 4.5.3 SPR——超收敛点恢复

SPR（Zienkiewicz-Zhu 1992, Bathe §4.3.6）对每个节点的单元补丁做局部最小二乘拟合：

$$\boldsymbol{\sigma}^*(\mathbf{x}) = \mathbf{P}(\mathbf{x}) \mathbf{a}, \quad \mathbf{P} = \begin{bmatrix} 1 & x & y \end{bmatrix}, \quad \mathbf{a} = \arg \min \sum_{e \in \text{patch}} \|\boldsymbol{\sigma}^h - \mathbf{P} \mathbf{a}\|^2 \quad (23)$$

恢复值取拟合多项式在节点处的值。纪律：必须先恢复分量  $\sigma_x, \sigma_y, \tau_{xy}$ ，再从恢复分量计算 Mises/主应力——SPR 是线性算子，与非线性函数不可交换（历史教训：SPR(f(Mises))  $\neq$  f(SPR(Mises))）。

### 4.5.4 L2 投影

全局一致质量阵意义下的最小二乘投影：

$$\left( \int_{\Omega} \mathbf{N}^T \mathbf{N} d\Omega \right) \boldsymbol{\sigma}^* = \int_{\Omega} \mathbf{N}^T \boldsymbol{\sigma}^h d\Omega \quad (24)$$

全局求解（解一致质量阵），精度最高、代价最大；已向量化（300k 单元 12.8s  $\rightarrow$  4.2s，3 倍提速，逐位一致）。



## 4.6 误差估计

### 4.6.1 Z2 能量误差估计器（简要）

Z2 估计器（Zienkiewicz–Zhu 1987, Bathe §4.3.6 Eq. 4.108）用恢复场  $\sigma^*$ （SPR/L2）与有限元场  $\sigma^h$  之差的能量范数估计误差：

$$\eta = \frac{\left[ \int_{\Omega} (\sigma^* - \sigma^h)^T \mathbf{D}^{-1} (\sigma^* - \sigma^h) d\Omega \right]^{1/2}}{\|\sigma^*\|} \times 100\% \quad (25)$$

实现细节（恢复源选择、SPR 的组件先恢复纪律）见 §4.5 与 §6.9.1；h-加密效应指数  $\theta \approx 1$  的验证见 §7.4 E3。误差估计不是本求解器的理论重心——它是**网格加密的指引工具**，不是精度保证。

### 4.6.2 应力跳跃——牵引跳跃

相邻单元应力不连续的正确度量是**内部边上的牵引跳跃**（Bathe Eq. 4.107 的直接推论）：

$$[\mathbf{t}] = (\sigma^+ - \sigma^-) \mathbf{n} \quad (26)$$

（分量差  $|\sigma_{x,1} - \sigma_{x,2}|$  依赖坐标系取向且无物理意义，历史教训。）

`compute_traction_jumps()` 实现之，边界边用 Neumann 牵引残差  $\sigma \mathbf{n} - \bar{\mathbf{t}}$  度量；归一化用  $|\Delta t|/\sigma_{ref}$  避免过零虚高（ $|s_1 - s_2|/\max$  公式在正负交界处饱和到 200% 的教训）。

## 4.7 分片试验与收敛性

**分片试验**（patch test, Bathe §4.4.2）：构造满足常应变/刚体模态的位移场施加到任意网格，验证单元能否精确再现——线性完备性的标准检验。`run_patch_test()` 对每个注册单元执行，是内核注册的必经门槛（实现走读见 §6.9.3）。

再看**收敛性**：线性单元的能量范数误差  $\|e\| \sim Ch$ （一阶）。

`run_cantilever_convergence()` 用光滑解（Timoshenko 抛物线剪流梁，无点载荷奇异性）做多层双向加密收敛研究，输出误差随  $h$  的衰减率，拟合斜率应  $\approx 1$ （与解析解包 E3 互相印证）。

分片试验不必是抽象声明——一个扭曲四边形就能手算到底。取**梯形单元**（非平行四边形，故意破坏”规则几何”）：

$$\mathbf{x}_1 = (0, 0), \quad \mathbf{x}_2 = (2, 0), \quad \mathbf{x}_3 = \left(\frac{5}{2}, 1\right), \quad \mathbf{x}_4 = (0, 1) \quad (27)$$

施加单向拉伸常应变场  $u = x, v = 0$ （ $\varepsilon_x = 1$ ，其余分量为零）。节点位移由场值直接给出： $u_e = [0, 2, 5/2, 0]^T, v_e = \mathbf{0}$ 。

**第一步——插值精确性**。由线性完备性（形函数第三条性质）， $u^h(\xi, \eta) = \sum_i N_i u_i = \sum_i N_i x_i = x(\xi, \eta)$ ，插值场与精确场在**每个点**相等。这一步不用任何数值：它就是” $\sum_i N_i x_i = x$  恒成立”的直接代入。注意完备性对**任意扭曲四边形**成立——分片试验不要求单元是平行四边形。

第二步——应变恒为常数。梯度由链式法则给出：

$$\begin{bmatrix} \partial u^h / \partial x \\ \partial u^h / \partial y \end{bmatrix} = \mathbf{J}^{-1} \begin{bmatrix} \partial u^h / \partial \xi \\ \partial u^h / \partial \eta \end{bmatrix} = \mathbf{J}^{-1} \begin{bmatrix} \partial x / \partial \xi \\ \partial x / \partial \eta \end{bmatrix} = \mathbf{J}^{-1} \mathbf{J}_{(:,1)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad (28)$$

第二列 ( $v^h = 0$ ) 同理给出零梯度。关键的一步是  $\mathbf{J}^{-1} \mathbf{J}_{(:,1)}$ ：任意  $(\xi, \eta)$  处、无论  $\mathbf{J}$  怎么扭曲，恒等于第一列单位向量——因此  $\varepsilon = [1, 0, 0]^T$  在单元内每一点精确成立， $\mathbf{B}$  矩阵代入节点位移在  $2 \times 2$  高斯点处给出完全相同的应变。这正是“ $\mathbf{B}$  矩阵在扭曲单元上随点变化、但  $\mathbf{B}\mathbf{u}_e$  不变”的机理：对精确的线性场， $\mathbf{B}$  的变化与位移插值的变化精确抵消。

第三步——内力自平衡。常应力  $\boldsymbol{\sigma} = \mathbf{D}\boldsymbol{\varepsilon}$  对应的节点力  $\mathbf{f}^{(e)} = \int \mathbf{B}^T \boldsymbol{\sigma} t dV$  之和：由散度定理， $\sum_i \mathbf{f}_i^{(e)} = \int_{\Gamma} \boldsymbol{\sigma} \mathbf{n} t ds$ ——常应力在闭合边界上的积分恒为零（本例梯形四边上的面力相互抵消）。于是该场无需任何外载即可保持平衡： $\mathbf{K}\mathbf{u}_e = \mathbf{f}^{(e)}$  精确成立，内部节点（若剖分后存在）自动得到精确位移。

结论：Q4 对任意扭曲几何精确通过分片试验——这正是“协调 + 完备”的联合效应：协调性保证跨边连续 (§4.3.1 的证明)，完备性保证常应变精确再现。本项目 `run_patch_test()` 对每个注册单元数值验证的正是这件事；Q4I 等非协调单元通过分片试验（而非协调性）获得收敛性的理论依据，这是两类单元的根本区别。

## 4.8 网格质量指标

`quality.py` 对网格单元做 bad/warn/ok 三级互斥分类（评分：角度 OK 比例  $\times 40$  + 长宽比 OK 比例  $\times 30$  + Jacobian OK 比例  $\times 30$ ，warn/bad 记 0 分，合计 100）：互斥性是硬要求——bad 与 warn 条件不得重叠，否则评分失真（历史教训：半开区间写错导致双重计数）。

| 指标  | bad              | warn                     | ok            |
|-----|------------------|--------------------------|---------------|
| 最小角 | $\leq 15^\circ$  | $(15^\circ, 30^\circ]$   | $> 30^\circ$  |
| 最大角 | $\geq 150^\circ$ | $[120^\circ, 150^\circ)$ | $< 120^\circ$ |
| 长宽比 | $\geq 5$         | $[3, 5)$                 | $< 3$         |

输出 `evaluate_mesh_quality()` 返回分类统计与加权评分，`report_mesh_quality()` 打印中文报告（质量等级 A/B/C）。实现走读见 §6.9.2。

## 4.9 载荷的有限元离散

四类载荷在弱形式 (2) 中对应不同的积分项，装配实现由 `loads_core.assemble()` 逐类完成：

### 4.9.1 集中力

节点集中力直接叠加到全局载荷向量的对应自由度 ( $\mathbf{F}_c = \sum_i \mathbf{N}(x_i)^T \mathbf{P}_i$ ——由于形函数的 Kronecker 性质，等价于在节点  $i$  的自由度上直接相加)。

### 4.9.2 边界分布面力与压力

边  $\Gamma$  上的面力  $\bar{\mathbf{t}}$  的等效节点载荷 (Bathe Eq. 4.31 的边界项):

$$\mathbf{F}_s^{(e)} = \int_{\Gamma^{(e)}} \mathbf{N}^T \bar{\mathbf{t}} d\Gamma \quad (29)$$

沿边的线积分用边上的等参一维高斯积分 (边  $L$  上以弧长参数  $s$ ):

$$\mathbf{F}_s^{(e)} = L \sum_g w_g \mathbf{N}(\xi_g)^T \bar{\mathbf{t}}(s(\xi_g)) \quad (30)$$

支持三类分布 **profile** (`--traction` 语法 `edge:tx,ty[:p|l|n]`, 见 §3.3): 线性 (l)、抛物线 (p) 按边端点归一化坐标插值; 法向压力 (n) 取  $p$  为压强, 方向由单元外法向确定 (`boundary_outward_normal()`, 历史教训: 法向必须从相邻单元 CCW 方向确定, 不依赖调用者节点顺序)。

`loads_core.py` 的 `assemble()` 把每条 `surface_tractions` 记录逐边处理, 核心是 **3 点一维 Gauss-Legendre** (模块 docstring 写明: 形函数沿直边退化为线性, 常数/线性面力 2 点已精确, 3 点是为将来二次分布保留精度余量):

```
LINE_GAUSS = [(5/9, -0.774596669241483), # xi = -sqrt(3/5)
               (8/9, 0.0),
               (5/9, 0.774596669241483)] # xi = +sqrt(3/5)
```

边的参数化与批量累积 (`_accumulate_surface_batch` 的核心三行):

```
dofs = np.stack([2*lo, 2*lo+1, 2*hi, 2*hi+1], axis=1)
fe = thickness * _LGAUSS_W[None, :] * L[:, None] / 2.0
np.add.at(F, np.repeat(dofs, 3, axis=0).ravel(), contrib.ravel())
```

数学上  $fe = t * (L/2) * w_k * N_i(xi_k) * t_k$ :  $(L/2)$  是弧长参数化的雅可比 ( $ds = (L/2)dxi$ ),  $N_i$  是边两端形函数 ( $(1 \pm xi)/2$ ), 三个 Gauss 点各贡献一次、按边两端 4 个自由度叠加。**求和顺序逐位一致**: 批量路径按”记录序  $\times$  Gauss 点序  $\times$  DOF 序”展开, 与逐点路径的浮点求和顺序完全相同——P- $\delta$  性能包的逐位断言锁定 (`tests/test_p_delta_loads_batch.py`), 这是”向量化不得改变数值”纪律的实例。

**压力法向不信任调用者**: `is_pressure` 记录施加  $\mathbf{t} = -p\mathbf{n}$ , 外法向由相邻单元的局部 CCW 边匹配得到 (`_pressure_normals_batch` 遍历 `kernel.local_edges`,  $\mathbf{n} = (dy/L, -dx/L)$ ) ——调用者写 (a,b) 还是 (b,a) 都得到同一外法向 (历史教训, §6.7.4 约定)。

**每边四道校验** (`_classify_edges_batch`): ①节点 id 合法; ②该边必须是网格边 (否则”is not a mesh edge”); ③必须是**边界边** (内部边报”interior edge shared by 2 elements”, 面力只支持边界边——载荷加在内边上属于工况错误, 必须响亮拒绝); ④退化边 (长度  $\leq 64\epsilon \max |x|$  判据, 微尺度模型不会被误判, §10 教训)。callable 面力逐 Gauss 点求值, NaN/Inf 带边号拒绝。

### 4.9.3 体力

体力  $\mathbf{f}(x, y)$  的等效节点载荷:

$$\mathbf{F}_b^{(e)} = \int_{\Omega^{(e)}} \mathbf{N}^T \mathbf{f}(x, y) d\Omega = t \sum_g w_g \mathbf{N}(\xi_g)^T \mathbf{f}(x(\xi_g), y(\xi_g)) \det \mathbf{J}(\xi_g) \quad (31)$$

--body bx,by 支持空间函数表达式 (AST 白名单: 变量  $x, y$ , 函数 sin, cos, tan, exp, sqrt, log, abs 与常量  $\pi$ ), 在真实高斯点求值。

### 4.10 等应力带法 (isoband)

应力不连续性的定性判读工具 (Bathe §4.3.6 + Sussman–Bathe "stress band plots")。三个关键实现要求:

1. 带宽固定 (如 5 MPa): 不能随数据范围自动变化——否则粗/细网格带宽不同无法比较;
2. 离散着色: 每带一个颜色, 非连续渐变——否则色条与离散单元着色不一致;
3. 定性判读: 带的不连续断裂程度反映网格误差, 是目视工具而非定量公式 (定量用 Z2)。

visualize 的 isoband 输出即按此实现, 与 Gouraud 连续云图互补 (实现走读见 §6.9.4)。

### 4.11 一个完整的手算示例: CST 单元拉伸

以最简单的模型演示全部公式: 单位厚度正方形  $[0,1] \times [0,1]$ , 单个 CST 单元 (节点  $0=(0,0)$ 、 $1=(1,0)$ 、 $2=(0,1)$ ),  $E=1$ 、 $\nu=0$ ,  $x$  方向均匀拉伸  $\sigma_x = 1$  (等价于  $u(1,0)$  处施加  $F = 0.5$ )。

第一步——形函数 (面积  $A = 0.5$ ,  $2A = 1$ ):

$$N_0 = 1 - x - y, \quad N_1 = x, \quad N_2 = y \quad (32)$$

第二步——B 矩阵 ( $\partial N_i / \partial x$  与  $\partial N_i / \partial y$ ):

$$\mathbf{B} = \begin{bmatrix} -1 & 0 & 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 1 \\ -1 & -1 & 0 & 1 & 1 & 0 \end{bmatrix} \quad (33)$$

第三步——本构 ( $\nu = 0$  时  $\mathbf{D} = \text{diag}(1, 1, 0.5)$ ), 单元刚度  $\mathbf{k} = tA\mathbf{B}^T\mathbf{D}\mathbf{B}$  ( $t = 1, A = 0.5$ )。

第四步——装配与求解: 约束  $u_{0x} = u_{0y} = u_{1y} = 0$ , 施加  $F_{1x} = 0.5$ , 消去法得  $u_{1x} = 1$ 。

第五步——应力:  $\boldsymbol{\sigma} = \mathbf{D}\mathbf{B}\mathbf{u} = (1, 0, 0)^T$ ——精确恢复均匀应力场, 这就是分片试验能通过的原因: CST 能精确再现常应力。

此例可在源码中直接运行复现:

```

import numpy as np
import fem2d
nodes = np.array([[0.,0.],[1.,0.],[0.,1.]])
elems = np.array([[0,1,2]])
m = fem2d.Mesh(nodes, elems, E=1.0, nu=0.0, thickness=1.0)
m.fix_node(0, "both")
m.fix_node(1, "y")
m.add_force(1, 0.5, 0.0)
res = fem2d.solve(m)
print(res["u"][2])          # u_x(1,0) = 1.0
stress, _, _ = fem2d.compute_stresses(m, res["u"])
print(stress)               # [[1., 0., 0.]] (sigma_x, sigma_y,
                             tau_xy)

```

#### 4.12 手算示例：Q4 协调单元（2×2 高斯积分）

与 §4.11 的 CST 相对照，本小节把 Q4 单元的完整计算过程逐步行算一遍。模型同样是单位厚度（ $t = 1$ ）正方形  $[0, 1] \times [0, 1]$ ，节点  $0=(0,0)$ 、 $1=(1,0)$ 、 $2=(1,1)$ 、 $3=(0,1)$ （逆时针），材料  $E=1$ 、 $\nu = 0$ ，平面应力（ $\mathbf{D} = \text{diag}(1, 1, 0.5)$ ）， $x$  方向均匀拉伸。

**第一步——等参映射与 Jacobian。**双线性形函数（ $\xi, \eta \in [-1, 1]$ ）：

$$N_i = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta), \quad \mathbf{x}(\xi, \eta) = \sum_{i=1}^4 N_i \mathbf{x}_i \quad (34)$$

对单位正方形， $\mathbf{x}_i = (0, 0), (1, 0), (1, 1), (0, 1)$ ，几何映射为

$$x = \frac{1}{2}(1 + \xi), \quad y = \frac{1}{2}(1 + \eta) \implies \mathbf{J} = \begin{bmatrix} \frac{1}{2} & 0 \\ 0 & \frac{1}{2} \end{bmatrix}, \quad \det \mathbf{J} = \frac{1}{4} \equiv \text{常数} \quad (35)$$

矩形单元是仿射映射，Jacobian 在单元内恒定——这是与任意四边形（ $\mathbf{J}$  随  $\xi, \eta$  变化）的唯一区别，令本手算大幅简化。

**第二步——高斯点与形函数导数。**2×2 高斯规则：积分点  $(\xi_q, \eta_q) = (\pm 1/\sqrt{3}, \pm 1/\sqrt{3}) \approx (\pm 0.5774, \pm 0.5774)$ ，每个积分点权重  $w_q = 1$ （合计 4，等于参数域面积）。四个高斯点处  $N_i$  对自然坐标的导数：

| $q (\xi_q, \eta_q)$    | $\partial N_i / \partial \xi$ |        |        |         | $\partial N_i / \partial \eta$ |         |        |        |
|------------------------|-------------------------------|--------|--------|---------|--------------------------------|---------|--------|--------|
|                        | $N_1$                         | $N_2$  | $N_3$  | $N_4$   | $N_1$                          | $N_2$   | $N_3$  | $N_4$  |
| 0 $(-0.5774, -0.5774)$ | -0.3943                       | 0.3943 | 0.1057 | -0.1057 | -0.3943                        | -0.1057 | 0.1057 | 0.3943 |
| 1 $(+0.5774, -0.5774)$ | -0.3943                       | 0.3943 | 0.1057 | -0.1057 | -0.1057                        | -0.3943 | 0.3943 | 0.1057 |
| 2 $(+0.5774, +0.5774)$ | -0.1057                       | 0.1057 | 0.3943 | -0.3943 | -0.1057                        | -0.3943 | 0.3943 | 0.1057 |
| 3 $(-0.5774, +0.5774)$ | -0.1057                       | 0.1057 | 0.3943 | -0.3943 | -0.3943                        | -0.1057 | 0.1057 | 0.3943 |

**第三步——B 矩阵。**  $\mathbf{J}^{-1} = 2\mathbf{I}$ , 故  $\partial N_i/\partial x = 2\partial N_i/\partial \xi$ 、 $\partial N_i/\partial y = 2\partial N_i/\partial \eta$ 。以  $q = 0$  为例（其余点按高斯点符号交替，见下表）：

$$\mathbf{B}(\xi_0, \eta_0) = \begin{bmatrix} -0.7887 & 0 & 0.7887 & 0 & 0.2113 & 0 & -0.2113 & 0 \\ 0 & -0.7887 & 0 & -0.2113 & 0 & 0.2113 & 0 & 0.7887 \\ -0.7887 & -0.7887 & -0.2113 & 0.7887 & 0.2113 & 0.2113 & 0.7887 & -0.2113 \end{bmatrix} \quad (36)$$

四点的应变矩阵行 1（对应  $\partial N_i/\partial x$ ）依次为

$$(-0.7887, 0.7887, 0.2113, -0.2113), \quad (-0.7887, 0.7887, 0.2113, -0.2113),$$

$$(-0.2113, 0.2113, 0.7887, -0.7887), \quad (-0.2113, 0.2113, 0.7887, -0.7887)$$

——只有  $\xi$  方向的符号分布不同，这就是矩形单元 B 矩阵的结构规律。

**第四步—— $2 \times 2$  高斯求和。** 单元刚度

$$\mathbf{k} = \sum_{q=1}^4 w_q t \det \mathbf{J} \mathbf{B}(\xi_q, \eta_q)^T \mathbf{D} \mathbf{B}(\xi_q, \eta_q) = \frac{1}{4} \sum_{q=1}^4 \mathbf{B}_q^T \mathbf{D} \mathbf{B}_q \quad (37)$$

四项之和（数值已由程序验证，读者可自行核对其中  $q = 0$  项： $0.25 \mathbf{B}_0^T \mathbf{D} \mathbf{B}_0$  对角元 0.2333, 0.2333, 0.1611, 0.0889, 0.0167, 0.0167, 0.0889, 0.1611）：

$$\mathbf{k}_{Q4} = \begin{bmatrix} 0.5 & 0.125 & -0.25 & -0.125 & -0.25 & -0.125 & 0 & 0.125 \\ 0.125 & 0.5 & 0.125 & 0 & -0.125 & -0.25 & -0.125 & -0.25 \\ -0.25 & 0.125 & 0.5 & -0.125 & 0 & -0.125 & -0.25 & 0.125 \\ -0.125 & 0 & -0.125 & 0.5 & 0.125 & -0.25 & 0.125 & -0.25 \\ -0.25 & -0.125 & 0 & 0.125 & 0.5 & 0.125 & -0.25 & -0.125 \\ -0.125 & -0.25 & -0.125 & -0.25 & 0.125 & 0.5 & 0.125 & 0 \\ 0 & -0.125 & -0.25 & 0.125 & -0.25 & 0.125 & 0.5 & -0.125 \\ 0.125 & -0.25 & 0.125 & -0.25 & -0.125 & 0 & -0.125 & 0.5 \end{bmatrix} \quad (38)$$

特征值：{0, 0, 0, 0.5, 0.5, 1, 1, 1}（程序验证 rank = 5）。三个零对应刚体平动  $x$ 、平动  $y$ 、转动——协调单元必然保留的三个零能模态。特别留意特征值 0.5 的两个特征向量：

$$\mathbf{v}_1 = \frac{1}{2}[1, 0, -1, 0, 1, 0, -1, 0]^T, \quad \mathbf{v}_2 = \frac{1}{2}[0, 1, 0, -1, 0, 1, 0, -1]^T \quad (39)$$

这是交替符号的 zigzag（折叠）模态：节点位移 +1, -1, +1, -1 交错。它们在  $2 \times 2$  全积分下具有正刚度 0.5——下一节的单点积分将证明，正是这两个模态在降阶积分下会丢失刚度。

**第五步——约束消去与求解。** 约束  $u_{0x} = u_{0y} = u_{2y} = u_{3x} = 0$ （节点 0 全固、节点 1 锁  $y$ 、节点 3 锁  $x$ ），自由度为  $[u_{1x}, u_{2x}, u_{2y}, u_{3y}]$ ，载荷  $F_{1x} = F_{2x} = 0.5$ （右边界合力为 1，对应  $\sigma_x = 1$ ）。缩减刚度

$$\mathbf{k}_{ff} = \begin{bmatrix} 0.5 & 0 & -0.125 & 0.125 \\ 0 & 0.5 & 0.125 & -0.125 \\ -0.125 & 0.125 & 0.5 & 0 \\ 0.125 & -0.125 & 0 & 0.5 \end{bmatrix}, \quad \kappa(\mathbf{k}_{ff}) = 3 \quad (40)$$

解  $\mathbf{u}_{ff} = [1, 1, 0, 0]^T$ , 即  $\mathbf{u} = [0, 0, 1, 0, 1, 0, 0, 0]^T$ 。

**第六步——应力。**任意高斯点处  $\boldsymbol{\sigma} = \mathbf{DB}(\xi_q)\mathbf{u}$ 。程序验证四个高斯点全部给出

$$\boldsymbol{\sigma} = [\sigma_x, \sigma_y, \tau_{xy}]^T = [1, 0, 0]^T \quad (41)$$

与解析解逐位一致——Q4 通过分片试验，能精确再现常应力场（这正是 §4.8 中 Q4 常应力精确性的出处）。

### 4.13 手算示例：Q4R 单点积分与沙漏控制

现在对同一单元改用单点积分，并逐步演示沙漏失稳与稳定化的完整数值过程——这是本软件 Q4R 单元（fem2d/element/q4r.py）的真实公式逐项复算，不是简化版本。

**第一步——单点运动学。**单点规则取单元中心  $(\xi, \eta) = (0, 0)$ ，权重  $w_0 = 4$ （等于参数域面积）。中心点应变矩阵

$$\mathbf{B}_0 = \begin{bmatrix} -0.5 & 0 & 0.5 & 0 & 0.5 & 0 & -0.5 & 0 \\ 0 & -0.5 & 0 & -0.5 & 0 & 0.5 & 0 & 0.5 \\ -0.5 & -0.5 & -0.5 & 0.5 & 0.5 & 0.5 & 0.5 & -0.5 \end{bmatrix} \quad (42)$$

单点刚度（ $\det \mathbf{J}_0 = 1/4$ ，故  $w_0 t \det \mathbf{J}_0 = 1$ ）：

$$\mathbf{k}_0 = \mathbf{B}_0^T \mathbf{DB}_0 = \begin{bmatrix} 0.375 & 0.125 & -0.125 & -0.125 & -0.375 & -0.125 & 0.125 & 0.125 \\ 0.125 & 0.375 & 0.125 & 0.125 & -0.125 & -0.375 & -0.125 & -0.125 \\ -0.125 & 0.125 & 0.375 & -0.125 & 0.125 & -0.125 & -0.375 & 0.125 \\ -0.125 & 0.125 & -0.125 & 0.375 & 0.125 & -0.125 & 0.125 & -0.375 \\ -0.375 & -0.125 & 0.125 & 0.125 & 0.375 & 0.125 & -0.125 & -0.125 \\ -0.125 & -0.375 & -0.125 & -0.125 & 0.125 & 0.375 & 0.125 & 0.125 \\ 0.125 & -0.125 & -0.375 & 0.125 & -0.125 & 0.125 & 0.375 & -0.125 \\ 0.125 & -0.125 & 0.125 & -0.375 & -0.125 & 0.125 & -0.125 & 0.375 \end{bmatrix} \quad (43)$$

特征值： $\{0, 0, 0, 0, 0, 1, 1, 1\}$ （rank = 3）。

**第二步——沙漏模态：**五个零能模态中的两个是假的。五个零特征值中，三个是刚体模态（正常），另外两个是

$$\mathbf{h}_x = [1, 0, -1, 0, 1, 0, -1, 0]^T, \quad \mathbf{h}_y = [0, 1, 0, -1, 0, 1, 0, -1]^T \quad (44)$$

交替符号 zigzag 模态——正是上一节 Q4 全积分下特征值 0.5 的那两个特征向量（仅差归一化常数）。它们零能量的原因是  $\mathbf{B}_0 \mathbf{h}_x = \mathbf{B}_0 \mathbf{h}_y = \mathbf{0}$ ：zigzag 位移在单元中心处不产生任何应变（中心点是应变采样点），因此

$$\mathbf{k}_0 \mathbf{h}_x = \mathbf{0}, \quad \mathbf{k}_0 \mathbf{h}_y = \mathbf{0} \quad (45)$$

同一模态的刚度对比： $2 \times 2$  全积分 0.5  $\rightarrow$  单点积分 0。单点积分丢掉了 zigzag 模态的全部弯曲刚度——这就是沙漏失稳的根源。

**第三步——失稳后果：**解直接失败。对 §4.12 相同的四个约束  $\{u_{0x}, u_{0y}, u_{2y}, u_{3x}\}$ ，单点刚度缩减后仍奇异（程序验证 np.linalg.LinAlgError: Matrix is singular）。原

因：约束只能消除与约束方向相交的零能模态，而这里零能模态共 5 个（3 刚体 + 2 沙漏），4 个约束不够。即使换一组约束恰好锁住全部零能模态，解也会在微小扰动下爆炸——单点刚度在 zigzag 方向“看不见”任何抗力，网格的任何载荷成分都可能激发它。这就是为什么单点积分单元必须配备沙漏控制（否则要么解不出，要么给出无物理意义的折叠位移场）。

第四步——仿射补投影稳定化（本软件实际公式）。稳定项

$$\mathbf{k}_{hg} = \alpha \mathbf{H} (\mathbf{H}^T \mathbf{k}_{shear} \mathbf{H}) \mathbf{H}^T, \quad \alpha = 0.10 \quad (46)$$

其中  $\mathbf{H}$  ( $8 \times 2$ ) 的列张成所有仿射节点位移场（刚体 + 常应变）的正交补。单位正方形上，\_affine\_complement 的余因子技巧给出  $\mathbf{h} = \frac{1}{2}[1, -1, 1, -1]$ （与  $\mathbf{h}_x$  相同的 zigzag 型向量、已归一化），

$$\mathbf{H} = \begin{bmatrix} \mathbf{h} & \mathbf{0} \\ \mathbf{0} & \mathbf{h} \\ \mathbf{h} & \mathbf{0} \\ \mathbf{0} & \mathbf{h} \end{bmatrix} \implies \mathbf{H}^T \mathbf{H} = \mathbf{I}_2 \quad (47)$$

$\mathbf{k}_{shear}$  是用纯剪切模量  $\mathbf{D}_{shear} = \text{diag}(2\mu, 2\mu, \mu)$  ( $\mu = E/[2(1+\nu)]$ ) 做  $2 \times 2$  高斯积分得到的  $8 \times 8$  参考刚度——只用剪切项避免给稳定化引入体积锁定。单位正方形数值：

$$\mathbf{H}^T \mathbf{k}_{shear} \mathbf{H} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \quad (48)$$

$$\mathbf{k}_{hg} = 0.10 \cdot 0.5 \cdot \mathbf{H} \mathbf{H}^T = 0.0125 \begin{bmatrix} 1 & 0 & -1 & 0 & 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 & 0 & 1 & 0 & -1 \\ -1 & 0 & 1 & 0 & -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 & 0 & -1 & 0 & 1 \\ 1 & 0 & -1 & 0 & 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 & 0 & 1 & 0 & -1 \\ -1 & 0 & 1 & 0 & -1 & 0 & 1 & 0 \\ 0 & -1 & 0 & 1 & 0 & -1 & 0 & 1 \end{bmatrix} \quad (49)$$

两个结构性保证值得注意：(1) 由于  $\mathbf{H}$  与仿射场正交，稳定项对刚体模态和常应变场贡献恒为零——不影响精确性，补丁试验仍通过；(2)  $\mathbf{H}$  的构造只用余因子（三角形两倍面积技巧），不需要每单元一次 LAPACK 分解（见 §6.4.1 仿射补：余因子技巧）。

第五步——稳定后的谱与求解。稳定刚度  $\mathbf{k} = \mathbf{k}_0 + \mathbf{k}_{hg}$ ：

$$\lambda(\mathbf{k}) = \{0, 0, 0, 0.05, 0.05, 1, 1, 1\}, \quad \text{rank} = 5 \quad (50)$$

两个 zigzag 模态从零能量恢复到 0.05 ( $\alpha \times 0.5$ )——正值即稳定，但只有全积分刚度 0.5 的十分之一，这就是 Q4R 弯曲偏柔的谱证据。同一约束与载荷下  $\kappa(\mathbf{k}_{ff}) = 21$ （全积分 Q4 为 3，量级正常），解仍精确为  $\mathbf{u} = [0, 0, 1, 0, 1, 0, 0, 0]^T$ 、 $\boldsymbol{\sigma} = [1, 0, 0]^T$ 。

小结。三个刚度矩阵的特征谱对照（ $E=1$ 、 $\nu=0$ 、单位正方形）：



|                                 | 零特征值            | zigzag 模态特征值 |
|---------------------------------|-----------------|--------------|
| Q4 全积分 (2×2 高斯)                 | 3 (刚体)          | 0.5, 0.5     |
| Q4R 单点 (无稳定)                    | 5 (3 刚体 + 2 沙漏) | 0, 0         |
| Q4R 单点 + 稳定 ( $\alpha = 0.10$ ) | 3 (刚体)          | 0.05, 0.05   |

$\alpha$  是沙漏控制的唯一旋钮 (HOURLASS\_COEFFICIENT, 默认 0.10): 过小则稳定不足 (接近失稳), 过大则过度刚化 (弯曲解偏刚)。本实现有两条已知局限 (q4r.py 文档字符串已如实记录, 非编码错误): 稳定项尺度  $\mathbf{H}^T \mathbf{k}_{shear} \mathbf{H}$  不随长宽比衰减, 而单点弯曲刚度按  $(h/L)^3$  衰减——薄板 / 少排网格上两者失衡, 长宽比  $\geq 50$  时稳定项主导导致明显过刚; 多排常规网格 (长宽比  $< 10$ ) 验证正常。细长模型建议改用 Q4I (CPS4I, 无稳定项)。

两个手算示例可用源码直接复现 (与 §4.11 的 CST 示例组成三单元对照):

```
import numpy as np
from fem2d.element.q4 import element_stiffness
from fem2d.element.q4r import element_stiffness as q4r_stiffness

coords = np.array([[0.,0.],[1.,0.],[1.,1.],[0.,1.]])
K_q4 = element_stiffness(coords, 1.0, 0.0, t=1.0, plane="stress")
K_q4r = q4r_stiffness(coords, 1.0, 0.0, t=1.0, plane="stress")
print(np.linalg.eigvalsh(K_q4)) # [0, 0, 0, 0.5, 0.5, 1, 1, 1]
print(np.linalg.eigvalsh(K_q4r)) # [0, 0, 0, 0.05, 0.05, 1, 1, 1]
```

## 5 代码结构

本章是静态结构地图 (模块/函数以符号引用标注, 免疫行号漂移)。数值行为由金标准/漂移门锁定, 结构变动需同步更新 docs/CODE\_MAP.md。

### 5.1 顶层结构

|                                 |                                             |
|---------------------------------|---------------------------------------------|
| run.py                          | 纯转发 → runner.main (编码安全网在 main 内)           |
| run_demo.py                     | demo_complex 示例 (椭圆孔内压+顶部压力+自重), 直接复用正式 API |
| fem2d/<br>子包)                   | 主包 (30 顶层模块 + element/ + boundary/ 两个子包)    |
| scripts/<br>脚本)                 | 工具层 (打包/探针/fuzz/漂移门/geo_spec 等 12+)         |
| tests/                          | 143 文件, ~1649 test 函数                       |
| docs/<br>coverage / performance | api_contract / boundary_plugins / ci /      |
| models/                         | 示例模型; tools/ 捆绑 Gmsh 4.15.2                 |

## 5.2 五阶段数据流

单入口 `run.py`  $\rightarrow$  `runner.main`，一次分析固定走五个阶段（本项目可维护性的第一支柱——单一主线、无旁路入口）：

|     |                                                                                                                                                     |                                                                                                                |                                           |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| 阶段1 | <code>_resolve_input</code>                                                                                                                         | <code>runner._resolve_input()</code>                                                                           | — <code>.spec/.geo/.txt/.msh</code> 统一分派  |
|     | <code>`- input_source.resolve_input_file()</code> <code>.spec</code> 键值 / <code>.geo</code> @FEM:注释 / <code>.txt</code> 中文几何 / <code>.msh</code> 直读 |                                                                                                                |                                           |
| 阶段2 | <code>_build_model</code>                                                                                                                           | <code>runner._build_model()</code>                                                                             | — 网格导入校验 + @FEM 合并 + 边界模型                 |
|     | <code> - gmsh_adapter.generate_from_geo()</code> 执行 <code>.geo</code> $\rightarrow$ 网格 + 语义区域 (API 路径)                                              |                                                                                                                |                                           |
|     | <code> - gmsh_adapter.import_msh()</code> 打开已有 <code>.msh</code> (子进程路径共用 <code>_extract_mesh()</code> )                                            |                                                                                                                |                                           |
|     | <code> - preprocess.validate_mesh()</code> 重复节点/单元、零边、退化、孤立、非流形 全家桶                                                                                 |                                                                                                                |                                           |
|     | <code>`- boundary.build_boundary_segments</code> <code>naming.build_boundary_segments()</code> — 优先级:                                               |                                                                                                                |                                           |
|     |                                                                                                                                                     | <code>RegionRegistry</code> $\rightarrow$<br><code>edge_labels</code> $\rightarrow$ 纯拓扑<br><code>detect</code> |                                           |
| 阶段3 | <code>_apply_conditions</code>                                                                                                                      | <code>runner._apply_conditions()</code>                                                                        | — <code>bc_apply.apply_bcs()</code>       |
|     |                                                                                                                                                     | (fix/traction/force/body 四类)                                                                                   |                                           |
| 阶段4 | <code>_analyze_and_report</code>                                                                                                                    | <code>runner._analyze_and_report()</code>                                                                      | — <code>solver.solve()</code> +           |
|     |                                                                                                                                                     | <code>error_est.estimate()</code> + 中文报告                                                                       |                                           |
| 阶段5 | <code>_plot</code>                                                                                                                                  | <code>runner._plot()</code>                                                                                    | — <code>visualize</code> (云图/isoband /交互) |

## 5.3 模块职责表

### 5.3.1 element/（内核注册协议——行为冻结区）

| 模块                   | 职责                                                                                             |
|----------------------|------------------------------------------------------------------------------------------------|
| <code>base.py</code> | <code>ElementKernel</code> ABC (协议) + 注册表 ( <code>register_element/get_element_kernel</code> ) |
| <code>cst.py</code>  | 常应变三角: 显式形状函数/B 矩阵/刚度; 自检 completeness + 刚体模态                                                  |
| <code>q4.py</code>   | 双线性四边形 ( $2 \times 2$ 积分), batch 运动学                                                           |

|        |                                                                  |
|--------|------------------------------------------------------------------|
| q4i.py | QM6 非协调模式: incompatible_derivatives + 静力凝聚(_condensation_blocks) |
| q4r.py | 单点积分稳定: affine complement + 沙漏系数 + hourglass_energy              |

Mesh 构造时浅拷贝 kernel (Q4R hourglass\_coefficient 是可变类属性, 防跨 Mesh 污染)。

### 5.3.2 boundary/ (识别管线——识别算法冻结, 只加插件/标签)

| 模块                                 | 职责                                                                            |
|------------------------------------|-------------------------------------------------------------------------------|
| topology.py                        | 邻接图 → 闭环分解 (_decompose_loops) → 嵌套定向 (_validate_and_nest_loops) → 曲率分割 → 自交检测 |
| geometry.py                        | 曲率 (双边滤波)/锐角断点/直线椭圆拟合 (fit_closed_ellipse)                                    |
| naming.py                          | build_boundary_segments() 总入口; 段命名/边名解析 (parse_edge_name())                   |
| detectors/                         | 插件识别体系: Detector 基类 + Registry (首个非 None 胜出, GeneralCurveDetector 兜底恒返回)      |
| plugins/                           | 轮 2 示例插件: arc_curvature / circle_label / ellipse_group_label                  |
| segment_builder /<br>segment_utils | 段构建 + 纯函数助手 (segment_sort_key 等)                                              |
| physical_mapping.py                | PhysicalEdgeMapper (A 轮已标记弃用)                                                 |
| registry_mapping.py                | RegionBoundaryMapper: RegionRegistry → 边界映射 (新主线)                             |
| selectors.py                       | BoundarySelector + resolve_boundary_selector                                  |
| conic_merge.py                     | 兼容 CAD 圆锥曲线合并 (组级椭圆标签)                                                        |
| model / validation /<br>predicates | 诊断模型 / schema 验证 / orient2d                                                   |

### 5.3.3 核心数值 (冻结区)

| 模块          | 职责                                                                                                   |
|-------------|------------------------------------------------------------------------------------------------------|
| mesh.py     | Mesh 容器: 数据不可变 (只读 setter + replace_* + 缓存失效); 外法向由相邻单元 CCW 确定                                       |
| assembly.py | 4 条路径: assemble_sparse / assemble_sparse_vectorized / assemble_lil_reference(教学参考) / assemble_expand |
| solver.py   | solve() 完整流程; 奇异守卫/平衡校验/沙漏监控                                                                         |
| bc.py       | apply_elimination() (Bathe §4.2.2 消去, direct/cg/ilu) + apply_penalty()                               |

|               |                                                                                                                             |
|---------------|-----------------------------------------------------------------------------------------------------------------------------|
| bc_apply.py   | apply_bcs() 四类载荷施加 (fix/traction/force/body)                                                                                |
| loads_core.py | assemble() ( $F=R_c+\Sigma R_s+\Sigma R_b$ ) ;<br>parse_traction/parse_vec2/make_edge_profile_func; _compile_expr (AST 白名单) |
| loads.py      | 兼容层 facade, 纯 re-export loads_core                                                                                          |

### 5.3.4 后处理

| 模块           | 职责                                                                                               |
|--------------|--------------------------------------------------------------------------------------------------|
| stress.py    | compute_stresses / principal_stresses / nodal_average / nodal_L2_projection / stress_at_point    |
| spr.py       | SPR 恢复: 节点补丁稀疏结构 (_node_patch_csr) + 局部最小二乘 (_fit_node_block)                                    |
| error_est.py | estimate() (SPR/L2/weighted); compute_traction_jumps(); element_refinement_indicator() (Z2 指示器)  |
| visualize.py | plot_contour / plot_three / interactive_plot / isoband (Bathe 等应力带) / Gouraud / traction jumps 图 |
| reporting.py | 中文结果摘要 (位移模长 np.hypot 等)                                                                         |

### 5.3.5 验证与辅助

| 模块                            | 职责                                  |
|-------------------------------|-------------------------------------|
| convergence.py                | 收敛研究 (run_cantilever_convergence()) |
| verification.py               | run_plane_verification()            |
| patch_test.py                 | 分片试验                                |
| quality.py                    | 网格质量指标 (bad/warn/ok 互斥分类)           |
| wizard.py                     | 交互建模向导 (无参数 + 终端时自动)                |
| regions.py / topology_core.py | 区域注册表 / 矢量化拓扑原语                     |
| checks.py / errors.py         | 预检项 / 错误类型体系                        |

## 5.4 solver.solve 内部流程 (Bathe §4.2)

|                                     |                                    |
|-------------------------------------|------------------------------------|
| 0. validate_state                   | 构造后字段可重写 → 求解前快速失败                 |
| 1. 拓扑 + Jacobian 报告                 | 无效单元先于组装抛错 (inverted/degenerate)   |
| 2. _partition_dofs                  | 约束划分 + 逐连通分量刚体模态检查 (rank < 3 禁止求解) |
|                                     | + Q4R 长宽比告警                        |
| 3. assemble_sparse + assemble_loads |                                    |

|                                           |                                                                                         |
|-------------------------------------------|-----------------------------------------------------------------------------------------|
| 4. <code>_solve_linear_system</code>      | 纯 Dirichlet (空矩阵特判) / 消去 / 罚 三<br>分支                                                    |
| <code>_solve_with_singular_guard()</code> | 秩警告→异常, 非秩警告转发                                                                          |
| <code>_check_solution_finite</code>       | NaN/Inf 快速失败                                                                            |
| 4'. 残差                                    | 后向误差 $\ r\ _{\infty} / (\ K\ _{\infty} \ u\ _{\infty} + \ F\ _{\infty})$ (Bathe §8.2.6) |
| 5. 应力响应                                   | 单元/积分点应变应力 + von Mises                                                                  |
| 5'. 沙漏监控                                  | 内能有限性 + Q4R 沙漏能占比                                                                       |
| 5.5 全局平衡检查                                | $\Sigma R + \Sigma F / \Sigma M$ , <code>tol_rel</code> 相对尺度 (无绝对阈值)                    |
| 6. 小变形 + 结果有限性检查;                         | 条件数可选 (默认关)                                                                             |

## 5.5 冻结区与契约

| 区域                                                | 规则                                                           |
|---------------------------------------------------|--------------------------------------------------------------|
| element/ 内核公<br>式、solver 数值<br>逻辑、error_est<br>公式 | 只改入口校验; 先金标准 → 逐位一致 → 回归 0.0                                 |
| 边界识别管线<br>(boundary/)                             | 只加插件/标签/输入, 识别算法冻结                                           |
| 段 schema                                          | 金标准锁定 (boundary_golden), 不得新增键                               |
| von_mises                                         | 逐位不变                                                         |
| / principal_stresses                              |                                                              |
| 对 (n,3) 输出                                        |                                                              |
| apply_penalty                                     | 数值逐位不变                                                       |
| 有限输入                                              |                                                              |
| cg_rtol                                           | 必须 1e-10 (实测 1e-8 污染支反力, 被平衡检查误杀)                            |
| DOF 约定                                            | $x \rightarrow 2n$ , $y \rightarrow 2n+1$                    |
| gmsh                                              | 必须 -nt N -2; .geo 视为"可信可执行输入" (SystemCall 拦截净化)              |
| 禁绝对阈值                                             | 相对尺度 / <code>np.finfo(float).tiny</code> (微尺度 1e-150 模型是硬指标) |

## 5.6 错误处理体系

- 错误类型: 用户误用 → `ValueError` / `TypeError` / `CliError(exit_code)` 带参数上下文; 禁止裸 `IndexError`/`KeyError`/`AttributeError` 冒出
- 消息格式: 函数名: 参数名 = 值 — 原因, 期望 (阶段 2 统一为共享 helper 的格式)
- NaN/Inf: 一律拒绝 (有限性), 不允许静默传播或静默夹值
- 布尔掩码: `[True, False]` 是布尔掩码, 一律拒绝 (禁止折叠成 DOF 0/1)

- 多余分量：静默忽略 = 最危险错误；必须报错
- 微尺度/大坐标：1e-150 几何 / 1e12 坐标 / 1e-310 载荷必须保持有限，禁止绝对阈值

## 6 代码深度解析

本章是全书的重心：展示**真实源码**，并解释**每一处安排背后的理由**。所有代码片段取自 fem2d/（行号随版本漂移，代码以 docs/CODE\_MAP.md 的符号索引为准）。读者将看到三个核心设计决策如何在代码中落地：**协议驱动**（内核注册表）、**冻结区**（数值公式不改）、**插件体系**（识别器/命名器可扩展）。

### 6.1 设计哲学：为什么代码这样安排

先给总览，再逐模块展开。四个支柱性决策：

| 决策                     | 收益                                                   | 代价                            |
|------------------------|------------------------------------------------------|-------------------------------|
| 单入口 + 五阶段<br>内核注册协议    | 数据流唯一，任何路径可复现<br>新单元 = 新文件 + 注册一行，<br>Mesh/装配/后处理零改动 | 旁路需求只能走主流程<br>协议必须稳定，改动影响所有内核 |
| 冻结区（金标准锁）<br>插件体系（识别器） | 数值行为永不被”顺手优化”破坏<br>边界语义可扩展，首个非 None 胜出               | 冻结区内部重构成本高<br>顺序敏感，需兜底探测器     |

### 6.2 单元内核协议（element/base.py）

#### 6.2.1 为什么需要协议层

早期版本把单元类型写死在 Mesh、装配和应力恢复里：每个新单元要改 3 个模块。引入 ElementKernel 抽象基类后，**单元的所有专属行为**（几何缓存、刚度、应力响应、形状函数反演、验证）都归内核所有，外部模块只消费协议。模块 docstring 明确声明：

```
"""Finite-element kernel protocol and registry.

The solver operates on a homogeneous element block. A kernel owns
all
topology-specific and interpolation-specific operations; Mesh,
assembly and
stress recovery only consume this protocol.

Adding a new displacement element therefore requires:

1. implement ElementKernel in a new module;
2. register one kernel instance with register_element.
"""
```

### 6.2.2 协议的核心方法

```

class ElementKernel(ABC):
    """Protocol implemented by a homogeneous 2-D displacement element
    ."""

    name: str
    aliases: tuple[str, ...]
    nodes_per_element: int
    local_edges: tuple[tuple[int, int], ...]
    recovery_family: str | None = None

    @property
    def dofs_per_element(self) -> int:
        return 2 * self.nodes_per_element

    def matches(self, elem_type: str) -> bool:
        key = str(elem_type).strip().upper()
        return key == self.name.upper() or key in {
            alias.upper() for alias in self.aliases
        }

    @abstractmethod
    def build_geometry(self, nodes, elements) -> dict[str, np.ndarray]:
        """Return standard and element-specific geometry caches."""

    @abstractmethod
    def stiffness_batch(self, mesh, element_slice=None) -> np.ndarray:
        """Return local stiffness matrices as (ne_batch, ndofe, ndofe
        ).

        Kernels should honor a slice/index array so large meshes can
        be
        assembled in bounded-memory chunks.
        """

    @abstractmethod
    def jacobian_determinants(self, mesh) -> np.ndarray:
        """Return det(J) samples shaped (n_element, n_sample)."""

    @abstractmethod
    def body_force_vector(self, mesh, eid, body_force):
        """Return the local consistent body-force vector."""

```

```

@abstractmethod
def shape_values_at(self, coords, x, y, tol=1e-12):
    """Return shape values for an interior point, or None if
        outside."""

@abstractmethod
def verify_mesh(self, mesh, verbose=True) -> bool:
    """Run element completeness/rigid-body verification."""

```

为什么这些方法是抽象的：它们恰好是单元与几何/材料/载荷交互的全部触点。`stiffness_batch` 接受 `element_slice` 参数是刻意设计——大网格按 5 万分批组装 (§6.6)，内核必须支持分块而不是一次算完全部（否则 30 万单元的 (300k,8,8) 中间数组也要 1.4 GB）。

### 6.2.3 注册表：import 顺序的防呆

```

_REGISTRY: dict[str, ElementKernel] = {}

def register_element(kernel: ElementKernel) -> ElementKernel:
    """Register a kernel and all of its aliases.

    Duplicate keys are rejected unless they point to the same kernel
    instance; this prevents import order from silently changing
        element
    behavior.
    """
    if not isinstance(kernel, ElementKernel):
        raise TypeError("register_element expects an ElementKernel
            instance")
    keys: Iterable[str] = (kernel.name, *kernel.aliases)
    for raw_key in keys:
        key = str(raw_key).strip().upper()
        if not key:
            raise ValueError("Element type names cannot be empty")
        existing = _REGISTRY.get(key)
        if existing is not None and existing is not kernel:
            raise ValueError(
                f"Element type '{key}' is already registered by "
                f"{existing.__class__.__name__}.")
        _REGISTRY[key] = kernel
    return kernel

```



```
def get_element_kernel(elem_type: str) -> ElementKernel:
    """Resolve an Abaqus/internal element name to its registered
    kernel."""
    key = str(elem_type).strip().upper()
    kernel = _REGISTRY.get(key)
    if kernel is None:
        supported = ", ".join(sorted(_REGISTRY))
        raise ValueError(
            f"Unsupported element type '{elem_type}'. "
            f"Registered element types: {supported or '(none)'}."
        )
    return kernel
```

**设计要点：重复键冲突抛错而非静默覆盖**——若两个模块恰好注册了同名别称（如 Abaqus 的 CPS4 被两个内核都认领），覆盖会让单元行为随 import 顺序悄悄改变。这里宁可让用户显式解决冲突。get\_element\_kernel 的报错列出全部已注册类型——把“查询失败”变成教学信息。

#### 6.2.4 Jacobian 检查：逐单元尺度容差

jacobian\_report() 是组装前的第一道防线，其容差设计体现了本项目的“相对尺度”纪律：

```
scale_per = np.max(np.abs(det_j), axis=1)
tol_per = np.maximum(1e-15 * scale_per, np.finfo(float).tiny)
inverted_mask = np.any(det_j < -tol_per[:, None], axis=1)
degenerate_mask = (~inverted_mask
                  & np.any(np.abs(det_j) <= tol_per[:, None], axis=1))

# 形状退化检查：内核可选提供无量纲指标（面积/最长边2），
# 数学塌缩级（< 1e-8）判退化，与单元绝对尺寸无关
measure = self.degeneracy_measure(mesh)
if measure is not None:
    degenerate_mask |= (~inverted_mask & (measure < 1e-8))
```

**为什么不用全局容差：**多尺度模型里小单元（面积  $10^{-8}$ ）的  $|\det J|$  天然小于大单元的容差——全局 max 容差会误杀合法小单元。逐单元尺度容差 + 无量纲退化度量（面积/最长边<sup>2</sup>）使检查与绝对尺寸无关。

#### 6.2.5 载荷求值统一校验

```
def evaluate_vector_field(field, x, y):
    """Evaluate a constant/callable two-component vector field.

    统一校验（体力/面力/压力共用）：callable 返回值必须为 (bx, by)
    二元组；分量必须可转 float 且有限。错误带求值点坐标 —
    曾裸 IndexError/TypeError/静默 NaN。
```

```

"""
if callable(field):
    value = field(x, y)
    if isinstance(value, np.ndarray):
        ok = value.ndim == 1 and value.shape == (2,)
    else:
        ok = isinstance(value, (tuple, list)) and len(value) == 2
    if not ok:
        raise ValueError(...) # 带求值点坐标 (x,y) 的上下文消息
...
if not (np.isfinite(bx) and np.isfinite(by)):
    raise ValueError(
        f"载荷分量 NaN/Inf 在 ({x:.4g},{y:.4g})")
return bx, by

```

为什么：体力/面力/压力三处载荷代码共用同一个求值器，错误消息带求值点坐标——用户写错表达式时能立刻定位到“哪个单元哪个高斯点”，而不是收到裸 `IndexError`。

## 6.3 Q4 内核代码解析 (element/q4.py)

### 6.3.1 形函数：教科书公式的直接翻译

```

_G = 1.0 / np.sqrt(3.0)
GAUSS_POINTS = np.array([
    [-_G, -_G], [_G, -_G], [_G, _G], [-_G, _G],
])

def shape_values(xi, eta):
    """Bilinear shape values [N1,N2,N3,N4]."""
    return 0.25 * np.array([
        (1.0 - xi) * (1.0 - eta),
        (1.0 + xi) * (1.0 - eta),
        (1.0 + xi) * (1.0 + eta),
        (1.0 - xi) * (1.0 + eta),
    ])

def shape_derivatives(xi, eta):
    """Natural derivatives shaped (2,4): rows d/dxi and d/deta."""
    return 0.25 * np.array([
        [-(1.0 - eta), (1.0 - eta),
         (1.0 + eta), -(1.0 + eta)],
        [-(1.0 - xi), -(1.0 + xi),

```

```
(1.0 + xi), (1.0 - xi)],
])
```

节点顺序遵循 Abaqus CPS4/CPE4 约定 (1→2→3→4 逆时针) ——与外部工具链对齐是刻意的: .inp 导入、Gmsh 导出、商业软件对比三者共享同一约定, 避免“顺时针单元在别的软件里是负面积”的经典事故。GAUSS\_POINTS 是  $2 \times 2$  高斯点 ( $\pm 1/\sqrt{3}$ ), 作为模块级常量被标量路径与批量路径共用——两份实现不能各写一遍数值 (重复实现是静默分歧的温床)。

### 6.3.2 B 矩阵: 链式法则

```
def B_matrix(coords, xi, eta):
    """Return (B, detJ) for one Q4 element and natural point."""
    coords = np.asarray(coords, dtype=float)
    dN_nat = shape_derivatives(xi, eta)
    jacobian = dN_nat @ coords
    det_j = float(np.linalg.det(jacobian))
    if abs(det_j) <= np.finfo(float).tiny:
        raise ValueError("Q4 Jacobian is singular")
    gradients = np.linalg.solve(jacobian, dN_nat)
    B = np.zeros((3, 8))
    B[0, 0::2] = gradients[0]          # eps_x 行: dN_i/dx
    B[1, 1::2] = gradients[1]          # eps_y 行: dN_i/dy
    B[2, 0::2] = gradients[1]          # gamma_xy 行: dN_i/dy (x 分量)
    B[2, 1::2] = gradients[0]          # gamma_xy 行: dN_i/dx (y 分量)
    return B, det_j
```

B 矩阵的行分配是三行三应变与八列 (4 节点  $\times$  2 DOF) 的交错布局:  $B[0, 0::2]$  是  $\varepsilon_x$  对每个节点 x-位移的贡献,  $B[2, 0::2]$  是  $\gamma_{xy}$  对 x-位移的贡献。这种布局对应全局 DOF 编号约定  $2n$  (x)、 $2n+1$  (y) ——全项目唯一定义, 装配与后处理共享 (历史教训: DOF 布局写反时, 非对称检查会抓住它, 见 §6.6)。

### 6.3.3 批量运动学: 为什么缓存、为什么局部坐标

```
def _batch_kinematics(coords, sample_points):
    """Return batched shape values, B matrices and Jacobian
    determinants.

    Jacobian 基于局部坐标 (每单元以其第一个节点为原点) — 形函数导数
    行和为零, 平移数学上不变, 但绝对坐标的浮点乘加会引入 ~ulp(原点)
    误差, 大坐标原点下刚度相对变化实测 ~9e-5。
    """
    ne = coords.shape[0]
    nq = len(sample_points)
    N_all = np.zeros((nq, 4))
```

```

B_all = np.zeros((ne, nq, 3, 8))
det_all = np.zeros((ne, nq))
local = coords - coords[:, :1, :]      # 每单元以其首节点为原点
for q, (xi, eta) in enumerate(sample_points):
    N_all[q] = shape_values(xi, eta)
    dN_nat = shape_derivatives(xi, eta)
    jac = np.einsum("ai,eib->eab", dN_nat, local)
    det = (jac[:, 0, 0] * jac[:, 1, 1]
           - jac[:, 0, 1] * jac[:, 1, 0])
    det_all[:, q] = det
... # 非奇异单元一次性批量解链式法则

```

两个设计理由:

1. **批量优先:** `build_geometry()` 一次算完全部单元的 B 矩阵与 Jacobian, 存入 Mesh 的几何缓存 (`q4_N_gauss/q4_B_gauss/q4_detJ_gauss` 等 9 个键), `stiffness_batch` 只是 `einsum` 的线性组合——300k 单元组装 1.5 秒 (vs 逐单元 Python 循环的分钟级)。缓存带失效纪律: 材料/几何改动必须 `replace_*` API 显式换新 (Mesh 数据不可变约定, 见 §5.4);
2. **局部坐标:** 每个单元平移到自己首节点为原点。绝对坐标在  $10^{12}$  偏移下, `np.einsum` 的浮点乘加引入  $\sim \text{ulp}(\text{原点})$  误差——实测刚度相对变化  $\sim 9 \times 10^{-5}$ , 对“大坐标小单元”模型 (如微电子封装, 全局原点在远处) 这是必须的处理。

### 6.3.4 刚度批量组装: 先加权再二次型

```

def stiffness_batch(self, mesh, element_slice=None):
    selector = slice(None) if element_slice is None else
        element_slice
    B_gauss = mesh.q4_B_gauss[selector]
    det_gauss = mesh.q4_detJ_gauss[selector]
    D = material.D_matrix(mesh.E, mesh.nu, mesh.plane_type)
    Ke = np.zeros((len(B_gauss), 8, 8))
    for q in range(4):
        B = B_gauss[:, q]
        # 先加权再二次型: B~ = sqrt(t*detJ)*B, Ke = B~^T D B~ —
        # 先算 B^T D B ~ E/L^2 中间量会在微尺度几何 (L~1e-150) 下溢出
        # Inf, 再乘 detJ 已来不及。数学等价,
        # 中间量尺度 O(sqrt(t*D)) 不随 L 发散。
        if np.any(det_gauss[:, q] <= 0.0):
            raise ValueError(...) # 显式拒绝负 Jacobian
        w = (np.sqrt(mesh.thickness) * np.sqrt(det_gauss[:, q]))[:,
            None, None]
        Bw = B * w
        Ke += np.einsum("eai,eaj->eij", Bw, np.einsum(
            "ab,ebj->eaj", D, Bw))

```

```
return 0.5 * (Ke + np.transpose(Ke, (0, 2, 1)))
```

先加权再二次型的数学理由：直接算  $\mathbf{B}^T \mathbf{D} \mathbf{B} \sim E/L^2$ ，在微尺度几何 ( $L \sim 10^{-150}$ ) 下中间量溢出 Inf，再乘  $\det \mathbf{J} \sim L^2$  已经来不及。先乘  $\sqrt{t \det \mathbf{J}}$  再二次型，中间量尺度  $O(\sqrt{t} D)$  不随  $L$  发散——数学等价，数值却从”必炸”变成”安全”。这就是为什么微尺度模型 (1e-150) 在全链路必须保持有限是硬指标。

### 6.3.5 单元级自检：单位分解 + 刚体模态

```
for eid, conn in enumerate(mesh.elements):
    coords = mesh.nodes[conn]
    for q, N in enumerate(mesh.q4_N_gauss):
        max_partition = max(max_partition, abs(np.sum(N) - 1.0))
        B = mesh.q4_B_gauss[eid, q]
        center = coords.mean(axis=0)
        modes = (
            np.tile([1.0, 0.0], 4),          # 刚体平移 x
            np.tile([0.0, 1.0], 4),          # 刚体平移 y
            np.column_stack([-(coords[:, 1] - center[1]),
                             coords[:, 0] - center[0]]).ravel(), # 转动
        )
        for mode in modes:
            rb = float(np.linalg.norm(B @ mode))
            # 相对判据: B ~ 1/h 且旋转模态幅值 ~ h, 绝对比较会被
            # 坐标偏移 (eps*|x0|/h) 放大; 连模态范数一起除 (||B||*||
            # mode||)
            # 后彻底无量纲
            denom = b_norm * mode_norm
            if denom > np.finfo(float).tiny:
                max_rb = max(max_rb, rb / denom)
ok = max_partition < 1e-12 and max_rb < 1e-10
```

为什么自检必须存在：单元公式”看起来对”和”真的对”是两回事。verify\_mesh() 把理论性质 (§4.3.2 的三条性质 + 刚体模态零应变) 变成数值断言——任何形函数/符号错误都会让 max\_partition 或 max\_rb 超阈。这是判别性自检：不是冒烟测试，而是”放回旧错误实现必失败”的守卫。

## 6.4 Q4R 沙漏控制代码解析 (element/q4r.py)

Q4R 是本求解器在单元层面最”硬核”的模块：理论 (§4.3.3) 里的三个设计决策——仿射补、仅剪切模量参考刚度、几何感知——全部落在 \_affine\_complement() 与 stiffness\_batch() 中。

## 6.4.1 仿射补：余因子技巧

```

def _affine_complement(coords):
    """Return an orthonormal basis for non-affine nodal displacements
    .

    coords is shaped (ne, 4, 2) and the result (ne, 8, 2).
    Centring/scaling changes conditioning only; it does not change
    the
    affine subspace.
    """
    ne = coords.shape[0]
    shifted = coords - coords.mean(axis=1, keepdims=True)
    scale = np.max(np.linalg.norm(shifted, axis=2), axis=1)
    scale = np.maximum(scale, np.finfo(float).tiny)
    xy = shifted / scale[:, None, None]

    def triangle_twice_area(a, b, c):
        ab = b - a
        ac = c - a
        return ab[:, 0] * ac[:, 1] - ab[:, 1] * ac[:, 0]

    # Cofactors of [1, x_i, y_i]^T give its one-dimensional nullspace
    .
    # The two displacement hourglass modes are that scalar vector in
    the
    # x- and y-DOFs respectively. This is exactly the same affine
    complement
    # as a complete 8x6 QR, without one LAPACK factorization per
    element.
    h = np.column_stack([
        triangle_twice_area(xy[:, 1], xy[:, 2], xy[:, 3]),
        -triangle_twice_area(xy[:, 0], xy[:, 2], xy[:, 3]),
        triangle_twice_area(xy[:, 0], xy[:, 1], xy[:, 3]),
        -triangle_twice_area(xy[:, 0], xy[:, 1], xy[:, 2]),
    ])
    norm = np.linalg.norm(h, axis=1)
    if np.any(norm <= 100.0 * np.finfo(float).eps):
        raise ValueError(
            "Q4R affine projector is singular; check element geometry
            ")
    h /= norm[:, None]
    bases = np.zeros((ne, 8, 2), dtype=float)
    bases[:, 0::2, 0] = h                # x-DOFs 的沙漏模式
    bases[:, 1::2, 1] = h                # y-DOFs 的沙漏模式

```

```
return bases
```

代码逐行讲解：

1. **居中 + 归一化**：坐标平移到单元形心并除以特征尺寸——”只影响条件数，不改变仿射子空间”。归一化后 `triangle_twice_area` 的量级  $\sim O(1)$ ，数值稳定；
2. **余因子**：仿射位移矩阵  $[1, x_i, y_i]^T$  ( $4 \times 3$ ) 的零空间基向量，恰好是”去掉某行后的  $3 \times 3$  矩阵行列式”——即三角形两倍面积。四个符号交替的  $2A$  正是式 (16)。注释里写明为什么不是 **QR**：每单元一次  $8 \times 6$  QR 的 LAPACK 调用换成  $\sim 12$  次乘加，30 万单元省下的时间可观；
3. **几何感知**： $h_i$  随单元形状自动调整——平行四边形给出教科书  $[1, -1, 1, -1]$ ，扭曲单元自动修正方向；
4. **奇异守卫**：范数退化（塌缩单元）时明确报错，不静默除以零；
5. **两方向**：`bases` 的 8 列 = x-DOFs 一个沙漏向量 + y-DOFs 一个沙漏向量（4 节点  $\times$  2 DOF）。

#### 6.4.2 稳定刚度：单点积分 + 仅剪切参考刚度

```
def element_stiffness(coords, E, nu, t=1.0, plane="stress",
                      hourglass_coefficient=HOURLASS_COEFFICIENT):
    """Return one stabilized one-point Q4 stiffness matrix."""
    coords = np.asarray(coords, dtype=float)
    if coords.shape != (4, 2):
        raise ValueError("Q4R coordinates must have shape (4, 2)")
    validate_hourglass_coefficient(hourglass_coefficient)
    _, B0_all, det0_all = _batch_kinematics(
        coords[None, :, :], np.array([[0.0, 0.0]])) # 单点 (0,0)
    _, B_gp_all, det_gp_all = _batch_kinematics(
        coords[None, :, :], GAUSS_POINTS)          # 2x2 参考点
    ...
    B0 = B0_all[0, 0]
    D = material.D_matrix(E, nu, plane)
    w0 = 2.0 * np.sqrt(t) * np.sqrt(det0)          # 单点高斯权重 2x2=4
    B0w = w0 * B0
    K_reduced = B0w.T @ D @ B0w                    # 式 (4.33) 第一项：单
    点刚度

    mu = E / (2.0 * (1.0 + nu))
    D_shear = np.diag([2.0 * mu, 2.0 * mu, mu])    # 仅剪切模量！
    K_shear = np.zeros((8, 8))
    for q in range(4):
        B = B_gp_all[0, q]
        w = np.sqrt(t) * np.sqrt(det_gp[q])
        Bw = w * B
```



```

        K_shear += Bw.T @ D_shear @ Bw          # 2x2 点上的参考刚度

    H = _affine_complement(coords[None, :, :])[0]
    reduced_hg = H.T @ K_shear @ H
    K_hourglass = hourglass_coefficient * H @ reduced_hg @ H.T
    K = K_reduced + K_hourglass                  # 式 (4.33)
    return 0.5 * (K + K.T)

```

三处细节的工程理由：

1.  $D_{\text{shear}} = \text{diag}(2\mu, 2\mu, \mu)$ ——为什么只有剪切模量：若用完整  $\mathbf{D}$ ，稳定项会把常体积应变方向也“罚”进刚度，重新引入体积锁定 (§4.3.2 的代价)。只保留剪切项，稳定能量正确量纲 ( $N/m$ ) 且不加重锁定；
2.  $w_0 = 2 \cdot \sqrt{t \cdot \det_0}$ ——单点高斯权重为  $2 \times 2 = 4$ ，开方后恰为 2。这个权重在批量路径 (§6.4.3) 中以  $(2.0 * \text{np.sqrt}(\dots) * \text{np.sqrt}(\det_0))$  出现，两处必须一致（同一常量不得各写一遍——重复实现纪律）；
3.  $0.5 * (K + K.T)$  对称化——浮点累加产生的毫埃级非对称在此抹平，配合装配层的对称性检查 (§6.6) 双保险。

### 6.4.3 批量路径与材料指纹

批量版 `stiffness_batch` 是标量版的向量化重写，额外做了两件事：

```

if mesh.q4r_reduced_hourglass is None:
    mesh.q4r_reduced_hourglass = np.full(
        (mesh.n_elements, 2, 2), np.nan, dtype=float)
mesh.q4r_reduced_hourglass[selector] = reduced_hg
# reduced_hg 依赖 D(E,nu,plane) — 记录材料指纹，hourglass_energy
# 据此检测装配后改材料/平面态导致的静默过期
mesh._q4r_hourglass_material = (
    mesh.E, mesh.nu, mesh.thickness, mesh.plane_type)
K_hourglass = self.hourglass_coefficient * np.einsum(
    "eia,eab,ejb->eij", H, reduced_hg, H)
K = K_reduced + K_hourglass

```

**缓存 + 失效：** $H^T K_{\{\text{shear}\}} H$ （每个单元  $2 \times 2$ ）被缓存进 `Mesh`，但缓存带材料指纹—— $E/\nu/t/\text{plane}$  任一变化 (`Mesh.replace_material` 或直接改属性)，`hourglass_energy` 检测到指纹不匹配就按当前材料重算。这是几何缓存 vs 材料相关量的经典冲突：几何缓存 ( $\mathbf{B}$  矩阵) 不随材料失效，而沙漏稳定阵随材料变——用指纹标记而不是无条件失效，既保性能又不静默过期（历史教训：plane 切换后沙漏能沿用旧  $\mathbf{D}$ ，积分点应力相对差  $\sim 9\%$ ，曾实测复现）。

### 6.4.4 沙漏能量监控：负能告警而非截零

```

def hourglass_energy(self, mesh, u_e):
    """Return non-negative stabilization energy for each element."""

```



```

if (mesh.q4r_reduced_hourglass is None
    or not np.all(np.isfinite(mesh.q4r_reduced_hourglass))
    or getattr(mesh, "_q4r_hourglass_material", None)
    != (mesh.E, mesh.nu, mesh.thickness, mesh.plane_type)):
    self.stiffness_batch(mesh)          # 材料指纹不匹配 → 重算
generalized = np.einsum("eia,ei->ea", mesh.q4r_hourglass_basis,
                        u_e)
energy = 0.5 * np.einsum(
    "ea,eab,eb->e",
    generalized,
    self.hourglass_coefficient * mesh.q4r_reduced_hourglass,
    generalized,
)
# K_hourglass is positive semidefinite by construction. Remove
# only
# signed round-off at its exact affine nullspace (e.g. -1e-29 J).
# 显著负能量 = 负系数或实现错误 — 无条件截零会掩盖问题。
neg_mask = energy < -1e-12 * max(
    float(np.max(np.abs(energy))), 1.0)
if np.any(neg_mask):
    warnings.warn(
        f"Q4R hourglass energy 显著为负 (min={...:.3e} J) — "
        "沙漏系数必须为正或存在实现错误", RuntimeWarning)
return np.maximum(energy, 0.0)

```

为什么负能要告警而不是直接截零：稳定项构造上半正定，负能量只可能来自负沙漏系数（用户错误）或实现错误。`max(energy, 0)` 只抹掉仿射零空间上的符号舍入（ $-10^{-29}$  级），显著负值原样告警——错误不会静默是本项目的铁律之一。

#### 6.4.5 Q4R 自检：3 个零模 + 仿射泄漏

```

Ke = self.stiffness_batch(mesh)
eig = np.linalg.eigvalsh(Ke)
scale = np.maximum(np.max(np.abs(eig), axis=1), np.finfo(float).tiny)
n_zero = np.sum(np.abs(eig) <= 1e-9 * scale[:, None], axis=1)
negative = np.any(eig < -1e-10 * scale[:, None], axis=1)
...
for mode in affine_modes:          # 6 个仿射模式逐一检查
    denom = kh_scale * mode_norm
    if denom > np.finfo(float).tiny:
        max_affine_leak = max(max_affine_leak,
                                float(np.linalg.norm(Kh @ mode) / denom))
ok = (np.all(n_zero == 3) and not np.any(negative)
      and max_affine_leak < 1e-10)

```

三个断言恰好对应理论三性质：**恰 3 个零特征值**（2 平动 + 1 转动——“沙漏已被稳定项”充满，不产生额外零模）、**无负特征值**（半正定）、**仿射泄漏**  $< 10^{-10}$ （稳定项对 6 个仿射模式  $\mathbf{K}_{HG} \mathbf{v}_{affine} \approx \mathbf{0}$ ——稳定化不污染任何物理正确的解）。注意仿射泄漏的判据是**无量纲**的：只除  $\|\mathbf{K}_{HG}\|$  会让泄漏量带长度量纲（ $10^6$  单元尺寸时误报 FAIL），连模态范数一起除后彻底无量纲（注释中记录了这个教训）。

## 6.5 CST 与 Q4I 内核要点

### 6.5.1 CST (element/cst.py): 最简内核的完整形态

CST 是内核协议的“最小完备实现”，代码三块结构清晰：

```
def _shape_coeffs(xi, yi, xj, yj, xk, yk):
    """CST 形函数系数 (a, b, c) 和单元面积。

    Bathe §5.3.2 Eq (5.43):
        a_1 = x2*y3 - x3*y2,  b_1 = y2 - y3,  c_1 = x3 - x2  (循环置换)
    """
    a = np.array([xj * yk - xk * yj,
                  xk * yi - xi * yk,
                  xi * yj - xj * yi])
    b = np.array([yj - yk, yk - yi, yi - yj])
    c = np.array([xk - xj, xi - xk, xj - xi])
    area = 0.5 * ((xj - xi) * (yk - yi) - (xk - xi) * (yj - yi))
    return a, b, c, area
```

**循环置换**：三个节点的系数由同一个公式轮转下标得到——面积坐标 (§4.3.1) 的直接翻译，比显式写三遍公式更不易出错。`_B_matrix` 按模板

$\mathbf{B} = (1/2A) \cdot [\mathbf{b}_1 \ 0 \ \mathbf{b}_2 \ 0 \ \mathbf{b}_3 \ 0; \dots]$  构造 (§4.3.1 循环置换)；`_element_stiffness` 用  $\mathbf{K}_e = \mathbf{t} \cdot \mathbf{A} \cdot \mathbf{B}^T \mathbf{D} \mathbf{B}$  (1 点积分精确，CST 刚度是常数阵)。

**面积判据的尺度纪律** (`_element_stiffness` 内)：

```
# 面积判据基于单元自身尺寸 (最大边长)，不是坐标绝对值 —
# 绝对 1e-30 会拒纳米单元；改用 max|x_i, y_i| 又误拒远离原点的
# 合法单元 (1e7 偏移的单位三角形，面积 0.5 < 1.42 判据)
scl = max(np.hypot(xj - xi, yj - yi), np.hypot(xk - xj, yk - yj),
          np.hypot(xi - xk, yi - yk), np.finfo(float).tiny)
if abs(area) <= 64.0 * np.finfo(float).eps * scl * scl:
    raise ValueError(f"Element area {area:.3e} ~= 0 - degenerate triangle")
```

注释记录了这条判据的**两代错误设计**：绝对阈值误杀纳米单元、全局坐标误拒远原点单元——最终用“单元自身最大边长”做尺度，与绝对位置/绝对尺寸都无关。这是本项目“禁绝对阈值”纪律的缩影：每一处数值判据都写明**为什么这个尺度是对的**。

### 6.5.2 Q4I (element/q4i.py): 静力凝聚

Q4I 在 Q4 基础上加两个非协调模式 (QM6, Bathe §5.5.1), 代码量集中在两处:

```
# (示意 — 完整实现见 fem2d/element/q4i.py)
def _condensation_blocks(...):
    """组装凝聚分块 K_dd, K_dl, K_ll, K_ld.

    Bathe Eq (4.65): k_cond = k_dd - k_dl * k_ll^-1 * k_ld
    """
    ...
```

**为什么非协调模式不破坏协调性:** 非协调位移在单元边界上不连续, 但弱形式里它们以”内部自由度  $\lambda$ ”身份出现, 只在单元内求解 (静力凝聚把它们消去) ——边界上不贡献任何自由度, 装配后的全局位移场依然是协调的。这就是”非协调但收敛” (满足 patch test 即可) 的理论依据。q4i.py 的 incompatible\_derivatives 提供  $\mathbf{B}_{inc}$  导数项, \_condensation\_blocks 实现凝聚公式——两处都是冻结区, 改前必须金标准逐位验证。

## 6.6 装配代码 (assembly.py)

### 6.6.1 为什么有四条路径

| 路径                         | 实现                        | 用途       | 定位   |
|----------------------------|---------------------------|----------|------|
| assemble_sparse            | = 向量化路径别名                 | 生产默认     | 主入口  |
| assemble_sparse_vectorized | COO 广播 + 分块               | 生产 (大批量) | 主实现  |
| assemble_lil_reference     | LIL 逐单元累加                 | 教学/交叉验证  | 参考实现 |
| assemble_expand            | $K = \sum L^T K_e L$ 稠密展开 | 教学       | 参考实现 |

**保留参考实现是刻意的:** LIL 逐单元循环与向量化路径逐位对比是装配 bug 的终极仲裁 (历史教训: 向量化 COO 的 rows/cols 顺序写反时, 对称矩阵数值不变、第三方非对称内核才暴露——参考实现就是那个”第三方”)。两条参考路径 docstring 都带  $\triangle$  标注, 防止生产误用。

### 6.6.2 向量化路径: 一次 COO 转换

```
def assemble_sparse_vectorized(mesh, batch_elements=
    ASSEMBLY_BATCH_ELEMENTS):
    """Vectorized COO scatter with bounded temporary memory."""
    mesh.build_connectivity()
    ne = mesh.n_elements
```

```

nd = mesh.n_dof
dofs = mesh.element_dofs          # (ne, nldof) 全局 DOF 编号
nldof = dofs.shape[1]

total = ne * nldof * nldof
# 行/列索引由 element_dofs 广播而来 — 一次性生成 (int32 省内存)
rows = np.broadcast_to(
    dofs[:, :, None], (ne, nldof, nldof)).reshape(-1)
cols = np.broadcast_to(
    dofs[:, None, :], (ne, nldof, nldof)).reshape(-1)
data = np.empty(total, dtype=float)
off = 0
for start in range(0, ne, batch_elements):
    stop = min(start + batch_elements, ne)
    Ke = np.asarray(stiffness_batch(mesh, slice(start, stop)),
        dtype=float)
    _check_local_symmetry(Ke, ...)    # scatter 前全量检查
    block = (stop - start) * nldof * nldof
    data[off:off + block] = Ke.ravel()
    off += block
K = coo_matrix((data, (rows, cols)), shape=(nd, nd)).tocsr()
K.sum_duplicates()                  # 共享边 DOF 的叠加
K.eliminate_zeros()
return _check_symmetry(K)           # 全局抽样兜底

```

四个设计点:

1. **COO 一次转换**: 所有单元刚度按 (rows, cols, data) 三元组一次性 tocsr()—— $O(\text{nnz})$  而非  $O(\text{batches} \times \text{nnz})$ 。峰值内存  $\approx$  最终矩阵 + 一个批次 (预分配替代全批次同时驻留的 concat 拷贝);
2. **rows/cols 广播**: dofs[:, :, None] (j 维) 对应 rows、dofs[:, None, :] (k 维) 对应 cols。注释明确警告写反的后果: 对称矩阵数值不变, 非对称内核得到转置——所以 \_check\_local\_symmetry 必须在 scatter 前跑;
3. **分块**: 默认 5 万一批 (ASSEMBLY\_BATCH\_ELEMENTS), 配合内核的 element\_slice 协议 (§6.2) ——30 万单元的内存峰值从“全部 Ke 同时驻留”降到“一批 + 最终矩阵”;
4. **双对称检查**: \_check\_local\_symmetry (scatter 前, Ke 级全量)  $\rightarrow$  \_check\_symmetry (全局抽样 256 行)。为什么抽样就够: 组装错位 (索引 bug) 破坏所有行, 抽样足以捕获; 而全量  $K - K^T$  的转置 + 相减在 10 万 DOF 级峰值内存翻倍, 不可接受。

### 6.6.3 对称检查里的 NaN/Inf 守卫

```

if K.nnz and not np.all(np.isfinite(K.data)):
    # 装配溢出 (微尺度几何 B^T D B ~ E/L^2 中间量 Inf) 会透传到 splu,
    # 误报 "Factor is exactly singular"; NaN > tolerance 恒 False,

```

```
# 对称检查也放行 — 显式拒绝
n_bad = int(np.count_nonzero(~np.isfinite(K.data)))
raise ValueError(
    f"Assembled stiffness contains {n_bad} NaN/Inf entries — "
    "element stiffness overflow (check geometry scale / E / t)")
```

为什么这个守卫必要：NaN > tolerance 恒为 False，‘NaN > tolerance’ 的比较语义让溢出条目静默通过对称检查——如果没有这个显式检查，微尺度溢出会一路透传到 SuperLU，被误报为“Factor is exactly singular”（指向完全错误的排查方向）。注释记录的正是这个误导链。

## 6.7 边界拓扑识别代码 (boundary/)

边界识别是“从网格拓扑还原几何语义”的管线，也是本项目最容易踩坑、因此最谨慎的部分（历史教训：闭环漏边、法向依赖调用者顺序、自交误判——详见 §9.1）。核思路一句话：从邻接图出发恢复有序闭环，再做几何分类。

### 6.7.1 总入口：detect() 的五步管线

```
def detect(mesh):
    """Detect validated, oriented geometric segments on every mesh
    loop."""
    mesh.build_connectivity()
    adjacency = _boundary_adjacency(mesh.boundary_edges)
    if not adjacency:
        return []
    node_loops = _decompose_loops(adjacency)
    if not node_loops:
        return []
    scale = mesh_scale(mesh.nodes)
    loops = _validate_and_nest_loops(mesh.nodes, node_loops, scale)
    segments = [
        segment
        for loop_id, loop in enumerate(loops)
        for segment in _segment_loop(mesh.nodes, loop, loop_id, scale)
    ]
    segments = _merge_adjacent_lines(segments, mesh.nodes, scale)
    _orient_closed_segments(segments)
    segments.sort(key=lambda segment: (
        int(segment.get("info", {}).get("loop_depth", 0)),
        int(segment.get("info", {}).get("loop_id", 0)),
        -len(segment["nodes"]),
    ))
    return segments
```

五步：邻接图（边界边 → 无向图）→ 闭环分解（图遍历还原有序边界环）→ 校验 + 嵌套（自交检测、面积容差、洞的深度）→ 分段（曲率断点切分）→ 合并 + 定向 + 排序。每一步都有对应的失败语义（见下），管线末尾的输出是段列表（type/nodes/coords/label/info），被载荷链（面力、压力）直接消费。

### 6.7.2 闭环分解：度检查 + 图遍历

```
def _decompose_loops(adj):
    """边界邻接图 → 有序闭环列表.

    Returned loops contain each vertex once; closure is a topological
    fact
    verified by returning to ``start``, not a duplicated coordinate
    sample.
    """
    invalid_degrees = {
        int(node): len(neighbors)
        for node, neighbors in adj.items()
        if len(neighbors) != 2
    }
    if invalid_degrees:
        preview = ", ".join(...)
        raise ValueError(
            "Open or non-manifold boundary topology: every node of a
            2-D "
            f"domain boundary must have degree 2; found {preview}."
        )

    visited = set()
    loops = []
    for start in sorted(adj):
        if start in visited:
            continue
        if len(adj[start]) == 0:
            visited.add(start)
            continue
        loop = [start]
        visited.add(start)
        cur = list(adj[start])[0]
        prev = start
        while cur != start:
            if cur in visited:
                break
            loop.append(cur)
            visited.add(cur)
            nxt = None
```

```

        for nb in adj[cur]:
            if nb != prev:
                nxt = nb
                break
            if nxt is None:
                break
        prev, cur = cur, nxt
    if cur != start:
        raise ValueError(
            "Open or non-manifold boundary component encountered
            near "
            f"mesh node {int(cur)}; ...")
    if len(loop) >= 3:
        loops.append(np.array(loop, dtype=int))
    return loops

```

两个前置校验是刻意的前置：

1. **度检查**：二维域边界上每个节点的度必须为 2。度  $\neq 2$  = 开放边界（度 1）或非流形接触（度 3+，如两个域的角点相接）——提前报错，而不是在图遍历里产出残缺环；
2. **闭合由拓扑证明**：注释强调“closure is a topological fact verified by returning to start”——闭环的闭合性由 `while cur != start` 的遍历证明，**不是在节点列表末尾重复首节点坐标**（历史教训：用 `zip(ns, ns[1:])` 遍历闭环会漏掉最后一条边  $n_{last} \rightarrow n_0$ ，压力净合力因此非零——闭环必须显式补回首节点，见 §9.1）。

### 6.7.3 校验与嵌套：自交检测 + 洞深度

```

def _validate_and_nest_loops(nodes, node_loops, scale):
    loops = []
    area_tolerance = (
        np.finfo(float).eps * max(float(scale) ** 2, np.finfo(float).
            tiny)
        * 128.0)
    for loop_id, node_ids in enumerate(node_loops):
        coords = nodes[node_ids]
        if has_boundary_self_intersection(coords):
            raise ValueError(
                "Self-intersecting, touching or overlapping domain
                boundary "
                f"detected in loop {loop_id}; repair the Gmsh
                geometry ...")
        area = abs(_signed_loop_area(coords))
        if not np.isfinite(area) or area <= area_tolerance:
            raise ValueError(
                f"Boundary loop {loop_id} has zero or numerically "

```



```

        f"degenerate enclosed area ({area:.6g}); ...")
    probe = _loop_probe_point(coords[:, 0], coords[:, 1])
    loops.append(BoundaryLoop(node_ids, area, probe))
# 不同环之间不得相交/相切/重叠
intersecting_pair = _intersecting_boundary_loop_pair(...)
if intersecting_pair is not None:
    raise ValueError(
        "Distinct domain boundary loops intersect, touch or
        overlap: "
        f"loops {first} and {second}. ...")
# 嵌套深度：环内点被多少更大的环包含（偶数 = 外边界，奇数 = 洞）
for index, loop in enumerate(loops):
    px, py = loop.probe
    loop.depth = sum(
        _point_in_loop(px, py, ...)
        for other_index, container in enumerate(loops)
        if other_index != index and loop.area < container.area)
_orient_loops(nodes, loops)
return loops

```

**四层防线：**环内自交（`has_boundary_self_intersection`）、零面积退化、环间相交（`_intersecting_boundary_loop_pair`，两个洞相切/相交时各自“合法”但整体非法）、嵌套深度（洞的判定）。**洞深度 = 偶奇规则：**探针点（环内一点）被多少个更大的环包含，偶数 = 外边界、奇数 = 洞——这是教科书 point-in-polygon 的射线法推广，容差全用相对尺度（`scale` 来自 `mesh_scale`，随模型尺度缩放，不设绝对阈值）。

#### 6.7.4 定向：材料 CCW、洞 CW

```

def _orient_loops(nodes, loops):
    """Orient full loops before segmentation: material CCW, holes CW.
    """
    for loop in loops:
        is_ccw = _signed_loop_area(nodes[loop.node_ids]) > 0.0
        if loop.is_outer != is_ccw:
            loop.node_ids = loop.node_ids[::-1].copy()

```

**为什么定向必须在这里完成：**后续所有段的法向计算（压力载荷）依赖“材料在左手侧”的方向约定。若依赖调用者传入的节点顺序，同一模型两次运行（邻接图遍历起点不同）可能得到不同方向——历史教训（§9.1）：压力法向必须从**相邻单元 CCW 方向**确定，不依赖调用者节点顺序。这里的统一定向让下游永远拿到一致的环方向。

#### 6.7.5 插件识别体系：首个非 None 胜出

分段之后，每段交给识别器判定几何类型（直线/圆弧/椭圆弧/一般曲线）。设计是插件注册表：



```

class Detector:
    """识别器基类 — 点链 + 可选原生实体信息 + 尺度 → Detection 或
        None."""

    name: str = "detector"

    def detect(self, points, *, scale, is_outer, closed,
               native_entities=()):
        raise NotImplementedError(f"{type(self).__name__}.detect 未实
            现")

class DetectorRegistry:
    """有序识别器注册表 — 首个返回非 None 的探测器胜出。

    兜底 GeneralCurveDetector 恒返回，故 classify 永不落空；
    注册表被清空/兜底缺失 → 响亮报错而非静默返回。
    """

    def add(self, detector: Detector) -> None:
        if not isinstance(detector, Detector):
            raise TypeError(...)
        if any(existing.name == detector.name for existing in self.
            _detectors):
            raise ValueError(
                f"register_detector: 识别器名 {detector.name!r} 已注
                    册 "
                "— 插件名必须全局唯一")
        self._detectors.append(detector)

    def classify(self, coords, scale, is_outer, closed=None,
                *, native_entities=()):
        for detector in self._detectors:
            detection = detector.detect(coords, scale=scale,
                                       is_outer=is_outer,
                                       closed=is_closed,
                                       native_entities=...)

            if detection is not None:
                return detection.type, detection.label, dict(
                    detection.params)
        raise AssertionError(
            "识别器注册表空转：兜底 GeneralCurveDetector 未注册， "
            "classify 必须恒有结果")

```

### 三个设计约束与理由：

1. **顺序敏感 + 兜底**：首个返回非 None 的探测器胜出；GeneralCurveDetector（一般曲线拟合）恒返回，故 classify 永不落空——语义是“最具体的先试，最一般的最后兜底”。注册表被清空/兜底缺失时 AssertionError 响亮报错，绝不静默返回；
2. **名字全局唯一**：插件名冲突抛错——两个插件同名会让移除/覆盖语义混乱；
3. **native\_entities 透传**：Gmsh 的物理实体信息（原生几何标签）原样传给每个探测器——语义信息与几何猜测并存，探测器可以“先用原生信息、几何做校验”（插件示例：circle\_label / ellipse\_group\_label，见 boundary/plugins/）。

### 6.7.6 语义优先级：命名总入口

```
def build_boundary_segments(mesh, *, registry=None, edge_labels=None,
                           geo_path=None, diagnostics=None, strict=False):
    """Build one validated boundary model from topology and Gmsh semantics.

    The mesh topology is always authoritative for edge existence,
    connectivity, loop orientation, inner/outer depth and geometric
    classification. Gmsh Physical Curves are authoritative for names
    and
    exact edge membership.

    Priority:
    1. in-memory Gmsh RegionRegistry (full command-language semantics);
    2. edge-label map (原 ELSET 语义的等价接口);
    3. topology/geometry-only detection for unlabelled external meshes.
    """
    segments = None
    if registry is not None and (registry.curves or ...):
        segments = segments_from_region_registry(mesh, registry, ...)
    elif edge_labels is not None:
        segments = segments_from_physical_curves(mesh, edge_labels, ...)
    if segments is None:
        segments = detect(mesh)  # 纯拓扑兜底
    # 段 schema 稳定化：管线出口的段必须完整 (type/nodes/coords/
    # label/info) — 残缺段会在载荷链中被静默消费，先于覆盖检查拒绝。
    validate_segment_schema(segments)
    validate_boundary_segments(mesh, segments)
    ...
    return segments
```

语义与拓扑的分工（docstring 里一句话说清）：拓扑对存在性/连通性/方向/嵌套权威，Gmsh 物理曲线对名称/精确边归属权威。优先级 1→2→3 是”信息越完整越优先”：有 RegionRegistry 用语义，没有就纯几何探测。出口处两道校验（schema 完整性 + 覆盖检查）保证”每条边界边恰好被消费一次”——残缺段在载荷链里被静默消费之前就被拒绝。

### 6.7.7 优缺点分析（诚实评估）

| 方案                   | 优点                 | 缺点                          |
|----------------------|--------------------|-----------------------------|
| 纯拓扑探测（detect）        | 无需 Gmsh 语义；任何网格可用  | 曲率断点对噪声敏感；尖角/圆弧分类有歧义        |
| Gmsh Physical Curves | 名称精确、边归属明确         | 依赖.geo 里的语义定义，外部.msh 无此信息   |
| RegionRegistry       | 全命令语义，可带参数         | 与 Gmsh API 强耦合，维护双路径成本（§10） |
| 插件识别器                | 新几何类型 = 新文件注册，不动主链 | 顺序敏感，需兜底；插件质量参差             |

为什么保留三条路径：教学场景覆盖”带语义的 Gmsh 模型”与”裸.inp/.txt 网格”两端——任何一端放弃都会让另一半用户无解。代价是双路径同步维护（安全净化、配置语义两处都要改），已在 §10 记为已知弱项。

### 6.7.8 拓扑识别的实战判读（优缺点实证）

优缺点表不是抽象说辞——用两个真实测试场景看它们如何在运行中显现（对应 §9.10 逐行走读的同一份测试）。

把正 24 边形的每条边再细分 9 段（测试辅助 `_subdivide_chords`），得到 216 个严格落在同一圆周上的节点。纯拓扑探测器按”逐点曲率断点”工作：曲率处处相等  $\Rightarrow$  无断点  $\Rightarrow$  判为一条圆弧，并拟合出半径 8.0（误差  $< 10^{-12}$ ，`classify` 的 `info["radius"]`）；而真正的正六边形（曲率集中在 6 个顶点）被干净地判为 6 条直线。两个判断都正确，但依据完全不同——这是纯拓扑的强项：不需要任何外部语义，纯几何即可完成分类。

它的代价在同一测试里看得见：`classify()` 的判据依赖特征尺度 `scale`。尺度错 10 倍，曲率容差跟着错 10 倍，细圆弧可能被误判成多段直线——这正是优缺点表里”曲率断点对噪声敏感”的量化含义。而 Gmsh 语义路径下 `$PhysicalCurve` 的名字直接给出”这条边是什么”，绕开几何推断，这类歧义根本不产生。

外 12 边形 + 内 12 边形孔（CPS4 网格）只来自裸节点表：没有 Gmsh 物理组。探测器必须自己判定哪条闭环是外边界（外法向朝外）、哪条是孔（外法向朝内）。`detect_boundaries` 用有向面积符号区分：外环逆时针（面积  $> 0$ ）、孔顺时针（面积  $< 0$ ），结果标注在 `info["is_outer"]`。压力载荷沿边界施加时，法向由相邻三角

形 CCW 方向确定 (§6.7.4 的定向约定)，内外环的法向因此自动正确——不需要用户在 .geo 里写任何东西。这是纯拓扑路径不可替代的价值：任何合法网格都能识别，哪怕来源只是一张节点表。

两条路径的不可覆盖之处正好互补：语义路径对裸网格无信息可用，纯拓扑路径对“高精度圆弧判读”依赖尺度与容差。本项目的选择不是二选一，而是**优先级链**：有语义用语义（准确），无语义用拓扑（通用），两路径出口统一为同一段结构 (type/coords/info)，下游载荷链只认段结构、不认来源。这样“优缺点”被消化成一条确定性规则——这也是 §9.10 测试里“同一出口结构、两路径结果一致”被锁定的原因。

## 6.8 求解器主流程 (solver.py)

solve() 是五阶段管线 (§5.2) 中“求解”一段的实现，也是全书交叉引用最密集的单体函数：§4 的单元/载荷/误差估计、§6.3–6.6 的内核与装配都在这里汇合。四个小节按“主流程 → 数值守卫 → 语义防线 → 契约”展开。

### 6.8.1 阶段化主流程

solve() 是“教科书流程 + 防御性检查”的逐行实现，主流程分为 8 个阶段 (docstring 与代码注释同步编号)：

```
def solve(mesh, method="elimination", verbose=True,
          check_condition=False, linear_solver="auto"):
    log = print if verbose else (lambda *a, **k: None)
    if not (hasattr(mesh, "validate_state") and hasattr(mesh, "n_dof"
    )):
        # 非 Mesh (dict/None/标量) 会冒裸 AttributeError — 类型契约前置
        raise TypeError(
            f"solve: mesh 必须是 fem2d.Mesh 实例, got {type(mesh).__name__}")

    # 0. 状态校验：构造后字段可被重写，求解前快速失败而非静默错解
    mesh.validate_state()
    # 1. 拓扑 + Jacobian 检查（组装前，快速失败）
    mesh.build_connectivity()
    jac = mesh.element_kernel.jacobian_report(mesh)
    if not jac.ok:
        raise RuntimeError(
            f"Mesh contains {len(jac.bad)} invalid ... elements: "
            f"{jac.inverted} inverted, {jac.degenerate} degenerate ...")
    # 2. 约束划分 + 刚体模态检查 + Q4R 长宽比告警
```

```

fixed_dofs, prescribed, free_dofs = _partition_dofs(mesh, method,
...)
_check_rigid_body_constraints(mesh)          # rank<3 禁止求解
_q4r_aspect_ratio_warning(mesh)
# 3. 组装 (验证全部通过后才进行)
K = assemble_sparse(mesh)
F = assemble_loads(mesh, n_dof)
# 4. 求解 + 解有限性检查
(u, reactions, K_mod, F_mod, info, dirichlet_only) =
_solve_linear_system(...)
Ku = K.dot(u)                                # K·u 只算一次, 各消费者
      共享
_check_solution_finite(u, reactions, method)
# 4'. 残差 (Bathe §8.2.6 backward error)
residual, _ = _compute_residual(K, F, K_mod, F_mod, u, free_dofs,
...)
# 5. 应力响应 + 沙漏监控
u_e = u[mesh.element_dofs]
stress, strain, vm, ... = _compute_element_response(mesh, u_e)
internal_energy, hg, hg_elem, hg_ratio = _hourglass_monitor(mesh,
...)
# 5.5 全局平衡检查 (残差小 != 载荷方向/厚度正确)
balance_data = _global_balance_check(mesh, K, F, u, reactions,
...)
# 6. 小变形 + 结果有限性 + 条件数 (可选)
small_deformation_ok = _small_deformation_check(mesh, u)
_check_result_finite(stress, strain, vm)
result = { ... }                             # 固定键集, 条件项按参数附加
return result

```

阶段划分的动机：每个 `raise` 都发生在“最不浪费计算”的位置——Jacobian 检查在组装前（无效单元先于组装抛错）、约束检查在组装前（无效 BC 不触发无意义求解）、结果有限性在返回前（NaN 不出主流程）。阶段注释给出教材出处（Bathe §4.2/§8.2.6），代码即文档。

## 6.8.2 奇异守卫：秩警告转异常

```

def _solve_with_singular_guard(fn, *args, **kwargs):
    """Run a sparse solve, converting MatrixRankWarning into a loud
    error."""
    with warnings.catch_warnings(record=True) as w:
        warnings.simplefilter("always")
        result = fn(*args, **kwargs)
    # catch_warnings 捕获即吞 — 非秩警告必须在块外原样转发
    for wi in w:

```

```

        if not issubclass(wi.category, MatrixRankWarning):
            warnings.warn_explicit(wi.message, wi.category,
                                   wi.filename, wi.lineno)

    for wi in w:
        if issubclass(wi.category, MatrixRankWarning):
            raise RuntimeError(
                "Linear system is singular or ill-conditioned. "
                "Check boundary constraints, isolated nodes, "
                "degenerate elements, and disconnected components."
            ) from None
    return result

```

**为什么要这个守卫：**SciPy 的 `spsolve` 对近奇异矩阵可能不抛 `MatrixRankWarning` 而返回全零/NaN——调用方拿着 NaN 继续画云图、做误差估计，所有输出全废（历史教训：欠约束模型必须抛异常，不能返回 NaN 或零解）。守卫把秩警告转成带排查方向的 `RuntimeError`，且只转秩警告——其他警告（如 `spsolve` 的近奇异 `RuntimeWarning`）原样转发，不选择性吞掉（注释解释了 `catch_warnings` 块内转发的自增长陷阱）。

### 6.8.3 平衡检查：残差之外的语义防线

```

# 5.5 全局力/力矩平衡检查 (Bathe §4.2.2)
# 残差小只说明线性方程解得准，不代表载荷方向/厚度/边界正确
balance_data = _global_balance_check(mesh, K, F, u, reactions,
                                     fixed_dofs, _dirichlet_only, log
                                     )

```

**为什么残差检查还不够：**后向残差  $\|r\|_\infty / (\|K\|_\infty \|u\|_\infty + \|F\|_\infty)$  只证明“线性方程组解得准”，不证明“方程组本身是对的”——压力方向反了、厚度忘设了，残差照样小。全局  $\Sigma F$ 、 $\Sigma M$  平衡检查（相对尺度 `tol_rel`，无绝对阈值）抓的是物理层错误。判据消息共享同一分母（`_balance_failure_message` 独立成函数使判据与消息不漂移）。

### 6.8.4 返回契约：固定键集

`solve()` 的返回 dict 有**固定键集**（约 20 个键，完整清单见 §7.11）：位移场、支反力、全局平衡、沙漏诊断各归其位；条件性键（`condition_info`、`hourglass_energy_elem`）按参数附加。**为什么契约必须固定：**下游（报告/后处理/测试）不检查键存在就消费——键漂移 = 静默崩溃（`KeyError` 是最温和的情况）。金标准逐位锁（§9.2）连**键集合**都锁定：多一个键、少一个键都红。

## 6.9 关键数值模块代码解析（`patch_test` / `quality` / `error_est` / `visualize`）

本章最后补四个**教学重心**模块的实现走读：分片试验（`patch_test.py`，223 行）、网格质量（`quality.py`，105 行）、Z2 误差估计（`error_est.py`，863 行）与等应力带绘制（`visualize.py` + `runner.py`）。它们的共同哲学与 §6.3–6.6 一致：**先校验、后计算、再归一**

化——把教科书公式（Bathe §5.3.3、§4.3.6）写成防微尺度、防大尺度、防静默错误的代码。”单元边界向量”（面力/压力的等效节点力向量）已在 §4.9.2 走读，B 矩阵实现见 §6.3.2 链式法则——两者是”把连续量离散到节点”这一哲学的两种形态。

### 6.9.1 Z2 估计器的实现 (error\_est.py)

estimate() 的入口契约先于一切计算：

```
if not isinstance(result, dict) or "stress" not in result:
    raise ValueError(
        "estimate_error: result 必须是 solve() 的返回 dict 且含 "
        "'stress' 键, got ...")
stress = np.asarray(result["stress"], dtype=float)
if not np.all(np.isfinite(stress)):
    raise ValueError("estimate: result['stress'] 包含 NaN/Inf ...")
```

理由：非 solve() 输出的 dict 会冒裸 KeyError；NaN/Inf 会让 SPR 恢复在坏数据上白跑（审查 2026-08-06 加的前置拒绝——非法数据不得静默，§10 教训：拿着 NaN 继续画云图全废）。

恢复源三选一（对应 §4.5 的三种恢复法）：method="SPR" → spr\_recovery（局部最小二乘，组件先恢复纪律：先恢复分量再从分量算 Mises/主应力，§4.5.3）；"L2" → nodal\_L2\_projection（全局一致质量阵投影，§4.5.4）；"weighted" → nodal\_weighted（传统面积加权）。

柔度归一化——能量内积的权重矩阵：

```
D = D_matrix(mesh.E, mesh.nu, mesh.plane_type)
D_inv = np.linalg.inv(D)
d_scale = max(float(np.max(np.abs(D_inv))), np.finfo(float).tiny)
D_inv = D_inv / d_scale
```

理由：不归一化时中间量  $t d A \mathbf{D}^{-1} \sigma^2$  在极端材料/几何尺度下溢出/下溢， $\eta$  静默跳到 100%/0%——归一化后三项乘积全部  $O(1)$ 。

对逐单元能量误差：走 \_element\_energy\_errors 的批量路径。

按 ASSEMBLY\_BATCH\_ELEMENTS 分块，保证内存有界：

```
shape = kernel.recovery_shape_matrix(mesh)      # (n_q, 4) 恢复形函数
weights = kernel.recovery_weights(mesh)         # (ne, n_q) detJ 积分权
maxw = float(np.max(np.abs(weights)))
norm_w = weights / maxw                         # 全局尺度下  $O(1)$  权重
s_star_pt = np.einsum("qn,enc->eqc", shape, s_star[conn])
error_density = np.einsum("...i,ij,...j->...", diff_n, D_inv, diff_n)
elem_err_sq[start:stop] = np.sum(norm_w * error_density, axis=1)
```

两个注释写明、不踩就会错的坑：① weighted 方法的  $\sigma^*$  来自单元均值恢复，与积分点原始应力量纲语义不可比——混用会让精确线性场报 ~14% 虚假误差；② Q4 的恢复积分点是  $2 \times 2$  Gauss 点，stress\_qp 形状不符即报错（契约，不静默）。

尺度乘回（对数空间）——物理量恢复只影响报告绝对值，不影响  $\eta$  比值：



```
log_factor = (np.log(s_scale) + np.log(vol_sqrt) + 0.5*np.log(d_scale
))
if log_factor > np.log(np.finfo(float).max): factor = np.inf
elif log_factor < np.log(np.finfo(float).tiny): factor = 0.0
else: factor = float(np.exp(log_factor))
```

三个因子跨度可达  $\pm 300$  阶，连乘必存在一种先溢出/下溢的顺序——对数相加再 `exp` 诚实给出 `0/inf`；`math.exp` 在越界时抛 `OverflowError (E=1e-150/t=1e150/ $\sigma$ =1e308` 复现过的全有限输入崩溃)，用显式分支代替。

**全局量与判读：** $\eta$  与逐单元贡献百分比都在归一化空间计算（比值尺度无关；乘回后微尺度下  $\sim 10^{-308}$  次正规求和失真）。输出三段建议： $\eta > 20\%$  全局加密； $> 10\%$  局部加密； $< 10\%$  OK 但奇点/集中载荷/约束角仍要人工核（输出里的 [ADVICE]/[NOTE]/[OK]）。

### 6.9.2 网格质量评分的实现（quality.py, 105 行）

`_compute()` 五组指标全向量化、无逐单元循环。先看“面积、长宽比、内角”三组几何指标：

```
polygon = mesh.nodes[mesh.elements]
following = np.roll(polygon, -1, axis=1) # 下一条边
lengths = np.linalg.norm(following - polygon, axis=2)
ratio = lengths.max(axis=1) / (lengths.min(axis=1) + np.finfo(float).
tiny)
```

内角计算的手性安全（最容易写错的一行）：

```
turn = np.degrees(np.arctan2(cross, dot)) # 外转角（带符号）
ccw = (np.asarray(mesh.signed_areas, dtype=float) > 0.0)[: , None]
interior = np.where(ccw, 180.0 - turn, 180.0 + turn)
interior = np.where(interior <= 0.0, interior + 360.0, interior)
```

外转角 = 相邻边 incoming/outgoing 的叉积/点积反正切；CCW 单元内角 =  $180^\circ - turn$ ，CW（负面积）单元 =  $180^\circ + turn$ ——只按 CCW 公式会把手性反转单元的内角算成补角（矩形 CW 单元报  $270^\circ$  而非  $90^\circ$ ，注释写明的坑）； $180^\circ$  附近的内角先做  $\leq 0$  修正再取 `min/max`。

**Jacobian 指标走内核共享容差：**`jacobian_determinants + jacobian_report` 直接来自 `element_kernel`——与求解器拦截（§6.8.1 第 1 阶段）共享同一容差，两处实现分叉会静默不一致（历史教训）；`detJ` 原始数组只用于 `jacobian_min` 统计。

**互斥分类**（§4.8 的硬要求，`bad/warn/ok` 严格互斥）：

```
ang_bad_mask = (min_ang <= 15) | (max_ang >= 150)
ang_warn_mask = (((min_ang > 15) & (min_ang <= 30))
| ((max_ang >= 120) & (max_ang < 150)))
ang_warn = int(np.sum(ang_warn_mask & ~ang_bad_mask))
ang_ok = n - ang_bad - ang_warn
```



warn 掩码再与 `~ang_bad_mask` 求与——计数之和恒等于单元总数（历史教训：半开区间写错导致双重计数、评分权重被稀释，§10.1 表）。

**评分与等级：** $score = a\_ok/n \cdot 30 + r\_ok/n \cdot 30 + ang\_ok/n \cdot 40$  ( $a\_ok$  = 通过 Jacobian 检查的单元数，即  $n - jacobian\_neg$ ；角度权重最高——它最直观反映形状质量)；等级  $A > 95 / B > 85 / C > 70 / D$ ；inverted 或 degenerate 大于零一票否决 → F（反向/退化单元没有讨论质量的空间）。分类统计三组权重合计恰好 100，评分可读作“OK 比例加权平均”。

### 6.9.3 分片试验的实现 (patch\_test.py)

`run_patch_test()` 的四个要素（对应 §4.7 的理论）：

①不规则 patch 网格——内部节点偏离中心，规则网格“刚好能过”会掩盖缺陷：

```
nodes = np.array([[0.3, 0.2],      # 0 内部节点（偏离中心）
                  [0.0, -1.0], [1.2, -0.3], [0.8, 1.2], [-0.9, 0.4]])
elements = np.array([[0, 1, 2], [0, 2, 3], [0, 3, 4], [0, 4, 1]])
```

Q4 版为 9 节点 4 单元（内部节点偏移 0.12, 0.08）。

②三个独立常应力状态（Bathe：平面问题必须测三个）： $\sigma_{xx} = 1 / \sigma_{yy} = 1 / \tau_{xy} = 1$ 。

③位移场按平面态分支（Hooke 逆变换 + 刚体位移取零）：

```
if plane == "stress":
    ex = (sx - nu*sy) / E; ey = (sy - nu*sx) / E
else:  # plane strain: 用 (1-nu^2) 修正
    ex = ((1-nu**2)*sx - nu*(1+nu)*sy) / E
    ey = ((1-nu**2)*sy - nu*(1+nu)*sx) / E
gxy = txy / G
# u = ex*x + 0.5*gxy*y ; v = 0.5*gxy*x + ey*y
```

④边界节点全约束 + 三判据——边界节点全部施加精确位移 (`fix_node x/y`)，内部节点自由：检验“内部节点自动得到精确位移”。

```
u_rel = u_error / (u_ref + np.finfo(float).tiny)  # 无条件相对化
s_rel = s_error / (s_ref + np.finfo(float).tiny)
qp_rel = qp_error / (s_ref + np.finfo(float).tiny) # 全部 2x2 Gauss
点
passed = u_rel < tol and s_rel < tol and qp_rel < tol
```

三个细节（注释写明，都是历史教训）： $u\_ref$  极小 ( $< 10^{-15}$ ) 时退回绝对误差——微尺度位移模型下相对误差 50% 也能过检验（静默错误）；Q4 的单元代表应力会掩盖单个积分点错误——分片检验必须覆盖全部  $2 \times 2$  Gauss 点 (`stress_qp`)，CST 同走此检查（单点等价）；判据三条全  $< 10^{-10}$  才 PASS，否则响亮 FAIL。`run_patch_test` 对每个注册单元执行 = 内核注册的必经门槛 (§6.2 协议层)。

### 6.9.4 等应力带法的实现 (visualize.py + runner.py)

§4.10 的三个关键要求加上两个实现细节（浮点整除坑、超界告警）在代码里的落点：

①**带宽固定**——runner 层数计算（校验已由 config.validate 统一拦截，这里只算层数）：

```
ratio = (config.band_max - config.band_min) / config.band_step
n_bands = int(round(ratio))          # 浮点整除坑：round 而非 floor
isoband_levels = config.band_min + np.arange(n_bands + 1)*config.band_step
isoband_levels[-1] = config.band_max # 尾部浮点误差归位
```

浮点整除坑（注释写明）： $0.3/0.1 = 2.99999\dots$ ，floor 截断会少一个带、末带静默加宽一倍（ $0..0.3@0.1$  生成  $[0, 0.1, 0.3]$ ，末带宽 0.2）——用 round + 尾部归位。

②**水平等距硬校验**——\_validate\_isoband\_levels: 1D / 至少 2 值 / 全有限 / 严格递增 / 等距（allclose rtol 1e-10）。”带宽固定”被实现为**入口校验**：非法水平直接报错，而不是画一张”看起来对”的图。

③**离散着色**——BoundaryNorm + 自定义离散 colormap：

```
band_rgba = CMAP(np.linspace(0.05, 0.95, n_bands))
band_cmap = ListedColormap(band_rgba, name='isoband')
band_norm = BoundaryNorm(band_levels, ncolors=n_bands, clip=True)
tpc = ax.tripcolor(tri, facecolors=values, cmap=band_cmap,
                  norm=band_norm, shading='flat')
```

每带一色、带边界精确落在水平值上；shading='flat' = 单元代表值（is\_element 位置）。

④**不画网格线**——Sussman-Bathe 的全部信息量在”带边界在单元边处断裂”，全屏网格线会让断裂与网格线无法区分（Bathe §4.3.6 Fig 4.15，代码注释原文）。

⑤**超界警告**——值超出固定水平的单元计数打印：

```
below = int(np.count_nonzero(values < band_levels[0]))
above = int(np.count_nonzero(values > band_levels[-1]))
if below or above:
    print(f' isoband warning: {below} elems below min=...')
```

clip=True 会把超界值静默夹到首末带——固定水平  $[0, 30\text{MPa}]$  而孔边峰值 40MPa 时，不提醒用户会误读云图。色条附带 [bands: 带宽 wide, N bands] 标注——与 Gouraud 连续云图（§4.5 磨平）互补：连续图看趋势，等应力带看断裂。

## 6.10 设计理由总结：为什么代码这样安排

把本章的”为什么”收拢成一张对照表：

**一句话总结**：本求解器的代码安排服务于三个目标——**公式可溯源**（每行都有教材出处）、**行为可验证**（每个数值行为都有测试锁定）、**错误不静默**（每个误用路径都有带上下文的报错）。三者互相支撑：可溯源让代码好审查，可验证让重构安全，不静默让失败可定位。

| 代码安排                        | 理由                                        | 反面代价                        |
|-----------------------------|-------------------------------------------|-----------------------------|
| 单元公式全部进内核类                  | 新单元零改外部模块；公式只出现一次                         | 协议变更波及所有内核                  |
| 几何缓存进 Mesh                  | 30 万单元组装 1.5s；数学前置在 build 一次完成            | 缓存失效纪律（指纹机制）                |
| 局部坐标计算                      | 大坐标/微尺度鲁棒（ $10^{12}$ 偏移 / $10^{-150}$ 尺寸） | 所有公式都要考虑平移不变性               |
| 向量化路径 + 参考路径并存<br>冻结区 + 金标准 | 教学可读 + 性能仲裁（逐位对比）<br>数值行为永不被”顺手优化”破坏      | 双份实现要同步演进<br>冻结区内重构成本高（§10） |
| 五阶段单主线<br>识别器插件注册表          | 任何结果可复现、可定位<br>新几何类型可插拔                   | 旁路需求只能走主流程<br>顺序敏感，需兜底探测器   |
| 错误全部带上下文                    | 用户误用定位快；错误不会静默                            | 消息字符串是隐式契约（探针锁定）            |

## 7 API 参考

本章为全部公开 API 的输入/输出/异常契约，来源于 docs/api\_contract.md（契约表与代码实测三方一致，探针 151 项断言全过）。状态图例：✓ = 已达标（全部误用路径有带上下文的领域错误）；△ = 有缺口（列明）；— = 不适用/内部。

### 7.1 全局约定

| 约定      | 内容                                                         |
|---------|------------------------------------------------------------|
| 错误类型    | 用户误用 → ValueError / TypeError / CliError(exit_code) 带参数上下文 |
| 消息格式    | 函数名：参数名 = 值 — 原因，期望                                        |
| DOF 编号  | 节点 i → $x=2i$ , $y=2i+1$ （Bathe Eq 4.18）                   |
| NaN/Inf | 一律拒绝（有限性），不允许静默传播或静默夹值                                     |
| 布尔掩码    | [True, False] 一律拒绝（禁止折叠成 DOF 0/1）                          |
| 多余分量    | 静默忽略 = 最危险错误；必须报错                                          |
| 微尺度/大坐标 | 1e-150 几何 / 1e12 坐标 / 1e-310 载荷必须保持有限                      |

### 7.2 Mesh 类

#### 7.2.1 构造器

```
Mesh(nodes, elements, thickness=1.0, E=210e9, nu=0.3,
      plane_type="stress", fixed_dofs=None, prescribed_vals=None,
      body_force=None, surface_tractions=None,
```

```
concentrated_forces=None, elem_type="CPS3")
```

| 参数                      | 合法形状                                      | 误用清单 → 应有错误                                                                                       | 现状                 |
|-------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------|
| nodes                   | (n_nodes,2) 有限实数数组; 非空                    | 标量/1-D/3-D → ValueError(形状); NaN/Inf → ValueError; 空 → ValueError                                 | ✓                  |
| elements                | (n_elem,npe) 整数索引; 非空; npe 与 elem_type 匹配 | 形状错 → ValueError; 浮点索引 → ValueError(拒绝截断); 负/越界 → ValueError; NaN → ValueError; 重复单元 → ValueError | ✓                  |
| thickness/<br>E/nu      | 有限正标量 / nu∈(-1,0.5)                       | NaN/Inf → ValueError; t≤0/E≤0 → ValueError; nu 越界 → ValueError; 非数值(str) → TypeError              | ✓                  |
| plane_type              | "stress" 或 "strain"                       | 其他字符串 → 构造期未查(validate_state 兜底)                                                                  | ✓ 有<br>意<br>决<br>策 |
| fixed_dofs              | (n_fixed,) 整数 DOF 索引                      | 布尔掩码 → TypeError; 浮点非整数 → ValueError; 负/越界 → ValueError; 重复 → 去重(幂等合法)                            | ✓                  |
| prescribed_<br>vals     | dict{DOF int → 有限值}                       | 键不在 fixed_dofs → ValueError; 值 NaN/Inf → ValueError                                               | ✓                  |
| body_force              | None   callable   (bx,by)                 | 1/3 分量 → ValueError; NaN → ValueError; 标量 → ValueError                                            | ✓                  |
| surface_<br>tractions   | list of dict                              | 非 dict/缺键/节点非法/内部边/形状错 → ValueError                                                               | ✓                  |
| concentrated_<br>forces | list of dict                              | 同上; 分量 callable → ValueError                                                                      | ✓                  |
| elem_type               | 已注册单元类型                                   | 未注册 → ValueError(带注册表); 构造后赋值 → AttributeError(只读)                                                | ✓                  |

### 7.2.2 节点索引 API (所有 nid/ni/nj 共用契约)

**契约:** 整数 (接受 "恰为整数" 的浮点如 1.0, 拒绝 bool); 范围 [0, n\_nodes); 越界/负/布尔 → 带函数名的 ValueError/TypeError。

| API                     | 签名                           | 误用清单 → 应有错误                                                                                                            |
|-------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------|
| fix_node                | (nid, dof="both", value=0.0) | nid 非整数/布尔 → TypeError; 越界 → ValueError; dof $\notin$ {x,y,both} → ValueError; value NaN → ValueError; 非数值 → TypeError |
| fix_nodes_func          | (node_list, func)            | 标量传入 → ValueError(需列表); 越界 → ValueError; func 返回 3+ 分量/NaN → ValueError; 不可迭代 → TypeError                              |
| add_force               | (nid, fx=0.0, fy=0.0)        | nid 非法/越界 → TypeError/ValueError; fx/fy NaN → ValueError; 非数值 → TypeError                                              |
| add_traction            | (ni, nj, tx, ty)             | 同上; 内部边 → ValueError; 零长边 → ValueError(ULP 相对判据)                                                                       |
| add_pressure            | (ni, nj, p)                  | 同上; p NaN → ValueError; 非数值 → TypeError; callable 不预调用                                                                 |
| boundary_outward_normal | (ni, nj)                     | 非法/越界 → TypeError/ValueError; 内部边/零长边 → ValueError                                                                     |

### 7.2.3 结构修改与查询

| API              | 签名                     | 契约要点                                                                                                     |
|------------------|------------------------|----------------------------------------------------------------------------------------------------------|
| replace_nodes    | (new_nodes)            | 形状 $\neq$ 原 → ValueError(保持节点数); NaN → ValueError                                                        |
| replace_elements | (new_elements)         | 形状/空/浮点/越界/重复 → ValueError (与构造器同族)                                                                      |
| nodes_on_edge    | (axis, edge, tol=None) | axis $\notin$ {x,y}/edge $\notin$ {min,max} → ValueError; tol=None → span $\times$ 1e-8 自动 (微尺度 tiny 兜底) |
| info             | ()                     | 内部 check_jacobian 异常可冒泡 (网格已校验)                                                                          |
| validate_state   | ()                     | 材料/BC/载荷/缓存一致性全查 → ValueError/TypeError                                                                  |
| check_jacobian   | ()                     | 返回 (ok, bad)                                                                                             |

|                                                            |                         |                                   |
|------------------------------------------------------------|-------------------------|-----------------------------------|
| <code>check_rigid_body_</code><br><code>constraints</code> | <code>()</code><br><br> | 返回 issues list; 逐连通分量中心化 +<br>归一化 |
|------------------------------------------------------------|-------------------------|-----------------------------------|

---

### 7.3 求解与 BC

| API                             | 签名                                                                                                                                              | 误用清单 → 应有错误                                                                                                                                                                                                                                                                       |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>solve</code>              | (mesh,<br>method="elimination",<br>verbose=True,<br>check_<br>condition=False, lin-<br>ear_solver="auto")                                       | method 非法 → ValueError; lin-<br>ear_solver 非法 → ValueError;<br>penalty×cg/ilu/cg-block → Val-<br>ueError("Penalty constraints cur-<br>rently require the direct solver");<br>mesh 非 Mesh → TypeError; 欠约<br>束 → RuntimeError(刚体模态);<br>奇异 → RuntimeError; NaN 解<br>→ RuntimeError |
| <code>estimate_condition</code> | (K, method="auto")                                                                                                                              | K 非数组/非方阵 → ValueEr-<br>ror "K 必须为方阵"; method<br>非法 → ValueError; 奇异<br>→ dict{condition_number:<br>inf/None, status}                                                                                                                                                             |
| <code>apply_elimination</code>  | (K, F, free_dofs,<br>fixed_dofs, pre-<br>scribed_vals, lin-<br>ear_solver="direct",<br>cg_rtol=1e-10,<br>cg_maxiter=None,<br>return_info=False) | 形状/NaN/布尔掩码/重叠/遗<br>漏/重复/长度不等 → Val-<br>ueError; 重复约束不同值 →<br>ValueError; cg 不收敛 →<br>RuntimeError                                                                                                                                                                                |
| <code>apply_penalty</code>      | (K, F, fixed_dofs, pre-<br>scribed_vals=None,<br>penalty=None)                                                                                  | 同上; penalty None= 自动<br>max K <sub>ii</sub>  ×1e8; penalty 非<br>法/超限 → ValueError(相对<br>判据); 自动罚因子溢出 →<br>OverflowError                                                                                                                                                           |
| <code>estimate_error</code>     | (mesh, result,<br>method="SPR",<br>verbose=True)                                                                                                | method 非法 → ValueError; re-<br>sult 非 dict/缺"stress" → Val-<br>ueError; 形状与 n_elem 不符 →<br>ValueError                                                                                                                                                                             |

---

### 7.4 载荷

---

| API            | 签名            | 误用清单 → 应有错误                                                                                                                                |
|----------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| assemble_loads | (mesh, n_dof) | n_dof $\neq 2 \times$ 节点数 → ValueError "n_dof 必须等于 $2 \times$ 节点数"; callable 求值失败 → ValueError(带边/高斯点); 零长边 → ValueError                   |
| parse_vec2     | (s: str)      | 非 str → ValueError 带类型; 分量数 $\neq 2$ → ValueError; 全角逗号 → 归一化接受; NaN/Inf 字面 → ValueError; 溢出 (1e999) → ValueError; 语法错误 → ValueError(带表达式) |
| parse_traction | (s: str)      | 非 str → ValueError; 无 ':' → (None,0,0,None) 由调用方处理; 3+ 段 → ValueError; profile 非法 → ValueError                                             |

## 7.5 应力与误差估计

| API                            | 签名                         | 误用清单 → 应有错误                                                                                                                                            |
|--------------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| compute_stresses               | (mesh, u)                  | u 形状错 → ValueError(带期望); u NaN/Inf → ValueError "u contains NaN/Inf"                                                                                   |
| nodal_simple<br>nodal_weighted | / (mesh, elem_stress)      | 形状错 → ValueError; 孤立节点 → ValueError; NaN → ValueError                                                                                                  |
| nodal_L2_projection            | (mesh, elem_stress)        | ndim $\notin \{2,3\}$ → ValueError; 首维 $\neq n_{\text{elem}}$ → ValueError; 采样数 $\neq n_{\text{qp}}$ → ValueError; 孤立节点 → ValueError; NaN → ValueError |
| principal_stresses             | (stress)                   | 形状 (n,2)/(n,)/(标量) → ValueError 带期望形状; NaN → ValueError; <b>(3,)</b> 单向量 → 返回 4 标量元组 $(\sigma_1, \sigma_2, \tau_{\max}, \theta)$                       |
| stress_at_point                | (mesh, result, x, y, mode) | mode 非法 → ValueError; 点不在网格 → ValueError; 缺键 → ValueError; x/y NaN → ValueError(带坐标)                                                                   |
| point_in_element               | (mesh, x, y)               | 返回 -1 (不在网格); NaN/Inf/复数 → ValueError                                                                                                                  |

|                                  |                                        |                                                                                                        |
|----------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------|
| spr_recovery                     | (mesh, elem_stress)                    | 首维 $\neq$ n_elem $\rightarrow$ ValueError;<br>NaN $\rightarrow$ ValueError; 孤立节点 $\rightarrow$<br>回退平均 |
| element_refinement_<br>indicator | (mesh, result)                         | result 非 dict/缺"stress" $\rightarrow$ Val-<br>ueError (曾裸 KeyError, 已修)                                |
| compute_traction_<br>jumps       | (mesh, elem_stress,<br>sigma_ref=None) | sigma_ref 非数值 $\rightarrow$ TypeError;<br>NaN/Inf/0/负 $\rightarrow$ ValueError; 校<br>验先于空数据快速返回        |

## 7.6 输入链

| API                            | 签名                                              | 契约要点                                                                                                                                                                           |
|--------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| resolve_input_file             | (fp, config,<br>ask=None)                       | 扩展名不支持 $\rightarrow$ CliError(exit<br>1); .inp $\rightarrow$ CliError(exit 1); .msh<br>导入失败 $\rightarrow$ CliError(exit 1);<br>.spec 网格不存在 $\rightarrow$ CliEr-<br>ror(exit 1) |
| resolve_geo / re-<br>solve_txt | (fp, config,<br>ask=None)                       | gmsh 生成失败 $\rightarrow$ CliError(exit<br>1); quad 验证失败 $\rightarrow$ Gmsh-<br>TopologyError; .txt 解析失败 $\rightarrow$<br>CliError                                               |
| resolve_spec_<br>overrides     | (fp, config)                                    | 格式错误 $\rightarrow$ ValueError(带行号);<br>键值无法转 float $\rightarrow$ ValueEr-<br>ror(带键名); plane 非法 $\rightarrow$ CliEr-<br>ror; 未知键 $\rightarrow$ WARN                            |
| physical_point_<br>from_geo    | (geo_path, name,<br>mesh)                       | 无源.geo $\rightarrow$ "no_geo_source";<br>gmsh 不可用 $\rightarrow$<br>"gmsh_unavailable"; 未找<br>到/歧义/域外 $\rightarrow$ 区分 reason                                                   |
| generate_geo_with_<br>topology | (geo_path, *,<br>quad=False, ...)               | gmsh 缺失/失败 $\rightarrow$ (None,<br>None); 拓扑验证失败 $\rightarrow$ 抛异<br>常; quad 混合 $\rightarrow$ GmshTopolo-<br>gyError                                                           |
| generate_from_geo              | (geo_path, *,<br>quad=False, ...)               | 文件不存在 $\rightarrow$ FileNotFoundError;<br>gmsh 不可用 $\rightarrow$ GmshU-<br>navailableError; 无单元/混合/非<br>平面 z $\rightarrow$ GmshTopologyError                                   |
| import_msh                     | (msh_path, *, re-<br>quire_quads=False,<br>...) | 文件不存在 $\rightarrow$ FileNotFoundError(前置); 无 PhysicalNames $\rightarrow$<br>WARN; 2-D 单元缺失 $\rightarrow$ Gmsh-<br>TopologyError                                                |



|                   |                                  |                                                         |
|-------------------|----------------------------------|---------------------------------------------------------|
| read_geo_groups   | (geo_path, *, gmesh_module=None) | 文件不存在 → None (设计); API 失败 → 文本解析回退                      |
| parse_spec_config | (filepath)                       | 缺 =/空值 → ValueError(行号); BOM → 自动剥离                     |
| validate_mesh     | (nodes, elements, ...)           | 负索引/重复单元/孤立节点/非流形边/退化单元/零长边 → MeshValidationError(分类报告) |

## 7.7 材料与单元注册

| API                 | 签名                                    | 误用清单 → 应有错误                                                                                            |
|---------------------|---------------------------------------|--------------------------------------------------------------------------------------------------------|
| D_matrix            | (E, nu, plane_type="stress")          | $E \leq 0$ /NaN → ValueError; nu 越界 → ValueError; plane 非法 → ValueError; 非数值 → TypeError               |
| von_mises           | (stress, plane_type="stress", nu=0.3) | plane 非法 → ValueError; 标量/1-D/末维 $\neq 3$ → ValueError; NaN → ValueError; <b>(3,)</b> 单向量 → 返回标量 float |
| get_element_kernel  | (elem_type: str)                      | 未注册 → ValueError(带注册表列表); None → ValueError                                                            |
| register_element    | (kernel: ElementKernel)               | 非实例 → TypeError; 空名 → ValueError; 重复键不同内核 → ValueError                                                 |
| verify_all_elements | (mesh, verbose=True)                  | 委托内核校验                                                                                                 |

## 7.8 边界识别

| API                                                  | 签名                                     | 契约要点                                                           |
|------------------------------------------------------|----------------------------------------|----------------------------------------------------------------|
| detect_boundaries                                    | (mesh, ...)                            | 微尺度/大坐标已相对化; 椭圆轴比 tiny 兜底; 闭合环验证                               |
| build_boundary_segments                              | (mesh, ...)                            | 无网格 → ValueError                                               |
| validate_boundary_segments                           | (mesh, segments)                       | 缺段/闭合缺失 → ValueError 带统计 (missing/extra/duplicated 计数, 调用链硬校验) |
| describe_geometry / print_segments / parse_edge_name | (segments) / (segments) / (name, segs) | 非法 name → ValueError(解析器)                                      |
| register_detector                                    | (detector: Detector)                   | 非 Detector → TypeError; name 重复 → ValueError                   |

|                       |                                        |                                                                      |
|-----------------------|----------------------------------------|----------------------------------------------------------------------|
| default_registry      | ()                                     | 顺序 = 插件优先 +<br>line→circle→ellipse→general                           |
| _resolve_edge_indices | (edge_str, segs, region_registry=None) | "@ 组名" 不存在 → ValueError;<br>编号 +@ 混用 → ValueError; 无注册表 → ValueError |

## 7.9 配置与质量

| API                                                 | 签名                                                                                             | 契约要点                                                                                                                                   |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| AnalysisConfig.<br>validate                         | ()                                                                                             | $E \leq 0$ /nu 越界/ $t \leq 0$ / $lc \leq 0$ → ValueError;<br>plane/solver/error_method 非法 → ValueError; band 三参数不齐/非整除/超限 → ValueError |
| AnalysisConfig.<br>from_dict                        | (data: dict)                                                                                   | 未知键 → WARN 忽略 (前向兼容, 文档化)                                                                                                              |
| AnalysisConfig.<br>from_args                        | (args)                                                                                         | 无 args 字段 → 跳过 (保持默认)                                                                                                                  |
| evaluate_mesh_<br>quality / report_<br>mesh_quality | (mesh)                                                                                         | 空网格不可构造 (Mesh 构造器拒绝 $n\_elem=0$ )                                                                                                      |
| run_patch_test                                      | ( $E=210e9$ , nu=0.3,<br>plane="stress",<br>tol=1e-10, ver-<br>bose=True,<br>elem_type="CPS3") | $E$ /nu/plane 非法 → D_matrix<br>ValueError; elem_type 非法 →<br>ValueError                                                                |

## 7.10 装配与其他导出

| API                                         | 签名                  | 契约要点                                                                                   |
|---------------------------------------------|---------------------|----------------------------------------------------------------------------------------|
| assemble_sparse                             | (mesh)              | 非对称内核 → RuntimeError(对称性检查); 奇异/NaN → 诊断                                               |
| assemble_sparse_<br>vectorized              | (mesh, ...)         | 同上; 形状错 → ValueError                                                                   |
| run_cantilever_<br>convergence              | (L, H, nx, ny, ...) | elem_type 非法 → ValueError (惰性导出, 见 <code>__init__</code><br><code>__getattr__</code> ) |
| GmshUnavailableError<br>/ GmshTopologyError | —                   | 领域异常类型                                                                                 |

## 7.11 Python 编程接口示例

除命令行外，fem2d 作为 Python 包可直接编程使用。一个完整的” 网格构建 → 求解 → 应力 → 误差估计 → 输出” 流程：

```
import numpy as np
import fem2d

# ---- 1. 构建网格 (2x2 Q4 单元正方形板)
# -----
n = 3    # 每边节点数
x = np.linspace(0, 1, n); y = np.linspace(0, 1, n)
xx, yy = np.meshgrid(x, y)
nodes = np.column_stack([xx.ravel(), yy.ravel()])
elems = []
for j in range(n-1):
    for i in range(n-1):
        a = j*n + i
        elems.append([a, a+1, a+n+1, a+n])
elems = np.array(elems)

# ---- 2. 定义模型 -----
m = fem2d.Mesh(nodes, elems, E=2.1e11, nu=0.3,
               thickness=0.01, elem_type="Q4")
m.fix_node(0, "both")          # 左下角全约束 (dof 只能取 'x'/'y'/'
    both')
m.fix_node(1, "y")             # 右下角竖向约束
m.add_force(n*n-1, 0.0, -1e4)  # 右上角集中力 (向下)
m.add_pressure(0, n, 1e6)      # 左边下段 1 MPa 内压
m.add_pressure(n, 2*n, 1e6)    # 左边上段 1 MPa 内压

# ---- 3. 求解 (默认消去法)
# -----
res = fem2d.solve(m, verbose=True)

# ---- 4. 应力与误差估计 -----
stress, strain, _ = fem2d.compute_stresses(m, res["u"]) # 三元组解包
nodal = fem2d.nodal_weighted(m, stress)                # 面积加权节点恢复
spr = fem2d.spr_recovery(m, stress)                    # SPR 恢复
eta = fem2d.estimate_error(m, res, method="SPR")
from fem2d.error_est import compute_traction_jumps     # 子模块接口
jumps = compute_traction_jumps(m, stress)              # 内部边牵引跳跃列表

# ---- 5. 导出量 -----
vm = fem2d.von_mises(stress)                            # (n_elem,) 单元
    Mises
```

```
s1, s2, tmax, theta = fem2d.principal_stresses(stress[0]) # (3,) → 4
    标量
print("max|u| =", np.abs(res["u"]).max())
print("Z2 eta  =", round(eta["eta"], 2), "%") # 2x2 粗网格偏大, 加
    密收敛
```

逐接口讲解（输出形状均为 2×2 Q4 网格实测）：

| 调用                            | 语义                                                   | 返回形状                                |
|-------------------------------|------------------------------------------------------|-------------------------------------|
| fem2d.Mesh(...)               | 构造模型; elem_type 取 "CPS3"/"Q4"/"Q4R"/"Q4I"（或 CPS 系别名） | —                                   |
| m.fix_node(nid, mode)         | 位移约束; mode 取 "x"/"y"/"both"; 错误写法 "uy" 抛 ValueError  | —                                   |
| m.add_force(nid, fx, fy)      | 节点集中力                                                | —                                   |
| m.add_pressure(ni, nj, p)     | 边 $(n_i, n_j)$ 法向压力, 方向由单元外法向确定                      | —                                   |
| fem2d.solve(m)                | 求解; 返回完整结果字典                                         | dict (约 20 个键)                      |
| compute_stresses(m, u)        | 单元代表应力/应变/Mises 三件套                                  | (stress, strain, vm), 各 (n_elem, 3) |
| nodal_weighted / spr_recovery | 节点应力恢复（面积加权 / 超收敛点）                                  | (n_nodes, 3)                        |
| von_mises(stress)             | 单元 Mises 等效应力                                        | (n_elem,)                           |
| principal_stresses(stress[0]) | 单向量 → 4 个标量; (n,3) 批量 → 四元组各 (n,)                    | 4 标量 / 四元组                          |
| estimate_error(m, res, ...)   | 误差估计; 键 eta 为 Z2 相对误差百分比                             | dict                                |
| compute_traction_jumps        | 内部边牵引跳跃 (§4.9 定义)                                    | list[dict]                          |

res 字典的键（求解报告的所有量都在这里）：

- u（位移）;
- stress/strain/vm\_stress（单元级）;
- reactions/reaction\_vector/reaction\_dofs（支反力）;
- external\_force\_vector、internal\_energy、residual;
- force\_balance/moment\_balance/balance\_ok（全局平衡, §6.8.3）;
- linear\_solver;
- hourglass\_energy/hourglass\_energy\_ratio（Q4R 沙漏监控, §6.4.4）;
- small\_deformation\_ok。

拓扑识别 API——build\_boundary\_segments() 是边界模型总入口 (§6.7 管线: 邻接图 → 闭环分解 → 定向 → 曲率分类), 纯 Python 网格即可用, 不需要 gmsh:

```

segs = fem2d.build_boundary_segments(m)    # 方形板 -> 4 段
print(segs[0]["type"])                    # 'line' (或 'arc'/'ellipse' 等)
print(segs[0]["nodes"])                   # [0, 1, 2] 段上的节点链
print(segs[0]["label"])                   # 自动命名, 如 "底 直线 (0,0)-(1,0)"
print(fem2d.describe_geometry(segs))      # "矩形板 | 1.00x1.00m"

```

每段是一个 dict: `type` (几何分类)、`nodes` (节点链)、`coords` (坐标数组)、`label` (自动段名)、`info` (起终点/曲率等元数据)。`detect_boundaries(mesh)` 是纯几何检测入口 (无 gmsh 物理组语义时由 `GeneralCurveDetector` 兜底), ”首个非 None 胜出”的插件体系保证外部网格也能得到完整边界模型 (§6.7.5)。

**错误路径**——所有误用都在调用点抛出带上下文的异常, 不会静默:

```

m.fix_node(1, "uy")
# ValueError: fix_node: dof='uy' -- 必须为 'x', 'y' 或 'both'

m.add_pressure(0, 2, 1e6)    # (0,2) 跨节点不是网格边, solve 时校验
# ValueError: Edge (0,2) is not a mesh edge -- nodes must be
#           connected
#           by an element side.

```

统一的错误语言是项目契约的一部分 (探针 151 项断言覆盖): 形状/类型/NaN/越界/布尔掩码各有一致的异常类型与消息模板, 教学场景下排错成本极低 (§10.1 错误消息速查)。

要点:

- 所有 API 的误用路径 (形状/类型/NaN/越界/布尔掩码) 都会得到带上下文的 `ValueError/TypeError`, 不会静默出错
- 构造后不可直接改 `nodes/elements` (只读 setter)
- 改网格须用 `replace_nodes()/replace_elements()`, 二者触发缓存失效
- 求解前 `validate_state()` 自动做材料/BC/载荷一致性全查; 欠约束在求解时被刚体模态检查拦截
- 可编程调用 `run_patch_test()`、`run_cantilever_convergence()` 进行单元与求解器验证

## 7.12 缺口修复历史 (K1–K13)

| 编号 | 缺口                                                                                   | 位置                              |
|----|--------------------------------------------------------------------------------------|---------------------------------|
| K1 | 标量有限性检查对非数值类型冒裸 <code>Type-Error</code>                                              | mesh/solver/checks.py 共享 helper |
| K2 | <code>fix_nodes_func</code> : 标量 <code>node_list</code> / <code>func</code> 多余分量静默忽略 | mesh.py                         |

|     |                                                        |                             |
|-----|--------------------------------------------------------|-----------------------------|
| K3  | principal_stresses: 形状未校验 → 裸 IndexError               | stress.py                   |
| K4  | estimate_error/stress_at_point: result 缺键 → 裸 KeyError | error_est.py / stress.py    |
| K5  | solve: mesh 非 Mesh → 裸 AttributeError                  | solver.py                   |
| K6  | parse_vec2/parse_traction: 非 str → 裸 AttributeError    | loads_core.py               |
| K7  | import_msh: 文件不存在 → gmsh 异常冒泡                          | gmsh_adapter.py             |
| K8  | compute_stresses/spr/nodal_*: NaN 静默传播                 | stress.py / spr.py          |
| K9  | DOF 数组校验三处重复                                           | mesh.py / bc.py → checks.py |
| K10 | 载荷记录节点整数校验重复                                           | mesh.py                     |
| K11 | 材料参数校验重复                                               | mesh.py                     |
| K12 | von_mises: 形状/NaN 裸错误 (fuzz 发现)                        | material.py                 |
| K13 | bc_apply 解包绕开 schema                                   | bc_apply.py                 |

每个修复都附判别性测试（放回旧实现必须失败），契约映射见 docs/api\_contract.md §M2。

## 8 验证与质量保障

本项目把”验证”做成了制度：不是跑一次算例就声称正确，而是每一层数值行为都有独立机制锁定，任何改动都逃不过。

### 8.1 六层测试体系

| 层次      | 机制                                                         | 作用             |
|---------|------------------------------------------------------------|----------------|
| 1. 金标准  | boundary_golden 段 schema + solve_refactor_lock 逐位锁         | 边界段结构、求解输出逐位不变 |
| 2. 漂移门  | scripts/regression_compare.py, 基线 7ee65fc, 6 项指标相对差 < 1e-9 | 数值行为跨平台逐位一致    |
| 3. 探针   | scripts/audit_contract_probe.py, 151 项契约断言 0 FAIL          | 契约表与代码实测一致     |
| 4. fuzz | scripts/fuzz_api.py 500 (固定种子) + combo_fuzz                | 随机输入不冒裸异常      |
| 5. 分支锁  | test_{solver,...}_branches.py                              | 关键分支行为锁定       |
| 6. 解析解  | E1 悬臂梁 / E2 Kirsch / E3 Z2 效应指数 (见 §7.4)                   | 数值解 vs 理论解对标   |

## 8.2 统一验收入口 verify\_all

scripts/verify\_all.py 是每次改动的门禁：

```
python scripts/verify_all.py --fast      pytest 全量 + 探针 + 漂移门
python scripts/verify_all.py --full      九阶段 (+ lint 全命令 + 无
gmsht 模拟)
python scripts/verify_all.py --no-gmsht 模拟无 gmsht 环境全量回归
python scripts/verify_all.py --stage <名> 单阶段
python scripts/verify_all.py --fake-fail 红侧自证
```

退出码：0 = 全绿；1 = 用法错误；2 = test 组 FAIL；3 = lint 组 FAIL。lint 与漂移门命令与 .github/workflows/ci.yml 双处同步。

漂移门 6 个基线值（钉死，相对差必须 0.000e+00）：

```
max|u|=4.8551138165e-06  energy=2.2668188447e-02  eta=1.5523972662e
+01
sxmax =1.0279627164e+06  syxmax=2.9161118509e+05  txyxmax=2.7962716378e
+04
```

## 8.3 契约测试

契约表配套判别性测试（6 文件）锁定行为：每个 ✓ 有”放回旧实现必须失败”的测试在场。测试分组：contract / boundary / regression\_audit\_2026080{2,3a-e} / usability / analytic / structure。fuzz 曾抓到真实缺口（estimate\_condition K 非数组 → 裸 AttributeError，已修 + 补判别性测试）。

## 8.4 解析解验证（E1/E2/E3）

三个案例补齐”数值解没有解析解对标”的理论盲区——全部确定性（结构化网格 + elimination 直接求解，无随机数），CI 无 gmsht 的 test-core job 同样可跑。

### 8.4.1 E1 悬臂梁（Euler-Bernoulli 梁理论）

$$w_{max} = \frac{PL^3}{3EI}, \quad \sigma_x(x, y) = \frac{P(L-x)y}{I}$$

模型：L=5 m、H=0.5 m（细长，剪切贡献 ~0.4%）、E=210 GPa、 $\nu=0.3$ 、端部中点集中力 P=1000 N。加密序列 nx=16/32/64/128。

端部挠度相对误差（%）：

| nx  | 16    | 32    | 64    | 128          |
|-----|-------|-------|-------|--------------|
| CST | 32.49 | 10.50 | 2.46  | <b>0.17</b>  |
| Q4  | 12.93 | 3.20  | 0.377 | <b>0.377</b> |

根部应力相对误差 (%) (根列上半单元平均, 排除角点邻接单元——固支角点应力奇异性随加密发散, 属已知物理解):

| nx  | 16    | 32    | 64   | 128         |
|-----|-------|-------|------|-------------|
| CST | 34.99 | 12.63 | 4.94 | <b>3.08</b> |
| Q4  | 13.91 | 5.19  | 3.10 | <b>2.89</b> |

门槛 5%: 最细层 0.17%/0.377% (挠度) 与 3.08%/2.89% (应力) 全过, 且严格单调下降 (测试断言)。

#### 8.4.2 E2 Kirsch 圆孔 (应力集中)

$$\sigma_{\theta\theta} = \frac{\sigma_{\infty}}{2} \left( 1 + \frac{a^2}{r^2} - \left( 1 + \frac{3a^4}{r^4} \right) \cos 2\theta \right), \quad K_t = 3$$

有限板大域近似 ( $w=10a$ ), 1/4 板 + 对称边界, 径向 CST 网格加密 ( $n_{\text{ang}}, n_{\text{rad}}$ ) = (32,16) / (64,32) / (128,64):

| ( $n_{\text{ang}}, n_{\text{rad}}$ ) | (32,16) | (64,32) | (128,64) |
|--------------------------------------|---------|---------|----------|
| Kt                                   | 2.739   | 3.048   | 3.132    |
| $ K_t - 3 /3$                        | 8.7%    | 1.6%    | 4.4%     |

门槛 10% 通过; 网格方向性极差  $3.6\% < 10\%$ ;  $w=20a$  时  $K_t=2.977$  (0.8%) —— 确认 +4.4% 是有限宽效应而非离散化伪差 (Howland 型修正的正方向)。

#### 8.4.3 E3 Z2 效应指数 $\theta$

Timoshenko 抛物线剪流悬臂梁 (光滑解, 无点载荷奇异性), 5 层双向  $2\times$  加密, SPR 恢复的  $\eta$  序列, 跳过最粗非渐近层做  $\ln \eta \sim \ln h$  线性回归:

| 层 (nx)          | 4     | 8     | 16    | 32    | 64    | $\theta$     |
|-----------------|-------|-------|-------|-------|-------|--------------|
| $\eta_{CST} \%$ | 79.88 | 65.73 | 40.86 | 20.95 | 10.31 | <b>0.898</b> |
| $\eta_{Q4} \%$  | 64.87 | 38.68 | 20.14 | 10.26 | 5.23  | <b>0.963</b> |

两种单元实测  $\theta$  均落在理论区间  $[0.8, 1.2]$  (线性单元  $O(h)$  能量误差速率) —— Z2 估计器的能量误差速率得到解析级验证。CST 略低 (常应力单元 SPR 恢复收敛更慢) 属预期。

### 8.5 性能基线

数据由 `scripts/perf_benchmark.py` 生成 (规则方形网格, 4 角全约束 + 右下角  $x$  向集中力), 单机双次取中位数。

CPS4 (Q4):



| 档位   | 单元      | 节点      | 求解器    | 连接/几何  | K 组装   | 求解      | 误差估计   |
|------|---------|---------|--------|--------|--------|---------|--------|
| 1k   | 1,024   | 1,089   | direct | 3.5 ms | 5.4 ms | 24.8 ms | 5.3 ms |
| 10k  | 10,000  | 10,201  | direct | 29.1   | 47.6   | 242     | 43.8   |
| 100k | 100,489 | 101,124 | cg     | 332    | 477    | 919     | 513    |
| 300k | 300,304 | 301,401 | cg     | 1,007  | 1,541  | 3,353   | 1,548  |

### CPS3 (CST):

| 档位   | 单元      | 节点      | 求解器    | 连接/几何  | K 组装   | 求解     | 误差估计   |
|------|---------|---------|--------|--------|--------|--------|--------|
| 1k   | 1,058   | 576     | direct | 1.1 ms | 1.9 ms | 7.4 ms | 2.7 ms |
| 10k  | 10,082  | 5,184   | direct | 8.0    | 11.7   | 75.6   | 14.5   |
| 100k | 100,352 | 50,625  | cg     | 94     | 116    | 295    | 158    |
| 300k | 301,088 | 151,321 | cg     | 293    | 359    | 1,238  | 419    |

**读法：**组装/连接/后处理全部随规模近线性（向量化路径）；求解（direct）超线性（稀疏分解 fill-in）；求解（CG）依赖网格条件数——规则网格收敛快，条件差网格慢 1–2 个数量级。

**瓶颈本质：**CG 迭代数由刚度矩阵谱性质决定（条件数  $\propto$  网格质量），不是 Python 层可微优化对象——29 万单元真实模型 4161 迭代 54 s，同规模规则网格仅 1.5–4 s。ILU 预条件在 58 万 DOF 下 fill-in 不可行且不保证 SPD（与 CG 前提冲突）。cg\_rtol 已试验  $1e-8$ （9% 迭代收益）但污染固定 DOF 反力（ $\sim 4.3e-3$  N），被平衡检查误杀——已回退  $1e-10$  并注释原因：**收敛判据是正确性的一部分，不是性能旋钮。**

**CI 性能门**（test-perf job）：阈值 =  $\max(\text{基线} \times 4.0, \text{基线} + 200\text{ms})$ ，实测超阈值才红（边界值绿防 flaky）；基线为脚本内 CI\_BASELINE 常量，`--update-baseline` 刷新；失败也上传 JSON artifact 诊断。`--ci` 禁 `--mem`（tracemalloc 与计时语义冲突）。

#### 8.5.1 求解器选型建议

| 模型规模                 | 建议                                                                      |
|----------------------|-------------------------------------------------------------------------|
| $\leq 10$ 万 DOF      | 默认 direct（消去法）——快且精确，反力干净                                               |
| 10 万 $\sim$ 50 万 DOF | auto（自动选 Jacobi-PCG）——内存友好                                              |
| $> 50$ 万 DOF         | 必须 cg（ <code>--linear-solver cg</code> ）；若迭代数爆炸，优先检查网格质量（长宽比/扭歪），或粗网格先行 |
| 教学/粗网格               | 1k–10k 单元一切瞬发，无需关心性能                                                    |

迭代数预警信号：求解报告中的迭代数（如 4000+）与同规模规则网格（几十次）对比，差距就是网格条件差的代价。**粗网格 + 高质量单元（CST  $\rightarrow$  Q4I）通常比细网格 + CG 迭代爆炸更省时。**

| 项                    | 位置                                   | 改前             | 改后             | 验证                                              |
|----------------------|--------------------------------------|----------------|----------------|-------------------------------------------------|
| L2 fallback 批量<br>堆叠 | stress.py<br>nodal_L2_<br>projection | 300k<br>12.8 s | CST 4.2 s (3×) | 7 场景 bitwise<br>(CST/Q4R × 3 输入形态 + 非均匀<br>nqp) |

### 8.5.2 微优化记录 (2026-08-04, 包 4)

没改 (已 profile, 无安全空间): Q4/Q4I/Q4R 批量组装热点在 element 内核 einsum (文件边界禁止触碰); assemble\_sparse\_vectorized 的 rows/cols 每次调用重建 (~3% 收益, 需缓存失效管理, 风险/收益不划算); error\_est 已按 5 万单元分块向量化; SPR 占 estimate 时间 ~78% 但位于冻结区外文件边界; L2 剩余时间为质量阵 splu 分解 (算法级)。

## 8.6 CI 流水线

GitHub Actions 8 个 job: lint (ruff/mypy/vulture/self-test)、测试矩阵 (Python 3.9-3.13 × Windows/Linux)、全量验证 (覆盖率 fail\_under=90)、wheel 构建、test-perf 性能回归门。无 gmsh 环境模拟由 test-core job 承担 (1600+ 测试仍通过)。漂移门 6 值在 CI 与本地逐位一致。

## 8.7 发布检查清单

任何改动合入 main 前后, 按以下清单逐项执行 (指挥官 workflow):

1. **全量测试:** `pytest -v 0` 失败 (判据: 进度 100% + 0 FAILED/ERROR + exit 0; 注意 pytest 9.1.1 的 `-q` 会省略最终汇总行, 用 `-v` 判读)
2. **探针:** `python scripts/audit_contract_probe.py 0 FAIL`
3. **fuzz:** `python scripts/fuzz_api.py 500 0 problems`
4. **漂移门:** `python scripts/regression_compare.py 6` 值相对误差  $0.000e+00 (< 10^{-9})$
5. **lint 五连:** `ruff check / ruff format --check / mypy / vulture / self-test (python -W error::SyntaxWarning)`
6. **无 gmsh 模拟:** 干净 worktree (无 tools/gmsh.exe) 下 `verify_all.py --no-gmsh` 全量, skip 不失败
7. **CI:** push 后等待 8 job 全绿 (含 test-perf 性能门); 判断重跑是否生效看日志时间戳
8. **数字同步:** README / CODE\_MAP / 本类文档中的版本号、测试函数数、覆盖率数字一次刷新到位 (文档守卫测试会红)
9. **打包:** `make_release_zip` 后 zipfile 检查无泄漏文件 (.fem2d-msh-\* 等临时文件不得入包)

历史教训: 验收必须独立复跑, 从不采信执行者汇报; 跨平台逐位要求下底层库多列调用一律逐列 (SuperLU 教训)。

## 8.8 独立审查历史

外部逐行审查 3 轮、七维度 (R1-R7)，P0/P1 缺陷归零。审查发现过：CST 磨平峰值下降 17% (接受为物理预期)、角弧识别悬崖 (粗网格下弧被当线，14→8 段跳变，`topology.py` 20° 门 + `circle.py` ≥4 点门，已文档化并接受)、1 ULP 级累加顺序差异 (P- $\eta$  向量化后.at 顺序，不在钉死漂移值中)。审查报告归档于 `docs/review_20260805.md`。

## 9 测试代码深度解析

测试代码放在最后讲，因为它是“代码为什么这样安排”的终极证据。本项目测试约 3.1 万行 / 1649+ 个测试函数 / 143 个文件，而源码约 1.9 万行——测试比源码多 60%。为什么？一句话：每一个数值行为都有测试锁定，每一处修复都附判别性测试。本求解器经历过数轮大规模重构 (等价拆分、并行合并)，数值行为零漂移——这不是运气，是本章的机制锁出来的。

### 9.1 测试的组织：143 个文件怎么排

测试按分组前缀归档在 `tests/`，文件名即分组：

| 分组前缀                                                         | 内容          | 锁定什么                                    |
|--------------------------------------------------------------|-------------|-----------------------------------------|
| <code>test_solve_refactor_lock</code>                        | 求解金标准逐位锁    | <code>solver.solve</code> 输出逐位不变 (§9.2) |
| <code>contract</code>                                        | 契约测试 (6 文件) | 契约表每个 ✓ 有判别性测试在场                        |
| <code>boundary</code>                                        | 边界识别        | 段 schema、识别结果、插件体系                      |
| <code>regression_audit_2026080{2,3a-e}</code>                | 历次审计回归      | 已修缺陷不复发                                 |
| <code>usability</code>                                       | 易用性         | 错误消息、误用路径                               |
| <code>analytic</code>                                        | 解析解验证       | E1/E2/E3 (§9.8)                         |
| <code>structure</code>                                       | 结构/导入面      | 包结构、纯 facade 纪律                         |
| <code>test_{solver,runner, stress,error_est}_branches</code> | 分支锁         | 关键分支行为 (§9.7)                           |

命名约定：测试函数 `test_xxx`；模块级 helper 函数 `_xxx` (下划线前缀，不参与收集)；`_run` / `_plate` 类工厂统一构造模型。判别性测试单独成函数 (`test_xxx_discriminative` 或独立 `test`)，失败时错误信息直接点名”放回旧实现必须失败”的意图。

### 9.2 金标准逐位锁：`test_solve_refactor_lock.py` 逐行

这是测试体系里最硬核的一把锁。背景：2026-08 大重构把 `solver.solve` 等价拆分为多个阶段。教学求解器最怕的坑是“重构后结果悄悄变了、程序不报错”。金标准的

做法：重构前抓取代表性模型的**完整输出**（stdout 日志序列 + 完整 result dict）存入测试文件，重构只允许等价拆分，不允许任何数值/日志/键集合变化——**逐值精确比较**（非 allclose）。

### 9.2.1 文件头 docstring 就是设计文档

```
"""solver.solve 重构行为锁定测试（包 5 任务 2）。

GOLDEN 是重构前（2026-08-03 基线）对代表性模型（单元族 x 求解方法 x
微尺度 x 大坐标 x 纯 Dirichlet）抓取的精确输出：stdout 日志序列 + 完
整
result dict。重构只允许等价拆分，不允许任何数值/日志/键集合变化 --
逐值精确比较（非 allclose）。生成脚本：仓库外 gen_solve_golden.py。

环境敏感例外（2026-08-04 CI 确认）：跨平台（Linux OpenBLAS vs Windows
求和序）使所有数值派生量漂移  $\pm 1$  ulp -- Residual/ $\Sigma F$ / $\Sigma M$  行（backward-
error
与平衡残差，观测  $4.15e-17$  vs  $2.07e-17$  恰  $2\times$ ）、result 的 u/stress/
reactions 等数值键（观测最后一位  $\pm 1$ ）。这些行/键改为相对容差比较
（诊断行  $rtol=2.0 + atol=1e-12$  覆盖近零分量；数值键  $rtol=1e-10$  覆盖
 $\pm 1$  ulp 且留  $1e4$  倍余量，真回归  $1e-8+$  必被抓住）；stdout 其余行逐字符，
result 键集合/类型/bool/str 逐值。金标准是 Windows 录制 -- Linux 上
逐位相等本就不可能，锁的意图是“同平台 solver 重构等价”。
"""
```

注意 docstring 的写法：它不是装饰性注释，而是取舍的判决记录——为什么锁“逐值”？为什么允许  $\pm 1$  ulp？容差怎么选的？每句话都能追溯到一次真实观测（CI 平台差异  $4.15e-17$  vs  $2.07e-17$ ）。读到这个文件的维护者不需要考古就知道：“环境敏感例外”是刻意设计的，不是偷懒放宽。

### 9.2.2 容差常量的语义

```
# 环境敏感诊断行 -- 结构必须逐字符匹配（regex 锚定整行），数值容差比较
.
_RESID_RE = re.compile(
    r"\[OK\] Residual = ([0-9.eE+-]+) \ (backward error\)\n")
_SIGF_RE = re.compile(
    r"\[Solver\]  $\Sigma F$  = ((([0-9.eE+-]+), ([0-9.eE+-]+)\) N "
    r"\ (rel: ([0-9.eE+-]+)\)\n")
_SIGM_RE = re.compile(
    r"\[Solver\]  $\Sigma M$  = ([0-9.eE+-]+) N*m \ (rel: ([0-9.eE+-]+)\)\n")
# (pattern, rel 分量索引); None = 无 rel 分量
_DIAG_RE = ((_RESID_RE, None), (_SIGF_RE, 2), (_SIGM_RE, 1))
_BALANCE_KEYS = ("residual", "force_balance", "moment_balance")
# rtol=2.0: 观测到  $2\times$  舍入差异 ( $4.15e-17$  vs  $2.07e-17$ ); atol=1e-12:
```

```
# 覆盖近零分量（如 0.0 vs -5.68e-14）-- 微尺度模型（1e-166）亦远小于此
_DIAG_RTOL, _DIAG_ATOL = 2.0, 1e-12
# rel 分量：近零噪声的比率，观测放大 4.2× → rtol=1e2 留 24× 余量
_REL_RTOL = 1e2
# 数值派生键（u/stress/reactions/...）：跨平台 ±1 ulp 噪声（观测 1 ulp）；
# 近零分量（理论为 0 的 tau_xy 等）是积分求和序噪声（观测 2.27e-12 vs
# 4.09e-12，应力尺度 1e5 的 ~1e-17 相对）-- rtol 对近零值失效，补
# 键内尺度相对 atol = max|键值| × 1e-12（留 1e4 倍余量，微尺度 1e-150
# 模型同样安全 -- 尺度随键自动缩放，非绝对阈值）
_NUM_RTOL = 1e-10
_NUM_ATOL_FACTOR = 1e-12
```

容差参数表（每个数字背后都有一行注释记录观测值）：

| 参数                            | 值     | 理由（注释原文）                                          |
|-------------------------------|-------|---------------------------------------------------|
| <code>_DIAG_RTOL</code>       | 2.0   | 观测到 2× 舍入差异（4.15e-17 vs 2.07e-17）                 |
| <code>_DIAG_ATOL</code>       | 1e-12 | 覆盖近零分量（0.0 vs -5.68e-14）；微尺度模型亦远小于此               |
| <code>_REL_RTOL</code>        | 1e2   | rel 分量是近零噪声的比率，观测放大 4.2×，留 24× 余量                 |
| <code>_NUM_RTOL</code>        | 1e-10 | 覆盖 ±1 ulp 且留 1e4 倍余量，真回归 1e-8+ 必被抓住               |
| <code>_NUM_ATOL_FACTOR</code> | 1e-12 | 键内尺度相对 <code>atol = max 键值  × 1e-12</code> ，非绝对阈值 |

两个关键设计：其一，`atol` 是键内尺度相对的（`atol = scale * _NUM_ATOL_FACTOR`），不是绝对常数——微尺度 1e-150 模型的数值键尺度自动收缩，容差随之收缩，不存在“绝对阈值放行微尺度漂移”的缺口；其二，`_REL_RTOL` 单独放宽 rel 分量——它是近零噪声的比率，跨平台放大可达 4.2×，但分量本身仍由 `atol=1e-12` 主锁，平衡真破坏（分量 1e-10+）照样拒绝。

### 9.2.3 环境敏感行的剥离：结构锁死、数值容差

`stdout` 的其余行逐字符比较；被识别为环境敏感的行先剥离再比较。剥离用正则锚定整行——行格式/单位/前缀变了 = 结构漂移，红。

```
def _compare_stdout(name, out, golden_out):
    """逐字符比较，环境敏感诊断行剥离后容差比较."""
    for pattern, rel_idx in _DIAG_RE:
        om, gm = pattern.search(out), pattern.search(golden_out)
        assert om is not None or gm is None, (
            f"[{name}] 金标准无 {pattern.pattern[:24]}... 行但当前输出有")
```

```

    assert gm is not None or om is None, (
        f"[{name}] 金标准有 {pattern.pattern[:24]}... 行但当前输出缺失")
    if om is None:
        continue
    for i, (a, g) in enumerate(zip(om.groups(), gm.groups())):
        if i == rel_idx:
            assert np.isclose(float(a), float(g), rtol=_REL_RTOL,
                                atol=0.0), (
                f"[{name}] {pattern.pattern[:16]}: rel 数值漂移 "
                f"{a!r} vs 金标准 {g!r}")
        else:
            assert np.isclose(float(a), float(g), rtol=_DIAG_RTOL,
                                atol=_DIAG_ATOL), (
                f"[{name}] {pattern.pattern[:16]}: 数值漂移 "
                f"{a!r} vs 金标准 {g!r}")
    out = out[:om.start()] + out[om.end():]
    golden_out = golden_out[:gm.start()] + golden_out[gm.end():]
    assert out == golden_out, (
        "[" + name + "]" stdout 与金标准不一致 (诊断行已剥离), 前 400
        字符: "
        + repr(out[:400]))

```

细心的读者会问：为什么“该行存在性”也要断言（两个 `assert om is not None` 或 `gm is None`）？因为“诊断行缺失”和“诊断行漂移”是两种不同的回归——前者意味着代码路径变了（比如平衡检查被跳过），后者只是舍入噪声。逐字符比较剥离后剩下的部分，任何日志文案修改（哪怕加一个空格）都会红。这是契约的文本层：日志格式本身就是 API。

#### 9.2.4 递归数值比较：键内尺度 `atol`

`result dict` 的数值键用递归比较器：结构（键集合/长度/类型）逐值，数值按 `rtol` + 键内尺度 `atol`。

```

def _key_scale(value):
    """键内最大绝对值 -- 相对 atol 的尺度基准（递归，键内共用）。

    scale=0（全零键）→ atol=0：任何非零漂移都被拒，无放行缺口。
    """
    if isinstance(value, dict):
        return max((_key_scale(v) for v in value.values()), default=0.0)
    if isinstance(value, (list, tuple)):
        return max((_key_scale(v) for v in value), default=0.0)
    if isinstance(value, (float, int, np.floating, np.integer)):

```

```

        return float(abs(value))
    return 0.0

def _compare_numeric(name, key, av, gv, scale=None):
    """递归数值容差比较: list/tuple/dict/数值标量按 rtol + 键内尺度
        atol,
        其他逐值. bool 是 int 子类但语义逐值 -- 0/1 的 isclose 与逐值等价
        .
    """
    if scale is None:
        scale = _key_scale(gv)
    if isinstance(gv, dict):
        assert isinstance(av, dict) and sorted(av) == sorted(gv), (
            f"[{name}] result[{key}] 结构漂移: {av!r} vs {gv!r}")
        for sub in gv:
            _compare_numeric(name, f"{key}.{sub}", av[sub], gv[sub],
                             scale)
        return
    if isinstance(gv, (list, tuple)):
        assert isinstance(av, (list, tuple)) and len(av) == len(gv), (
            f"[{name}] result[{key}] 结构漂移: {av!r} vs {gv!r}")
        for i, (a, g) in enumerate(zip(av, gv)):
            _compare_numeric(name, f"{key}[{i}]", a, g, scale)
        return
    if isinstance(gv, (float, int, np.floating, np.integer)):
        assert isinstance(av, (float, int, np.floating, np.integer)), (
            f"[{name}] result[{key}] 类型漂移: {type(av).__name__} "
            f"vs {type(gv).__name__}")
        assert np.isclose(float(av), float(gv), rtol=_NUM_RTOL,
                           atol=scale * _NUM_ATOL_FACTOR), (
            f"[{name}] result[{key}] 数值漂移: {av!r} vs {gv!r}")
        return
    assert av == gv, f"[{name}] result[{key}] 与金标准不一致: {av!r}
        vs {gv!r}"

def _compare_result(name, result, golden_result):
    assert sorted(result) == sorted(golden_result), (
        "[" + name + "]" result 与金标准不一致, 键差: "
        + repr(sorted(result) ^ sorted(golden_result)))
    for key, gv in golden_result.items():
        av = result[key]

```

```

if key in _BALANCE_KEYS:
    # 平衡残差：舍入噪声级且可近零 (0.0 vs -5.68e-14) --
    # 相对 rtol=2.0 + atol=1e-12 (微尺度 1e-166 亦远小于此)
    ga, aa = np.asarray(gv, dtype=float), np.asarray(av,
        dtype=float)
    assert ga.shape == aa.shape, (
        f"[{name}] result[{key}] 形状漂移: {aa.shape} vs {ga.
            shape}")
    assert np.all(np.isclose(aa, ga, rtol=_DIAG_RTOL,
        atol=_DIAG_ATOL)), (
        f"[{name}] result[{key}] 数值漂移: {aa!r} vs {ga!r}")
else:
    _compare_numeric(name, key, av, gv)

```

这个比较器锁了四层东西：键集合（多一个键、少一个键都红）、结构（嵌套 dict/list 的形态漂移）、类型（float 变 int 也红）、数值（rtol + 键内尺度 atol）。\_key\_scale 的注释点出一个精妙细节：全零键 scale=0 → atol=0 → isclose 退化为精确相等（np.isclose 对 0 vs 1e-30 用 atol=0 + rtol 会拒绝）——“不存在”“全零键被容差放行”的缺口。

### 9.2.5 九个锁定场景

金标准锁的是单元族 × 求解方法 × 极端尺度的代表性组合：

```

def _plate(elem_type, scale=1.0, origin=(0.0, 0.0), forces=-500.0):
    x0, y0 = origin
    if elem_type == "CPS3":
        nodes = np.array([[0., 0.], [1., 0.], [0., 1.], [1., 1.]])
        elements = np.array([[0, 1, 2], [1, 3, 2]], dtype=np.int64)
    else:
        nodes = np.array([[0., 0.], [1., 0.], [1., 1.], [0., 1.]])
        elements = np.array([[0, 1, 2, 3]], dtype=np.int64)
    nodes = nodes * scale + np.array([x0, y0])
    mesh = Mesh(nodes=nodes, elements=elements, E=2.1e11, nu=0.3,
        thickness=0.01, plane_type="stress", elem_type=
            elem_type)
    for node in (0, 1):
        mesh.fix_node(node, "both", 0.0)
    for node in (2, 3):
        mesh.add_force(node, 0.0, forces)
    return mesh

def _dirichlet_only():
    mesh = Mesh(
        nodes=np.array([[0., 0.], [1., 0.], [1., 1.], [0., 1.]])
        elements=np.array([[0, 1, 2, 3]], dtype=np.int64),

```



```

        E=2.1e11, nu=0.3, thickness=0.01, plane_type="stress",
        elem_type="CPS4")
    for node in range(4):
        mesh.fix_node(node, "both", 1e-3)
    return mesh

MODELS = [
    ("cst", _plate("CPS3"), {}),
    ("cst_penalty", _plate("CPS3"), {"method": "penalty"}),
    ("q4", _plate("CPS4"), {}),
    ("q4_cg", _plate("CPS4"), {"linear_solver": "cg"}),
    ("q4_cond", _plate("CPS4"), {"check_condition": True}),
    ("q4r", _plate("CPS4R"), {}),
    ("dirichlet_only", _dirichlet_only(), {}),
    ("micro", _plate("CPS3", scale=1e-150, forces=-5e-150), {}),
    ("large_coord", _plate("CPS4", origin=(1e12, 1e12)), {}),
]

```

| 场景             | 覆盖的代码路径                        |
|----------------|--------------------------------|
| cst            | CST 单元 + 消去法（默认路径）             |
| cst_penalty    | 乘大数法分支（加法式罚因子，§6.8）            |
| q4             | Q4 单元 + 消去法                    |
| q4_cg          | Jacobi-PCG 迭代分支（rtol=1e-10）    |
| q4_cond        | 条件数检查分支（condition_info 键）      |
| q4r            | Q4R 沙漏稳定 + hourglass_energy 键族 |
| dirichlet_only | 全约束特判分支（空矩阵，不求解直接算反力）          |
| micro          | 微尺度 1e-150——禁绝对阈值硬指标（§10.1）    |
| large_coord    | 大坐标 1e12 平移——局部坐标消差（§6.3）      |

**注意 micro 和 large\_coord:** 这是两个“极端尺度”场景，专门守护 §10.1 的微尺度纪律——任何绝对阈值混进求解链路，micro 场景的数值键立刻漂移。

### 9.2.6 主测试与金标准结构

```

def _run(mesh, kwargs):
    buf = io.StringIO()
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        with redirect_stdout(buf):
            result = solve(mesh, **kwargs)
    return buf.getvalue(), _np(result)

```

```
GOLDEN = {'cst': {'stdout': '[Solver] 约束: 4 DOFs, 4 free DOFs\n'
                    '[Solver] 组装总刚 K (8×8) ...\n'
                    '[Solver] 组装等效载荷 F ...\n'
                    '[Solver] 消去法 + SuperLU ...\n'
                    '[OK] Residual = 4.15e-17 (backward error)\n'
                    '[Solver] max|u| = 4.855114e-07\n'
                    '[Solver] ΣF = (-5.684e-14, -2.274e-13) N',
            'result': {'u': [0.0, 0.0, 0.0, 0.0,
                             -3.462050599201064e-08,
                             -4.2121615623612943e-07, ...],
                       'stress': [[-29161.118508655112, ...], ...],
                       'vm_stress': [...],
                       'reactions': [159.78695073235684, ...],
                       'residual': np.float64(4.147910649287126e-17),
                       'balance_ok': True, ...}}, ...}

def test_solve_behavior_identical_to_golden():
    for name, mesh, kwargs in MODELS:
        out, result = _run(mesh, kwargs)
        _compare_stdout(name, out, GOLDEN[name]["stdout"])
        _compare_result(name, result, GOLDEN[name]["result"])
```

主测试只有 4 行：遍历 9 场景，分别比较 stdout 与 result。金标准是 Windows 录制（docstring 明说：跨平台逐位相等本就不可能，锁的意图是“同平台 solver 重构等价”）——这把锁的矛头是**重构**，不是跨平台移植。

### 9.2.7 判别性测试：锁自身的有效性证明

如果容差参数放宽到“什么都过”，金标准就成摆设。因此同文件内附判别性测试——用已知的“应接受噪声”和“应拒绝漂移”直接调比较器，证明容差窗既不吃真回归也不卡平台噪声：

```
def test_residual_tolerance_accepts_2x_platform_noise():
    """容差逻辑判别性（本地即生效）：CI 观测的 2× 舍入差异必须接受，
    真漂移（1e-10）必须拒绝。若改回逐字符比较，2× 用例在此失败。"""
    cst_out = GOLDEN["cst"]["stdout"]
    _compare_stdout("cst", cst_out.replace("4.15e-17", "2.07e-17"),
                    cst_out)  # Linux 2× → 接受
    for drift in ("1.00e-10", "5.00e-12"):
```

```

        with pytest.raises(AssertionError):
            _compare_stdout("cst", cst_out.replace("4.15e-17", drift)
                            ,
                            cst_out)                                # 真漂移 → 拒绝

def test_result_numeric_tolerance_1ulp_accepted_drift_rejected():
    """result 数值键容差判别性 (本地即生效): 1 ulp 平台噪声 (CI 观测
    量级) 必须接受, 1e-6 相对真漂移必须拒绝. 若改回逐值比较,
    1 ulp 用例在此失败 (Linux 语义下主测试本身也会失败)."""
    cst = GOLDEN["cst"]["result"]
    _compare_result("cst", cst, cst)                                # 恒等 →
    过
    noisy = dict(cst)
    noisy["u"] = list(cst["u"])
    noisy["u"][4] = cst["u"][4] + np.spacing(abs(cst["u"][4]))
    _compare_result("cst", noisy, cst)                                # 1 ulp →
    接受
    drifted = dict(cst)
    drifted["u"] = list(cst["u"])
    drifted["u"][4] = cst["u"][4] * 1.000001
    with pytest.raises(AssertionError):
        _compare_result("cst", drifted, cst)                        # 1e-6 漂
    移 → 拒绝
    broken = dict(cst)
    broken["u"] = list(cst["u"])
    broken["u"][0] = 1.0                                            # 约束
    DOF 非零 → 拒绝
    with pytest.raises(AssertionError):
        _compare_result("cst", broken, cst)

```

断言矩阵 (注释里每行都写明观测出处):

| 输入                                        | 预期 | 对应观测        |
|-------------------------------------------|----|-------------|
| Residual 4.15e-17 → 2.07e-17 (2×)         | 接受 | Linux 求和序差异 |
| Residual → 1e-10 / 5e-12                  | 拒绝 | 真回归量级       |
| u[4] + 1 ulp (np.spacing)                 | 接受 | 平台舍入噪声      |
| u[4] × 1.000001 (1e-6 相对)                 | 拒绝 | 真漂移         |
| 约束 DOF 置非零                                | 拒绝 | 结构级错误       |
| $\tau_{xy}$ 近零分量 2.27e-12 → 4.09e-12      | 接受 | 积分求和序噪声     |
| 同一分量 → 1e-6                               | 拒绝 | 超键内 atol    |
| $\Sigma F$ rel 1.26e-09 → 5.27e-09 (4.2×) | 接受 | q4r 平台差异    |
| rel → 1e-3                                | 拒绝 | 真回归         |

判别性测试的价值: 主测试在 Linux 上本来就该红 (金标准是 Windows 录的), 而

判别性测试**本地即生效**——任何人改了容差逻辑，本地 pytest 立刻抓出“该接受的被拒绝了 / 该拒绝的被放了”。

### 9.3 判别性测试原则：每修复必附”放回旧实现必失败”

这是贯穿整个测试体系的第一纪律：**任何缺陷修复，必须附一个”放回旧实现必然失败”的测试**。理由很简单——一个永远通过的测试没有价值：它能证明测试代码活着，不能证明被测试的行为是对的。

典型流程：最小复现（把缺陷场景固化成测试，先红）→ 修复 → 全量回归。判别性测试使修复**可证伪**：后来者如果”优化”掉了修复，测试立刻红，且错误消息直接说出被违反的契约。历史上 fuzz 抓到的真实缺口（estimate\_condition 收到非数组 K → 裸 AttributeError）就是按这个流程闭环的：最小复现测试 → 修复（带上下文的 TypeError）→ 判别性测试锁死新行为。

### 9.4 漂移门：regression\_compare.py

```
python scripts/regression_compare.py      # 基线 7ee65fc vs main
```

漂移门锁的是**端到端报告**的 6 项指标：max|u|、energy、eta、sxmax、symax、txymax。基线 7ee65fc 录制，**相对差必须 0.000e+00**——不是容差，是逐位。为什么这么苛刻？因为这 6 个数是用户从报告里直接读到的**最终输出**，任何一级（网格/组装/求解/应力/误差估计）的微小变化都会在他们身上放大。金标准锁 solver 内部 (§9.2)，漂移门锁用户可见结果——两层互补，一层都不该松。

### 9.5 探针：audit\_contract\_probe.py

```
python scripts/audit_contract_probe.py    # 151 项契约断言，必须 0
FAIL
```

探针是**文档驱动**的测试：docs/api\_contract.md 的契约表声明了每个接口的形状/类型/错误语义，探针逐条用真实代码验证”文档说的 = 代码做的”。契约表是”易读的真相”，探针是”可执行的真相”——文档与代码的漂移在提交前被抓，而不是在用户踩坑后。verify\_all 把探针并入门禁 (§7.2)，‘-stage’可单跑。

### 9.6 fuzz：固定种子 500

```
python scripts/fuzz_api.py 500            # 固定种子，可复现
python scripts/combo_fuzz.py              # 组合式随机
```

fuzz 用**固定种子**保证可复现——随机输入不冒裸异常（裸 AttributeError/IndexError 都是失败，带上下文的 ValueError 才是正确行为）。fuzz 的价值是**撞出人想不到的输入组合**：已经抓到过真实缺口（非数组 K 传给 estimate\_condition），每次抓到都按 §9.3 的流程闭环。组合 fuzz（combo\_fuzz）进一步随机组合参数，覆盖单参数 fuzz 的盲区。

## 9.7 分支锁：test\_{solver,runner,stress,error\_\_est}\_branches.py

分支锁锁关键函数的分支行为：每个 if/else 分支有专属测试，”这个分支存在”本身就是被测试的契约。典型形态：构造触发某分支的输入（如空网格、全约束、NaN 载荷），断言分支专属的报错消息或返回值——分支被删、条件被改都会红。四把分支锁分别守护求解器、runner 主线、应力后处理、误差估计四个核心模块。

## 9.8 解析解验证测试（E1/E2/E3）

解析解验证是唯一”对答案”的测试层（§7.4 详述）：数值解必须收敛到独立计算的理论值。

- **E1 悬臂梁**：梁端挠度 vs 材料力学解；同时作为剪切锁定演示——同一网格 CST → Q4 → Q4R 的弯曲刚度恢复（§4.3.2 引用的 12.9% → 3.2% → 0.38% 数据即来自此验证路径）；
- **E2 Kirsch 孔边应力集中**：孔边峰值应力系数 → 3（弹性理论极限值）；
- **E3 Z2 效应指数**：h-加密后  $\eta$  的对数斜率  $\theta \in [0.8, 1.2]$ ——误差估计器必须表现出教科书级的收敛阶（§4.6）。

与金标准/漂移门不同，解析解验证是物理语义层的锁：它证明的不是”代码没变”，而是”代码算对了”。四类锁的职责边界：金标准锁重构等价，漂移门锁最终输出，探针锁文档契约，解析解锁物理正确。

## 9.9 无 gmsh 纪律

本求解器在无 gmsh 环境下必须**全量测试绿**——教学场景用户可能只有便携包、没装 gmsh。纪律如下：

- fem2d 顶层**不急切 import gmsh**（经 gmsh\_adapter 惰性加载），测试同理；
- 依赖 gmsh 的测试用 skip 守卫跳过（pytest.skip），**skip 不失败**；
- skip 守卫**只捕 ImportError**——曾经有个守卫写成捕 OSError（假模块抛出的类型不同），导致无 gmsh 环境下 1 个测试失败。教训：守卫捕获的异常类型必须与真实抛出一致，”能捕到”不等于”捕对了”；
- **verify\_all --no-gmsh** 用假模块模拟无 gmsh 环境跑全量回归——干净 worktree（无 tools/gmsh.exe）下自证”去掉 gmsh 一切照常”。

## 9.10 一个测试文件的逐行走读：test\_boundary\_adaptive.py

原则讲得再多，不如把一个**真实文件**从头走到尾。选 tests/test\_boundary\_adaptive.py（185 行）：它测试的是边界识别管线（§6.7），体积小、几乎每一行都有教学点，是”边界识别算法如何被锁死”的标本。全程只 185 行、7 个测试函数、零 mock。

### 9.10.1 文件头：docstring 就是设计契约

```
"""Regression tests for adaptive boundary primitive detection."""
import numpy as np

from fem2d import Mesh, detect_boundaries
from fem2d.boundary.geometry import classify
from fem2d.boundary.topology import _point_in_loop
```

三行 import 就是被测面的边界声明：detect\_boundaries（总入口）、classify（曲率分类）、\_point\_in\_loop（内部几何原语）。注意它直接 import 了私有函数 \_point\_in\_loop——本项目的纪律是”测试可以穿墙看内部”，前提是该函数有稳定语义（本例：点是否在环内）。穿墙看内部让测试直接覆盖单函数语义，而不必绕整个管线去间接验证。

### 9.10.2 辅助构建器：参数化几何的工厂

```
def _fan_mesh(points):
    points = np.asarray(points, dtype=float)
    nodes = np.vstack([points.mean(axis=0), points])
    n = len(points)
    elements = np.array([
        [0, i + 1, ((i + 1) % n) + 1] for i in range(n)
    ])
    return Mesh(nodes=nodes, elements=elements, E=1.0, nu=0.3,
                thickness=1.0)
```

中心节点 0 + 环上各点，构成”扇形网格”（i + 1 与 ((i + 1) % n) + 1 让最后一个单元从末点绕回首点）。这个 6 行工厂被 5 个测试复用——测试辅助函数与产品代码用同一标准：只写一次、语义清楚、被复用才有资格存在。环上点由 \_regular\_loop 按角度生成，\_rounded\_rectangle 用直线段 + 圆弧拼接出”G1 连续”（切向连续）的圆角矩形——后者是专门构造的边界用例：真实模型最常见的形状是直线 + 圆弧混合，而 G1 连续恰好是”曲率断点检测”最容易翻车的点（切向连续处曲率突变，但视觉上无尖角）。

### 9.10.3 第一个测试：无 CAD 语义时多边形退化为直线段

```
def test_coarse_regular_polygons_remain_lines_without_cad_semantics():
    for sample_count in (8, 10, 12, 16):
        segments = detect_boundaries(
            _fan_mesh(_regular_loop(sample_count)))
        assert len(segments) == sample_count
        assert {segment["type"] for segment in segments} == {"line"}
        assert all(len(segment["nodes"]) == 2
```

```
and not segment["closed"]
for segment in segments)
```

参数化循环代替复制粘贴：8/10/12/16 四个多边形一次跑完，奇数边数（11 的邻居 10 与 12 覆盖了奇偶两侧）确保不是只对某个数巧合成立。断言的是集合而非顺序：`{segment["type"] for segment in segments} == {"line"}` 不依赖输出顺序——识别管线的段顺序由邻接图遍历决定，本不是契约的一部分，顺序敏感断言会制造脆测试。

#### 9.10.4 对照组：细分弦线必须合并、真六边形必须保留

```
def test_subdivided_regular_polygons_preserve_line_primitives():
    for primitive_count in (8, 10, 12, 16):
        polygonized = _subdivide_chords(
            _regular_loop(primitive_count), subdivisions=4)
        segments = detect_boundaries(_fan_mesh(polygonized))
        assert len(segments) == primitive_count
        assert all(len(segment["nodes"]) == 5 ...)
```

每个“弦线”被插成 5 个节点——但分类结果必须是 1 条 line，即 4 段共线短边被合并，不能每段独立上报（否则下游载荷链会看到 4 段共线边、生成 4 个面力载荷条目）。`len(segment["nodes"]) == 5` 验证合并结果保留了全部采样点。与之相对，`test_true_hexagon_remains_six_lines` 断言真六边形是 6 条 line——这是对对照组：“细分后不误报为圆弧”与“真多边形不被误报为圆弧”两个方向都锁住，防止分类器“全报 line”这种偷懒退化通过测试。

#### 9.10.5 闭环分类：半径反解到 $1e-12$

```
def test_closed_physical_curve_classification_refits_chord_vertices():
    :
    polygonized = _subdivide_chords(_regular_loop(24, radius=8.0), 9)
    closed = np.vstack([polygonized, polygonized[0]])

    segment_type, _, info = classify(closed, scale=135.0, is_outer=False)

    assert segment_type == "arc"
    assert info["primitive_samples"] == 24
    assert np.isclose(info["radius"], 8.0, rtol=1e-12)
```

三个断言层层加码：“arc”（分类正确）、`primitive_samples`（原始 24 条边被识别为 1 个圆弧原语）、半径反解误差  $< 10^{-12}$ （拟合质量）。注意 `scale=135.0` 是显式传入的——圆外接半径 8.0 的环，特征尺度约  $2\pi R \approx 50$ ，135 是放大余量。把尺度做成显式参数而不是内部魔法常量，测试就能挑剔地验证“尺度感知”：容差不随几何尺寸漂移（呼应 §8 微尺度纪律）。

### 9.10.6 圆角矩形：G1 边界用例

```
def test_g1_rounded_rectangle_preserves_eight_primitives():
    segments = detect_boundaries(_fan_mesh(_rounded_rectangle()))
    assert len(segments) == 8
    assert len(lines) == 4 and len(arcs) == 4
    assert all(np.isclose(segment["info"]["radius"], 0.5, rtol=1e-10)
               for segment in arcs)
```

圆角矩形 = 4 直线 + 4 圆弧，且所有圆弧半径必须反解出 0.5——数量断言 + 物理量断言组合：只断言数量不够（可能把圆弧错分成多段），只断言半径不够（可能丢段）——两个都锁，才是“识别正确”的完整定义。

### 9.10.7 尺度感知：从 1e-20 到 1e12 的环判定

```
def test_point_in_loop_is_scale_aware_and_deduplicates_vertices():
    for origin, span in ((0.0, 1e-20), (1e12, 100.0), (-1e12, 1e-3)):
        ... # 三个数量级的正方形
        assert _point_in_loop(origin + 0.5 * span, ...) # 内点
        assert not _point_in_loop(origin + 2.0 * span, ...) # 外点
        assert _point_in_loop(origin, origin, xs, ys) # 重复顶点去重
```

三个数量级跨度：(0, 1e-20)（纳米级模型）、(1e12, 100)（大坐标模型）、(-1e12, 1e-3)（负坐标小模型）。环判定必须是**相对尺度**的——几何量永远相对自身尺度比较，绝对量在三个跨度里没有共同合理值。**重复顶点去重**（首尾闭合产生的重复点）是另一条历史教训的锁：闭环分解时 loop + [loop[0]] 补边，若不去重，点在“重复顶点”上会返回歧义结果。

### 9.10.8 环定向：鞋带公式分内外

```
def test_detected_ring_has_gmsh_outer_and_hole_orientation():
    ... # 外 12 边形 + 内 12 边形孔的 CPS4 环网格
    segments = detect_boundaries(mesh)
    oriented_area[loop_id] += 0.5 * np.sum(
        coords[:-1, 0] * coords[1:, 1] - coords[1:, 0] * coords[:-1,
        1])
    assert oriented_area[outer_loop] > 0.0
    assert oriented_area[hole_loop] < 0.0
```

这个测试自己重算了识别管线的核心输出（鞋带公式的有向面积），再断言 info["is\_outer"] 与重算一致——而不是断言“is\_outer 是 True”。测试重算而非信任输出，是判别性测试（§9.3）的基层形态：若识别器把内外环标反，这个测试立即失败；若输出字段改名，编译期/运行期也会立刻暴露。对 §6.7 的定向约定（材料 CCW、洞 CW）来说，这就是最后一道防线。



1. 参数化循环（8/10/12/16、四个尺度）替代复制粘贴——覆盖面与维护成本同时最优化；
2. 断言集合/物理量而非顺序——只锁契约内的事实；
3. 对照组（真六边形 vs 细分多边形）——防止分类器退化偷懒；
4. 显式传尺度参数——容差纪律可被测试直接审查；
5. 数量断言 + 物理量断言组合——”识别正确”的完整定义；
6. 测试重算输出（鞋带公式）——判别性测试的基层形态。

七个测试函数、185 行，把 §6.7 的识别算法钉死在”分类正确、合并正确、定向正确、尺度稳健”四个维度上——这就是本项目对”冻结区”（边界识别算法冻结）的执行方式：不是靠纪律声明，是靠可证伪的测试。

## 9.11 测试如何保护重构（总结）

四类”锁”的职责边界一句话版：

| 机制             | 锁什么             | 证明什么     |
|----------------|-----------------|----------|
| 金标准逐位锁 (§9.2)  | solver 输出逐位/键集合 | 重构等价     |
| 漂移门            | 端到端 6 指标相对差 0.0 | 用户可见结果不变 |
| 探针             | 契约表 151 断言      | 文档 = 代码  |
| fuzz           | 随机输入不冒裸异常       | 误用路径有护栏  |
| 分支锁            | 关键分支存在          | 行为不塌方    |
| 解析解 (E1/E2/E3) | 数值 vs 理论        | 物理上算对了   |
| 判别性测试          | 每个修复可证伪         | 测试本身活着   |

正是这套机制，让”多轮大重构 + 并行合并”成为可能：重构等价性由金标准保证，物理正确性由解析解保证，文档一致性由探针保证——每一条都独立运行、独立失败、独立报告。这就是本项目可维护性的引擎：不是流程文档，是七层互相咬合的测试。

## 10 注意事项

### 10.1 常见错误消息速查

错误消息统一格式函数名：参数名 = 值 — 原因，期望。常见消息与修复：

| 错误消息（片段）                                   | 原因与修复                         |
|--------------------------------------------|-------------------------------|
| <code>solve: mesh 必须是 fem2d.Mesh 实例</code> | 传入了 dict/None 等非 Mesh 对象；检查类型 |
| <code>Rigid-body modes are not</code>      |                               |

|                                                         |                                                                                      |
|---------------------------------------------------------|--------------------------------------------------------------------------------------|
| <code>fully constrained</code>                          | 欠约束；补足固定 DOF（2 平动 + 1 转动/连通分量）                                                       |
| <code>u contains NaN/Inf</code>                         | 求解结果非有限；先检查网格/材料/载荷是否合理                                                              |
| K 必须为方阵                                                 | <code>estimate_condition</code> 收到非方阵；检查矩阵形状                                         |
| n_dof 必须等于 2× 节点数                                       | <code>assemble_loads</code> 的 n_dof 与网格不一致                                           |
| Penalty constraints currently require the direct solver | penalty 方法与 cg/ilu 求解器不兼容；换 elimination 或 direct                                     |
| singular matrix / MatrixRankWarning                     | 约束不足或含孤立节点（Gmsh 导入的孤立节点教训）；检查连通性                                                     |
| No elements found                                       | 传入了 .geo 脚本而非网格文件 (.inp/.msh)；检查文件类型                                                 |
| GmshTopologyError                                       | 混合网格拓扑（三角形 + 四边形）被拒绝；改用纯三角形或纯四边形                                                     |
| must be a finite real number                            | 传入 str/complex 等非数值；转为 float                                                         |
| No CPS3/C2D3 elements found                             | .inp 里单元类型不匹配或文件不是网格                                                                 |
| value out of range / nu 越界                              | 材料参数语义错误； $\nu \in (-1, 0.5)$                                                        |
| 表达式解析失败（带行号）                                            | .spec 格式错误；缺 = 或空值，按行号定位                                                             |
| OverflowError（表达式）                                      | 超大指数（如 $9^{**}9^{**}9^{**}9$ ）在编译期被拦截；属安全保护                                          |
| penalty 罚因子量纲异常                                         | 乘法式罚因子 ( $k_{ii}$ × 罚因子) 使刚度平方；应为 <b>加法式</b> : $k_{ii} += \max  K_{ii}  \times 10^4$ |
| quality 分类计数不为互斥                                        | bad/warn 条件区间重叠导致双重计数；bad/warn/ok 计数和必须恰等于单元总数                                       |
| traction jump 饱和 200%                                   | $ s_1 - s_2 /\max$ 在应力过零处虚高；改用 $ \Delta t /\sigma_{ref}$ 归一化                         |
| rigid-body rank 不足                                      | 约束矩阵用原始大坐标致 SVD 阈值失真；先逐分量中心化 + 归一化再查 rank                                            |
| u 全零/含 NaN 且无警告                                         | 近奇异矩阵 spsolve 未必抛 MatrixRankWarning；显式 splu 分解 + isfinite 检查兜底                       |

错误消息只是入口，推荐的定位顺序：

1. **最小化复现**: 从 `models/` 里最接近的演示模型改参数, 避免从零建模引入第二个变量;
2. **隔离单测**: `pytest tests/test_xxx.py -k 名称` 只跑相关测试, 确认问题在测试面内还是测试面外;
3. **全量回归**: `python scripts/verify_all.py --fast` (pytest 全量 + 探针 + 漂移门), 排除基线漂移;

4. **--quad 对比**: 同一 `cantilever.geo` 分别以三角形（默认）与四边形（`--quad` 触发 `recombination`）求解，位移/应力云图若剧烈不同，多半是单元族行为差异（如 Q4R 细长网格过刚，见 §4.3.3 已知局限）而非代码错误。

以下 9 条来自历史事故的直接教训，每条都有对应排查记录（知识库 `fem/lessons/`）。

1. **单位制**: 默认 SI 单位 ( $E=2.1e11$  Pa,  $t=0.01$  m)。修改参数时保持量纲一致，否则结果数值将无法解释。
2. **几何文件与网格文件不可混传**: `.geo` 是 Gmsh 脚本，不是网格数据；`.inp/.msh` 才是网格。把 `.geo` 当网格传入会得到 “No elements found” 类错误（历史事故：同名 `.geo/.inp` 混传，报错信息指向错误方向）。
3. **必须充分约束**: 欠约束模型被刚体模态检查拦截并明确报错（“Rigid-body modes are not fully constrained”），不会静默给出无意义解。固定端在 CST 中等于锁住整条边所有节点的平动自由度（CST 无转角自由度，固支 = 一排固定铰的等效）。
4. **载荷表达式是安全的**: 表达式经 AST 白名单解析（变量仅  $x, y$ ，函数仅 `sin, cos, tan, exp, sqrt, log, abs` 与常量  $\pi$ ），禁止 `eval`。超大指数（如 `9**9**9**9`）在编译期被整型转浮点 `NodeTransformer` 拦截（2s 内 `OverflowError`），不会挂起。
5. **输入数值须为实数**: 复数节点坐标、复数应力数组被显式拒绝（`TypeError`，带三路检查），不会静默丢弃虚部。
6. **gmsh 缺失时**: 网格生成（`.geo` 路径）不可用，其余功能（读 `.msh/.spec`、求解、后处理、误差估计）全部可用。无 `gmsh` 环境下 1600+ 测试仍通过（`skip` 不失败）。
7. **混合网格不支持**: 三角形 + 四边形混用拓扑被明确报错拒绝（`GmshTopologyError`），属安全设计：宁可拒绝，不静默算错。
8. **中文输出**: 非中文 Windows 控制台（GBK）下，内部已做编码兜底（`reconfigure_streams`），中文以 “?” 显示，不会崩溃。
9. **验证纪律**: 任何改动后运行 `python scripts/verify_all.py --fast`（`pytest` 全量 + 探针 + 漂移门）确认数值基线未被破坏；漂移门 6 值要求相对误差  $< 10^{-9}$ 。修复必须附判别性测试（放回旧实现必失败）。

## 11 可扩展性

软件按插件化设计：每个扩展点都有**固定的挂接位置**（注册表或分发函数），扩展代码写在独立文件里，**不改动已验证的数值路径**。本节的用法：确定“我要加什么” → 查 §11.1 总表定位改动文件 → 按 §11.2 对应扩展点的最小步骤走 → §11.3 的流程收尾。

### 11.1 扩展点总览：改哪些文件、挂在哪

| 扩展类型 | 要改的文件 | 挂接点（注册/分发） 门禁验证 |
|------|-------|-----------------|
|------|-------|-----------------|

|          |                                                |                                      |                                |
|----------|------------------------------------------------|--------------------------------------|--------------------------------|
| 新增单元类型   | element/ 新文件 + element/__init__.py 的 import 一行 | register_element 注册内核实例              | verify_all_elements: 分片 + 刚体模态 |
| 新增边界检测器  | boundary/detectors/ 新文件                        | register_detector 注册即生效, 首个非 None 胜出 | 判别性测试 + 全量回归                   |
| 新增载荷形式   | loads_core.py 的 parse_* 族                      | parse_traction 等入口; AST 白名单          | 解析单测 + 物理量回归                   |
| 新增应力恢复方法 | stress.py 新函数 + visualize.py                   | _RECOVERY_METHODS 字典 (21-29 行) 注册    | 与既有方法对比单测                      |
| 新增误差估计器  | error_est.py                                   | estimate() 分发处加分支 (不改旧公式)            | 解析解 E1/E2/E3 对比                |
| 新增可视化模式  | visualize.py                                   | PLOTS 注册表 (632 行) 加 tag + 绘图函数       | 无 gmsh 模拟下出 PNG                |
| 新增输入格式   | input/ 链                                       | 扩展名分派加分支 (§3.1)                      | 样例文件端到端                        |
| 新增验证阶段   | scripts/ verify_all.py                         | 九阶段链追加                               | 全绿                             |

表中“门禁验证”一列是**硬性要求**: 扩展没配上对应门禁 = 不算完成。数值类扩展 (单元/恢复/估计器) 额外受冻结区纪律约束 (§5.5): 旧行为必须逐位不变, 新行为靠新增注册项承载。

## 11.2 各扩展点的最小改动步骤

①新增单元类型 (5 步, 模板 = 现有 CST/Q4/Q4R/Q4I):

1. fem2d/element/q4.py 复制为新文件, 实现 ElementKernel 协议 (刚度、应变位移阵 B、积分规则、沙漏控制等基类方法覆写——element/\_\_init\_\_.py 的 docstring 有 3 步流程说明);
2. fem2d/element/\_\_init\_\_.py 末尾补 from .< 新模块> import < 新内核> + register\_element(< 新内核>());
3. 注册必须过 verify\_all\_elements() (分片试验 + 刚体模态自检) ——不过关说明内核公式有误;
4. 写判别性测试 (放回旧实现必失败的测试, §9.3);
5. python scripts/verify\_all.py --fast 全绿收尾。

②新增应力恢复方法 (3 步, 模板 = stress.py 的 nodal\_simple / nodal\_weighted / nodal\_L2\_projection, 150/158/212 行):

1. `stress.py` 写 `nodal_<名字>(mesh, result)` 函数，签名与返回 `(n,3)` 数组必须与现有方法完全一致；
2. `visualize.py` 顶部 `_RECOVERY_METHODS` 字典加一项 `"<名字>": nodal_<名字>`——CLI 的 `--recovery` 自动可选到它；
3. 对比测试：同一 `result` 上与已有方法数值可比（新方法对同一网格不得出现数量级差异）。

③新增载荷/约束形式（3步，模板 = `loads_core.py` 456/514/590 行）：新增 `parse_*` 入口函数复用 `make_edge_profile_func` 做空间分布，需要新数学函数/常量时只改 `_compile_expr` 的 AST 白名单（这是安全边界：表达式里出现白名单之外的名字直接拒绝，历史教训 §2 库“AST 白名单禁止 eval”）；配解析单测 + 端到端物理量回归。

④新增边界检测器/命名器（3步，模板 = `boundary/plugins.py` 的 `circle_label / ellipse_group_label / arc_curvature`）：

1. 在 `boundary/detectors/` 写新文件，`register_detector()` 登记——注册即生效，无需改识别管线；
2. 首个非 `None` 胜出：新检测器只要在既有检测器之后注册，不抢已有标签；
3. 全量回归（识别结果的金标准文件若变，走 `golden` 更新流程）。

⑤新增可视化模式（3步）：`visualize.py` 的 `PLOTS` 注册表加 `tag` + 绘图函数（模板 = 既有 12 个 `tag`）；交互/保存自动接入（`interactive_plot` 按 `tag` 分发）；无 `gmsh` 环境下 `--save` 必须仍能出 PNG（Agg 后端兜底）。

### 11.3 扩展开发流程

1. 只读调用现有 API（`Mesh/solve/error_est/convergence`），不复制实现——重复实现是静默分歧的温床（历史教训：两处 `parse_vec2` 行为漂移）
2. 新增功能必须带判别性测试 + 全量 `pytest` 0 失败
3. 触碰冻结区需特殊程序：金标准逐位一致 + 回归 0.0 双验证
4. 提交前跑 `verify_all.py --fast` 与 `--no-gmsh` 模拟
5. 文档同步：新扩展点必须在本文档（对应小节）与 `docs/CODE_MAP.md` 里登记——测试锁定文档声明的关键数字，文档不同步会在 CI 报红（§9.1 文档守卫）。

## 12 可维护性

可维护性不是口号，是按场景走固定路径。下表是六类常见日常动作的标准流程——每一步做什么、跑哪条命令、以什么为准。

| 场景 | 操作路径 | 验证基准 |
|----|------|------|
|----|------|------|

|       |                                                                         |                           |
|-------|-------------------------------------------------------------------------|---------------------------|
| 修 bug | 最小复现（拿 models/ 现成.geo 或写最小.geo 定位）→ 修复 → 附判别性测试（放回旧实现必失败）→ 全量 pytest    | 0 失败；漂移门回归 0.0            |
| 加功能   | 查 §11.1 总表定位挂接点 → 按 §11.2 最小步骤 → 判别性测试                                  | verify_all -fast 全绿       |
| 动冻结区  | 金 标 准 逐 位 一 致<br>(test_solve_refactor_lock) → 回 归<br>0.0 双验证——唯一合法路径    | 逐位一致 + 漂移门 0.0            |
| 优化性能  | 先 perf_benchmark.py 打基线 → 改非冻结区 → 重测（禁止凭直觉改数值路径）                        | 性能回归门 + 漂移门 0.0           |
| 升级依赖  | numpy/scipy/pytest 升版后跑全量 + 漂移门；SciPy 升版曾引起 SuperLU 逐位分歧（多列 solve 必须逐列） | 跨平台逐位一致（Windows/Linux CI） |
| 发布新版本 | CHANGELOG 条目 → 版本号 → 便携包重建 (§2.6 目录结构) → 发布检查清单 (§8.7)                  | 解压到干净目录跑通 demo            |

- **一键验证：**verify\_all.py 九阶段自检——pytest 全量、探针、fuzz（500 例）、漂移门、compileall、ruff、mypy、vulture、self-test；--no-gmsh 模式模拟无 gmsh 环境全量回归。
- **数值基线钉死：**漂移门（regression\_compare）6 个物理量逐位比对，跨平台（Windows/Linux CI）一致，任何数值改动逃不过它。
- **契约表：**docs/api\_contract.md 记载全部公开 API 的输入/输出/异常契约，配套契约测试（6 文件）锁定行为；探针 scripts/audit\_contract\_probe.py 151 项断言与代码实测三方一致。
- **文档守卫：**测试锁定 README/CODE\_MAP 的关键数字声明，代码变更后文档不同步会直接报红。
- **CI 全链路：**GitHub Actions 8 个 job（lint、测试矩阵、全量、wheel、性能回归门）。
- **多轮独立审查：**外部逐行审查 3 轮（R1-R7 七维度），P0/P1 缺陷归零，审查报告归档于 docs/review\_20260805.md。
- **冻结区纪律：**element/ 内核公式、solver 数值逻辑、error\_est 公式、边界识别算法——改动需特殊程序（金标准 → 逐位一致 → 回归 0.0），教学求解器最怕“错误结果无报错”，此坑已被机制锁死。
- **工作流铁律：**每提交必全量 pytest 0 失败；修复流程 = 最小复现 → 修复 → 判别性测试 → 全量；注释只留“为什么”，不写历史叙事；禁绝对阈值。

## 13 未来展望

1. **非线性扩展：**当前求解器为线弹性。非线性（几何大变形、弹塑性材料、接触）需要增量步 + 牛顿迭代框架，属新项目范畴；本求解器可作退化基准——非线性代码退化为线性时必须与本软件逐位一致。

2. **混合网格**: 三角形 + 四边形混合拓扑当前明确拒绝。若需支持, 可在拓扑校验层放开限制并统一单元间界面自由度处理, 属独立开发轮次 (研究备选, 未拍板前不做)。
3. **性能**: 已完成的优化轮 ( $P-\alpha/\beta/\delta/\epsilon/\zeta$ ) 覆盖  $K \cdot u$  乘法 (100k solve  $-10.8\%$ )、刚体模态缓存 (二次调用  $\sim 6500\times$  提速)、面力批量向量化 ( $2.6-3.3\times$ )。进一步方向: 稀疏求解器调优、多线程/GPU 批量装配。任何优化先 profile 再动手 (`perf_benchmark.py`), 禁止凭直觉改冻结区。
4. **三维扩展**: 公式体系 (弱形式、误差估计、SPR) 可直接推广至三维; 单元库、网格接口与可视化需按维度抽象重构 (研究备选, 架构边界重构成本高)。
5. **交互/可视化**: 现有交互式向导与云图输出可演进为独立 GUI 或 Web 前端 (Jupyter 集成), 降低使用门槛。

## 14 版本与验证记录

| 版本     | 提交      | 验证结论                                            |
|--------|---------|-------------------------------------------------|
| 9.28.0 | e397f64 | 全部轮次闭环; pytest 1812 全过; 漂移门 0.000e+00; 覆盖 98.3% |

历史大轮: A 清理轮 (死代码/重复实现清除)、B 结构轮 (长函数拆分)、C 覆盖轮 (契约扩展)、D 易用性轮 (错误消息引导)、E 解析解轮 ( $E1/E2/E3$ )、R 审查轮 ( $R1-R7$  七维度)、P 性能轮 ( $P-\alpha-\zeta$  七包)、 $S-\alpha$  安全轮 (表达式 DoS/复数拒绝/打包卫生)。

本文档对应的代码仓库即为主项目目录; 便携包 (`PORTABLE_WIN64.zip`) 为免安装演示/交付形态, 已在全新路径模拟新电脑验证通过 (求解结果与参考值一致, 全量自检 ALL PASS)。

## A 常见问题 (FAQ)

按“从建模到出结果”的使用顺序编排的问答, 每条答案都在正文有对应章节——这里只给结论和指针。

### A.1 建模与输入

**Q1: 为什么不再支持.inp 输入?** A: 手写.inp 解析器在真实文件面前暴露了一批边界情况缺陷 (TYPE= 子串匹配误判、T3D2 孤立节点、.geo 混传), 改走 Gmsh API 直读.msh 后整类错误消失 (§3.8)。.inp 保留为导出格式, 不再是输入格式。

**Q2: 没有 Gmsh 能跑吗?** A: 能。网格生成 (.geo 路径) 不可用, 其余全部可用: 读.msh/.spec、求解、后处理、误差估计。无 gmsh 环境下 1600+ 测试仍通过 (skip 不失败, §9.9 无 gmsh 纪律)。

**Q3: .geo / .msh / .txt / .spec 有什么区别?** A: .geo 是 Gmsh 脚本 (几何 + 网格); .msh 是 Gmsh 网格数据; .txt 是中文几何描述 (交互向导生成); .spec 是求解参数文件。run.py 按扩展名自动分派 (§3.1)。

## A.2 单元选择

**Q4: CST / Q4 / Q4R / Q4I 怎么选?** A: 教学与收敛研究用 Q4 (全积分、无锁定配置); 大型模型/扭曲网格用 Q4R (快但细长网格失真); 薄板/细长弯曲首选 Q4I; 规则简单网格用 CST 入门 (§4.3.3 决策表)。

**Q5: Q4R 的沙漏能占比高是不是说明结果不可信?** A: 不是。沙漏能占比在两种情形下都可能 > 90% (稳定项主导或沙漏主导), 它不是可靠判据; 判据改用单元长宽比在求解前告警 (§4.3.3)。

**Q6: 为什么  $2 \times 2$  高斯积分叫“全积分”?** A: 被积函数对平行四边形是  $\xi, \eta$  的二次多项式,  $2 \times 2$  高斯点恰好能精确积分 3 阶以内——刚刚够, 故称“全”。单点积分只够线性项, 二次项被采样到零——沙漏的根源 (§4.3.1)。

**Q7: 固定端在 CST 里为什么等于一排固定铰?** A: CST 每个节点只有  $U_x/U_y$  两个平动自由度, 没有转角可约束; 固支只能锁住整条边上所有节点的平动——位移梯度在边内为零, 转角间接为零 (§10.1 第 3 条)。

## A.3 载荷与边界

**Q8: 压力方向反了怎么办?** A: 法向由相邻单元 CCW 方向确定 (§6.7.4 定向约定), 与调用时节点顺序无关; 若要主动改向, 检查单元是否全部 CCW 归一化 (导入器自动做, §3.8 硬规则 4)。

**Q9: 表达式载荷里能写什么?** A: 变量仅  $x, y$ , 函数仅  $\sin/\cos/\tan/\exp/\sqrt{\phantom{x}}/\log/\abs$  与常量  $\pi$ 。AST 白名单解析, 禁止 eval; 超大指数在编译期被拦截 (§10.1 第 4 条)。

**Q10: 为什么网格看起来约束充分却报奇异矩阵?** A: 最常见原因是孤立节点——节点存在但未被任何 2D 单元引用, 它在总刚中对应全零行/列, 任何约束都救不回来。Gmsh 导入已自动过滤 (§3.8 硬规则 3), 裸构造 Mesh 时需自查节点连通性。

## A.4 结果解读

**Q11: 结果与解析解差 10% 是算错了吗?** A: 先看单元族与网格密度: Q4 悬臂  $12.9\% \rightarrow 3.2\% \rightarrow 0.38\%$  随加密收敛 (§4.3.1); Q4R 在细长网格下可过刚到 2% (§4.3.3 已知局限)。误差估计用 Z2 与应力跳跃定位区域 (§4.6), 不要凭单个数值下结论。

**Q12: 应力云图峰值和单元应力峰值为什么不同?** A: 磨平 (nodal weighted/SPR) 把相邻单元常应力加权平均, 峰值被周围低值拉低 ~17% 是正常现象; 色条范围若用节点值应明确标注 (历史教训, §4.5.2)。

**Q13: 相邻单元应力跳跃 200% 是什么?** A:  $|s_1 - s_2|/\max$  公式在应力过零处饱和到 200%——是归一化公式的假象, 不是真实误差; 正确度量是牵引跳跃  $|\Delta t|/\sigma_{ref}$  (§4.6.2)。



A.5  开发与验证

- Q14:** 改了一行代码怎么确认没破坏数值？ A: 运行 `python scripts/verify_all.py --fast` ——`pytest` 全量 + 探针 + 漂移门（端到端 6 指标相对误差  $< 10^{-9}$ ）；修复必须附判别性测试——放回旧实现必失败（§9.3）。
- Q15:** 怎么加一种新单元？ A: 实现 `ElementKernel` 协议（形函数/B 矩阵/刚度/自检四件套），`register_element()` 登记即可被求解器使用；现有四种单元是模板（§6.2 协议层 + §11 扩展开发流程）。
- Q16:** 便携包和源码方式怎么选？ A: 演示/交付用便携包（免安装，`demo.bat` 一键全流程，§2.1）；开发/研究用源码方式（`pytest` 全量回归 + 探针可用，§2.2）。
- Q17:** 为什么 Q4 网格在纯弯曲下挠度偏小？ A: 全积分 Q4 的剪切锁定——双线性位移场无法表示  $\sigma_x \sim y$  的弯曲应力分布，寄生剪切使弯曲刚度抬高（§4.3.1）。粗网格明显（悬臂 12.9%），加密后消失；换 Q4R/Q4I 也可消除。
- Q18:** 怎么判断网格够不够细？ A: 三条独立证据：Z2 能量误差  $\eta$  随加密下降且收敛阶  $\approx 1$ （§4.6.1）；E3 效应指数  $\theta$  落在预期区间（§9.8）；加密一倍（ $h$  减半）后关键量（挠度/应力集中系数）变化显著变小。三者互相印证，不要只看一张云图。
- Q19:** 云图里红色区域就是危险区吗？ A: 先看色条标注的是单元值还是节点磨平值——磨平后峰值被拉低  $\sim 17\%$  属正常（§4.5.2），色条范围与标注必须一致；再看是哪个量（von Mises 是标量判据， $\sigma_x$  有符号无危险性）。
- Q20:** 怎么把结果导出给其他软件？ A: 网格导出：输出路径用 `.inp` 后缀即写出 Abaqus 风格文件（§3.8，仅导出、不再导入）；结果数据在 `_fem2d_out/` 下的报告与云图；也可在 Python 里直接读 `result` 字典（`u/vm_stress/` 各应力分量）自行后处理。

B  术语表

| 术语    | 含义                                                      |
|-------|---------------------------------------------------------|
| CST   | 常应变三角形（3 节点线性单元）                                        |
| Q4    | 双线性四边形（4 节点， $2 \times 2$ 积分）                           |
| Q4R   | 单点积分四边形 + 沙漏稳定                                          |
| Q4I   | QM6 非协调四边形（静力凝聚）                                        |
| DOF   | 自由度；约定 $x \rightarrow 2n$ 、 $y \rightarrow 2n+1$        |
| B 矩阵  | 应变-位移阵， $\boldsymbol{\epsilon} = \mathbf{B} \mathbf{d}$ |
| 弱形式   | 虚功原理的加权积分形式（式 (2)）                                      |
| 消去法   | Bathe §4.2.2 约束自由度消去（默认）                                |
| 罚方法   | 对角元加罚刚度施加约束（近似）                                         |
| SPR   | 超收敛点恢复（节点补丁局部最小二乘）                                      |
| L2 投影 | 一致质量阵意义下的最小二乘应力投影                                       |
| Z2    | Zienkiewicz–Zhu 能量误差估计器（Bathe Eq. 4.108）                |
| 效应指数  | 误差随 $h$ 的衰减率 $\theta$ （线性单元 $\approx 1$ ）               |
| 牵引跳跃  | 内部边上的牵引不连续度量 $[\mathbf{t}]$                             |
| 刚体模态  | 零应变变形模式（2 平动 + 1 转动）                                    |

|       |                                                    |
|-------|----------------------------------------------------|
| 沙漏模式  | 单点积分的零能量变形模式                                       |
| 漂移门   | 6 个基线值逐位比对的数值回归门                                   |
| 金标准   | boundary_golden 段 schema + solve_refactor_lock 逐位锁 |
| 冻结区   | element/ 内核、solver 数值逻辑、error_est 公式、边界识别算法        |
| 判别性测试 | 放回旧实现必须失败的测试                                       |

## C 公式-教材对照表

本软件的每一条核心公式都能在教材中找到出处。对照表（教学与审查用）：

| 软件公式/机制               | Bathe 出处                   | 实现位置                   |
|-----------------------|----------------------------|------------------------|
| 平衡方程                  | Eq. 4.1                    | error_est 理论依据         |
| 几何方程（应变-位移）           | Eq. 4.2                    | element/* B 矩阵         |
| 平面应力本构 D              | Eq. 4.11                   | material.D_matrix()    |
| 平面应变本构 D              | Eq. 4.12                   | material.D_matrix()    |
| DOF 编号 $x=2i, y=2i+1$ | Eq. 4.18                   | 全局约定                   |
| 虚功原理（弱形式）             | Eq. 4.33                   | 理论基础                   |
| 单元刚度/载荷               | Eq. 4.30–4.31              | element/* + loads_core |
| 消去法约束施加               | §4.2.2, Eq. 4.40           | bc.apply_elimination() |
| 静力凝聚（Q4I）             | Eq. 4.65                   | element/q4i.py         |
| 等参积分                  | Eq. 5.39                   | element/q4.py          |
| CST 形函数               | §5.2.3                     | element/cst.py         |
| Q4 形函数                | §5.2.4                     | element/q4.py          |
| QM6 非协调模式             | §5.5.1                     | element/q4i.py         |
| 分片试验                  | §4.4.2                     | patch_test.py          |
| SPR 恢复                | §4.3.6                     | spr.py                 |
| Z2 能量误差估计器            | §4.3.6, Eq. 4.108          | error_est.py           |
| 应力不连续（牵引跳跃）           | §4.3.6, Eq. 4.107          | compute_traction_jumps |
| 等应力带法                 | §4.3.6 + Sussman–<br>Bathe | visualize.isoband      |
| 后向误差（残差检查）            | §8.2.6                     | solver.py              |

## D 项目历史大事记

从 3 月理论预研到 8 月封盘发布，项目走过了半年。3–5 月以理论学习与手算验证为主，6 月起原型成形，7 月进入当前代码库的正式迭代，8 月完成审计、性能与打包 (§8.8 独立审查历史可对照)。每个月都能在代码库或知识库中找到对应痕迹。

| 日期 | 轮次 | 内容与结论 |
|----|----|-------|
|----|----|-------|

|            |           |                                                                           |
|------------|-----------|---------------------------------------------------------------------------|
| 2026-03-18 | 理论预研      | Bathe《Finite Element Procedures》精读起步：弱形式、单元刚度、误差评估体系——§4 理论基础的知识起点        |
| 2026-03-28 | 手算验证      | 杆/梁单双单元手算算例与教材解析解对标，建立”理论 → 代码 → 验证”的核对习惯（后成为分片试验与解析解轮的原型）                |
| 2026-04-10 | 数学基础      | 变分原理与弱形式推导复习；单刚 → 总刚组装的稀疏存储笔记（后续五阶段管线的算法源头）                               |
| 2026-04-22 | CST 原型    | 单文件 CST 教学原型：分片试验通过，验证”几何 → 网格 → 组装 → 求解 → 后处理”闭环可行                       |
| 2026-05-12 | 求解器骨架     | .inp 手写解析器初版（CPS3 网格导入）+ 边界条件命令行雏形（fix/force），CLI 形态确立                    |
| 2026-05-30 | Q4 与 Gmsh | 协调四边形单元理论研究 + Gmsh 建模语法学习（.geo 脚本体系确立，§2.7 的来源）                           |
| 2026-06-10 | 可视化雏形     | matplotlib 单元色块云图 + 色条初版；数值结果”看得见”，后处理章（§10）铺路                            |
| 2026-06-28 | 边界识别雏形    | 网格拓扑自动识别原型（边界链与标签体系前身）；平面应力/平面应变体系梳理                                      |
| 2026-07-12 | 立项        | 2D 平面三角形单元求解器起步（本科毕设/保研项目）                                                |
| 2026-07-20 | Gmsh 集成   | 多格式网格导入贯通（.geo/.msh/.inp）；沉淀网格拓扑校验系列教训（孤立节点/重复单元/负索引）                     |
| 2026-07-24 | 审计轮       | CST 求解器多轮外部审计，28 项问题修复 + 7 项保留                                            |
| 2026-07-25 | 可视化与误差轮   | SPR 应力恢复、等应力带法（Bathe §4.3.6）、网格质量指标、应力跳跃度量接入；沉淀 9 条经验教训（知识库 fem/lessons/） |
| 2026-07-28 | 维护性评估     | 桌面版源码盘点：可维护性差、单文件堆积 → 启动大规模重构（8 月 A-E 轮的前奏）                               |
| 2026-08-02 | A/B/C     | 清理轮（死代码）+ 结构轮（长函数拆分）+ 覆盖轮（契约扩展）                                           |

|            |                |                                      |
|------------|----------------|--------------------------------------|
| 2026-08-03 | D/E            | 易用性轮（错误消息引导）+ 解析解轮（E1/E2/E3 理论对标）    |
| 2026-08-03 | 契约复查           | K1-K13 缺口闭环 + 探针 151 项断言 + fuzz 500  |
| 2026-08-05 | R 审查           | R1-R7 七维度外部审查, P0/P1 缺陷归零, 评分 8.4/10 |
| 2026-08-06 | P 轮 I/II       | 性能七包（K·u 向量化/缓存/面力批量化等）+ CI 性能门      |
| 2026-08-07 | S- $\alpha$ 安全 | 表达式 DoS 修复、复数拒绝、打包卫生；封盘清理，维护模式       |
| 2026-08-07 | 便携包            | 免安装 Windows 便携包验证通过（模拟新电脑全流程）        |

E 参考文献

1. K.-J. Bathe. *Finite Element Procedures*, 2nd ed. Prentice Hall, 1996.（核心教材；本文公式编号（§4.2.2、§4.3.6、§5.2、§5.5、Eq. 4.18/4.30/4.107/4.108 等）均对应此书）

2. T. Sussman, K.-J. Bathe. “Studies of finite element procedures – stress band plots”, *Computers & Structures*, 1986.（等应力带法）

3. O. C. Zienkiewicz, J. Z. Zhu. “A simple error estimator and adaptive procedure for practical engineering analysis”, *IJNME*, 1987.（Z2 估计器）

4. R. L. Taylor et al. QM6 非协调单元, *Int. J. Num. Meth. Eng.*, 1976.（Q4I 理论基础）

5. S. P. Timoshenko, J. N. Goodier. *Theory of Elasticity*, 3rd ed.（E1 梁理论、E2 Kirsch 解出处）

6. Kirsch (1898). 无限板圆孔应力集中。（E2 解析解）